

Signal Trees

A Deep Dive into a High-Performance Alternative to Queue-Based Task Scheduling

Michael Maniscalco

Signal Trees

A Deep Dive into a High Performance Alternative to Queue-Based Task Scheduling

- Concurrency
- Async programming
- Networking
- Coroutines
- Game engines
- Task Graphs
- Real-time computing
- Finance
- Executors



The Goal Of This Talk

Why we need a scheduler?

- Concurrent work means multiple things want to run, but only one can run at a time in any given context
- Something has to decide - “what runs next?”
- That something is a scheduler

The Goal Of This Talk

Number of Threads is Irrelevant

- Even a single-threaded event loop has this problem:
 - There are N ready coroutines
 - There are N ready callbacks
 - There are N ready sockets
 - An Async I/O event has occurred
 - The mouse has been moved
- The scheduler makes the decision on what to do next
- The question is purely about *readiness* and *selection*, not about parallelism

The Goal Of This Talk

Queues As Schedulers

- The conventional solution for a scheduler is a queue
- But a queue conflates two separate concerns:
 - readiness signaling
 - data movement
- The problem:
 - Queues are data primitives answering "what is the next item?"
 - But scheduling is about "who goes next?"

The Goal Of This Talk

But *"What is the next item?"* and *"Who goes next?"* are different questions

Queues couple:

- Data transport
- FIFO readiness coordination

Signal Trees only:

- Preserve readiness state
- Provide fair readiness selection

Result:

- Less coordination around shared FIFO state
- High-throughput fairness-oriented selection under contention

Three Properties of an Ideal Scheduler - *And Why It Matters*

Throughput:

Signal scheduling is on the critical path of every task system. How fast can ready signals be identified and selected?

Scalability:

Modern hardware has many cores. Adding threads should add performance. How closely does throughput scale in relation to thread count?

Fairness:

Starvation is silent and catastrophic in production systems. Does every ready signal get serviced reliably and with equal priority?

Schedulers: Where We Are Today

Before introducing Signal Trees, let's look at the conventional answer: queues

- Queues are widely used because they are:
 - Simple
 - Familiar
 - Effective
- The question is not whether queues are useful. The question is:
 - What happens when we ask them to be schedulers under contention?
 - What happens if our scheduler is required to provide more nuanced orchestration than just FIFO?

Schedulers: Where We Are Today

Why Queues Are Attractive:

- Simple producer / consumer model
- Preserve FIFO ordering
- Transfer ownership of work
- Easy to reason about
- Supported by mature implementations

Schedulers: Where We Are Today

Queue-Based Scheduling Model:

- Ready work is pushed into a queue
- Workers pop the next available item
- The queue becomes both:
 - The transport mechanism
 - The scheduling decision point
- This is a reasonable design
 - But it concentrates coordination on the head/tail
 - Serializes access to the elements in the queue

Part 1

Queues As Schedulers

“When in doubt, use a queue.”

- Someone Smart, Somewhere

Queues As Schedulers - What The Industry Uses

These are all fundamentally queue-centered schedulers:

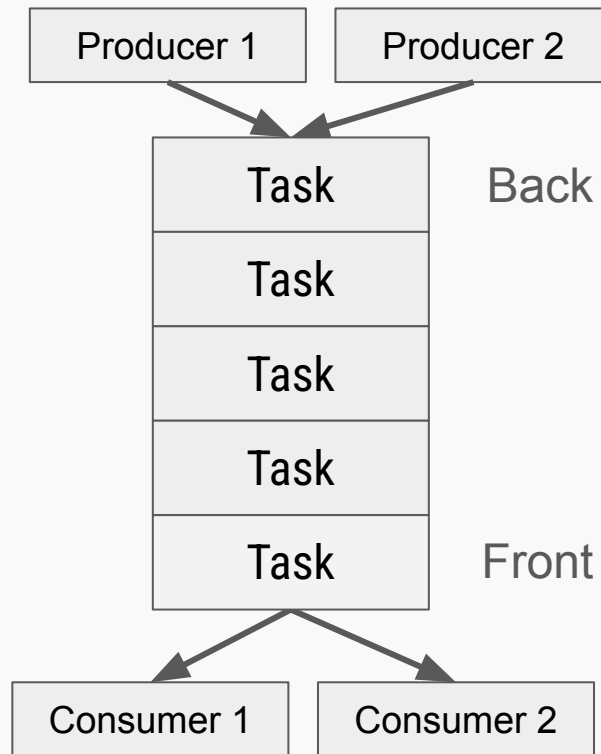
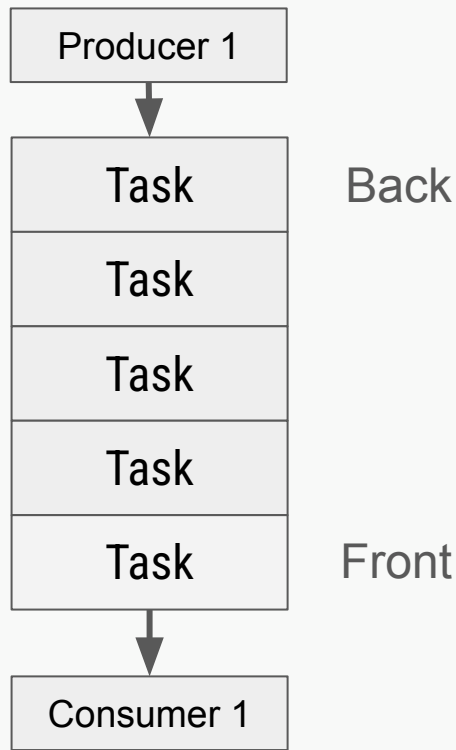
- mutex queue (`std::queue` with `std::mutex`)
- lock-free queue (TBB, Boost LockFree)
- sharded queue (Moody Camel)
- work stealing
- per-thread queues (SPSC etc)

Queues As Schedulers - What The Industry Uses

Queues naturally provide:

- Ordering (FIFO)
- Ownership Transfer
- Buffering
- Work Distribution
- A Simple Mental Model

Queues As Schedulers - Queue Scheduling Model



Benchmark

Queues - Throughput

Queues As Schedulers - Benchmarking Throughput

Set up:

- A pool of N signals, K of which are active at any time
- A single worker thread continuously selects a signal, then immediately replaces it with a new one from the inactive pool
- The inactive pool is kept in randomized order so signal ids are unpredictable
- When the test ends, total signals processed divided by elapsed time gives throughput in signals per second

Queues As Schedulers - Benchmarking Throughput

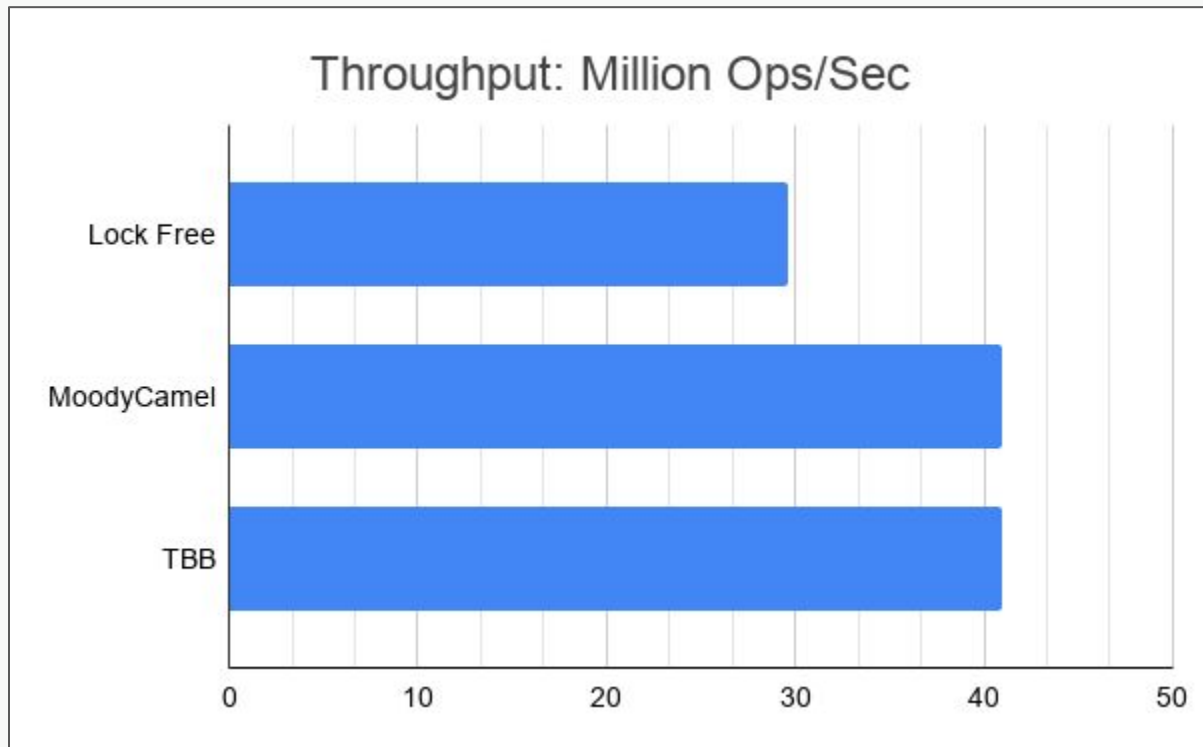
Parameters:

- $N = 16,384$ signals
- $K = 1,024$ signals
- Test Duration = 10 seconds

Queue Implementations Tested:

- Boost LockFree
- TBB concurrent queue
- Moody Camel concurrent queue

Queues As Schedulers - Benchmarking Throughput



Benchmark

Queues - Scalability

Queues As Schedulers - Benchmarking Scalability

Set up:

- A pool of N signals, K of which are active at any time
- The inactive pool is kept in randomized order so signal ids are unpredictable
- Worker threads continuously select a signal, then immediately replace it with a new one from the inactive pool
- When the test ends, total signals processed divided by elapsed time gives throughput in signals per second

Queues As Schedulers - Benchmarking Scalability

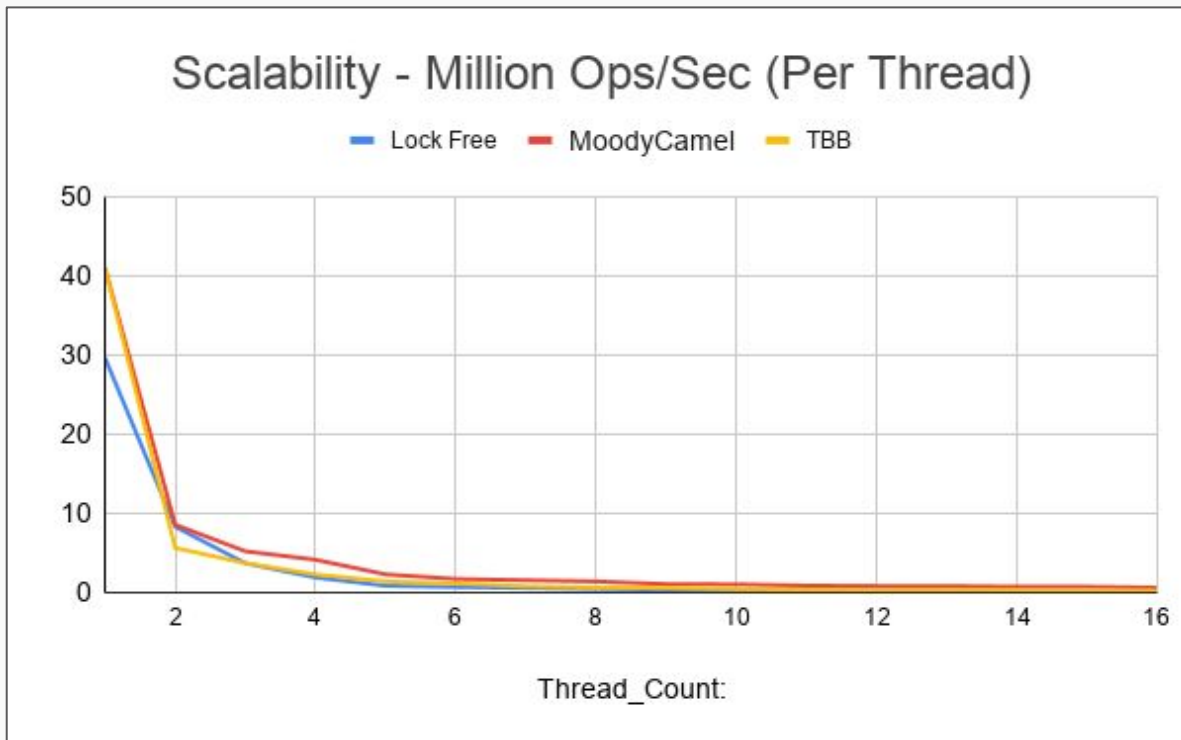
Parameters:

- $N = 16,384$ signals
- $K = 1,024$ signals
- Test Duration = 10 seconds
- Threads = 2–16

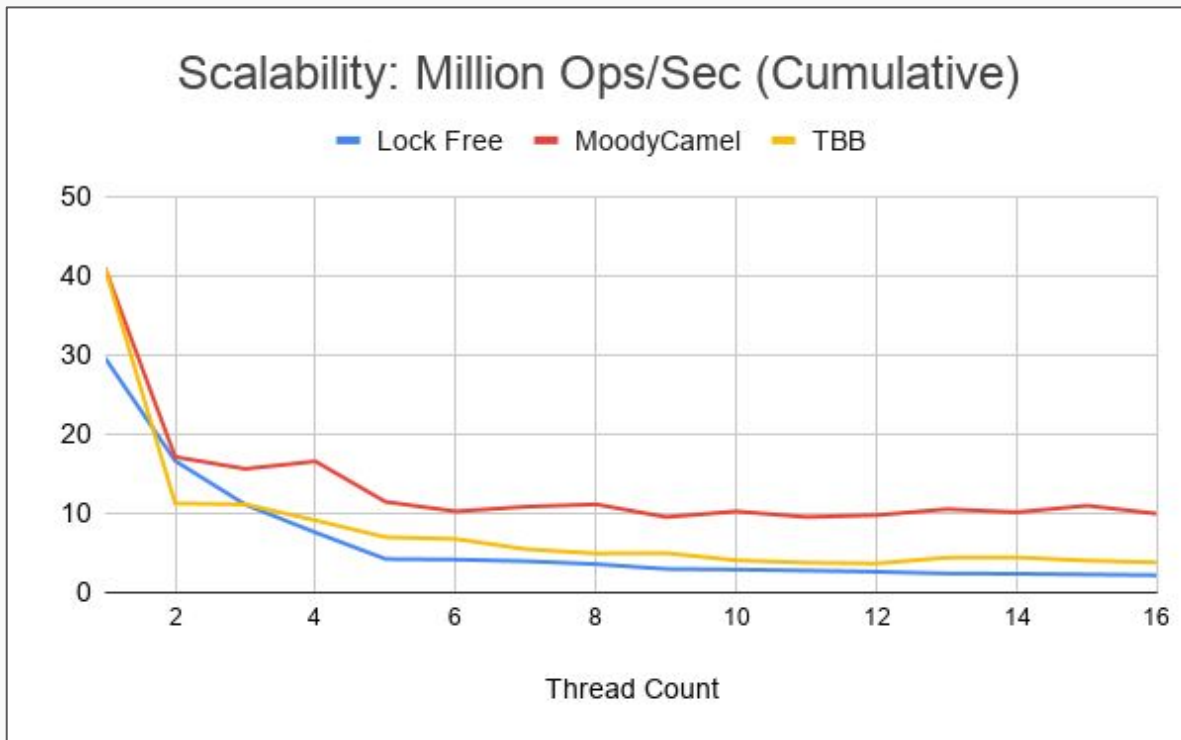
Queue Implementations Tested:

- Boost LockFree
- TBB concurrent queue
- Moody Camel concurrent queue

Queues As Schedulers - Benchmarking Scalability



Queues As Schedulers - Benchmarking Scalability



Benchmark

Queues - Fairness

Queues As Schedulers - Benchmarking Fairness

Set up:

- Signal Tree with capacity N
- Queues with capacity N
- All signals $[0 \rightarrow N - 1)$ placed into vector and order randomized
- A subset K of the N signals in the vector are removed and set/enqueued
- $[1 \rightarrow 16]$ worker threads are spawned to select/dequeue a signal
- The selected signal is placed at the back of the randomized vector
- A new signal is removed from the randomized vector and set/enqueued

Queues As Schedulers - Benchmarking Fairness

Set up:

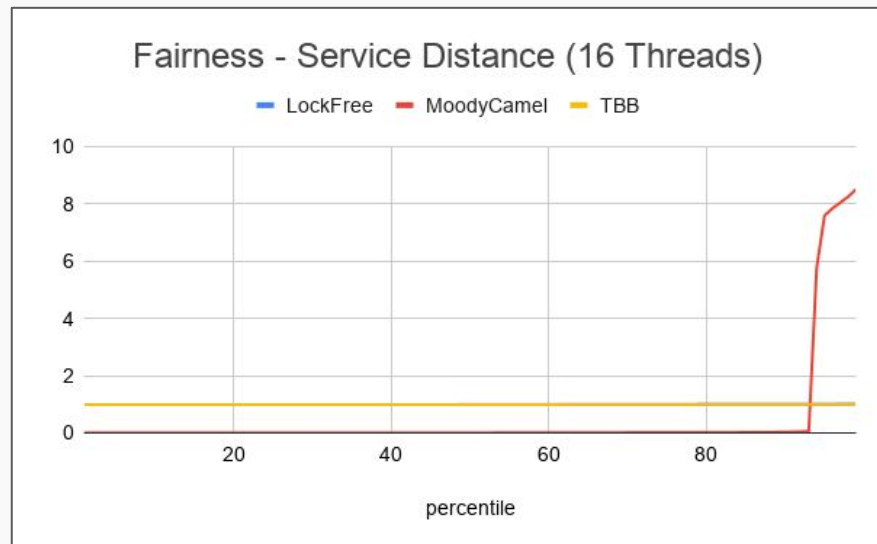
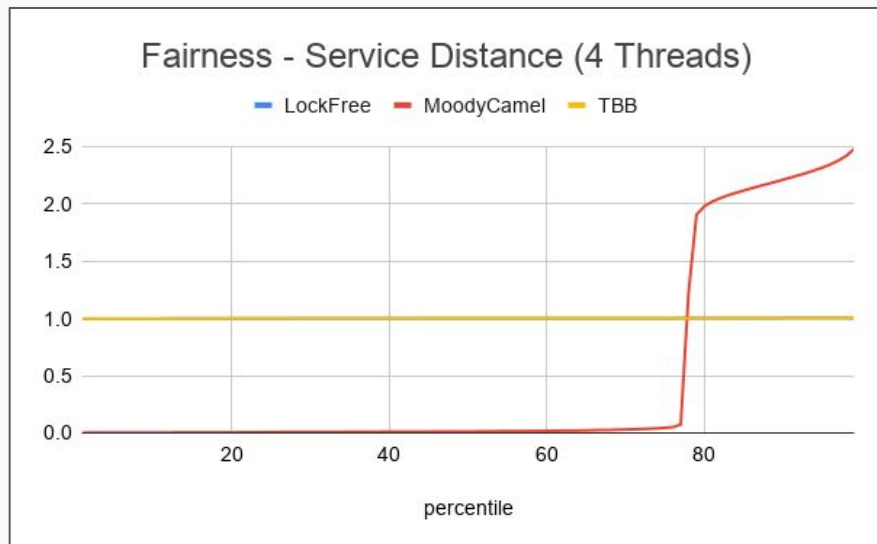
- With each signal selected the 'service distance' is calculated
- Service distance is the number of selections made between the signal being set and the time at which it was selected.
- Percentiles are plotted for the Service Distance samples
- A Service Distance of K indicates perfect fairness

Queues As Schedulers - Benchmarking Fairness

Parameters:

- For this test $N = 2^{14} = 16,384$ total signals
- For this test $K = 2^{10} = 1,024$ active signals
- All Service Distance Values are divided by K
- A service distance of 1.0 indicates perfect fairness
- Test Duration = 10 seconds
- Threads: $[1 \rightarrow 16]$

Queues As Schedulers - Benchmarking Fairness



Benchmark

Summing Up Results

Queues As Schedulers - Considering Benchmark Results

What The Queue Benchmarks Show:

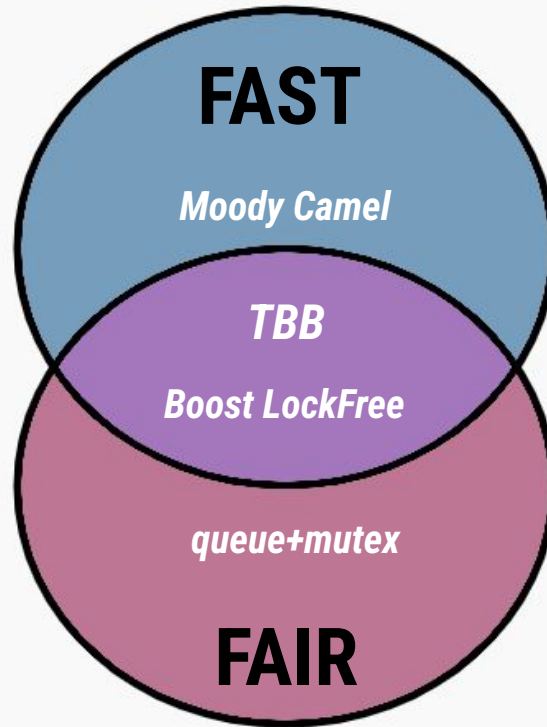
- Queues are effective scheduling primitives at low contention
- Different implementations choose different tradeoffs between throughput and fairness
- As thread count rises, shared queue state becomes the limiting factor
- Push/pop coordination forces threads to repeatedly contend on the same hot atomics and cache lines

Queues As Schedulers - Considering Benchmark Results

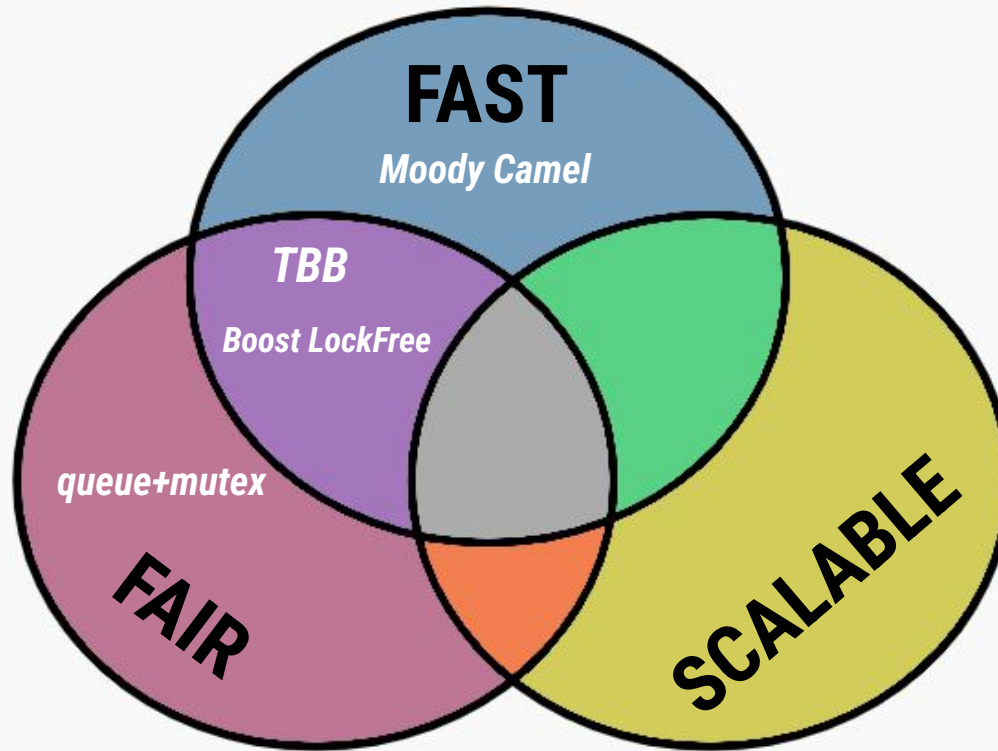
Summary:

The problem is not that queues are bad. The problem is that queue-based scheduling concentrates readiness, ordering, and coordination of data all in the same structure.

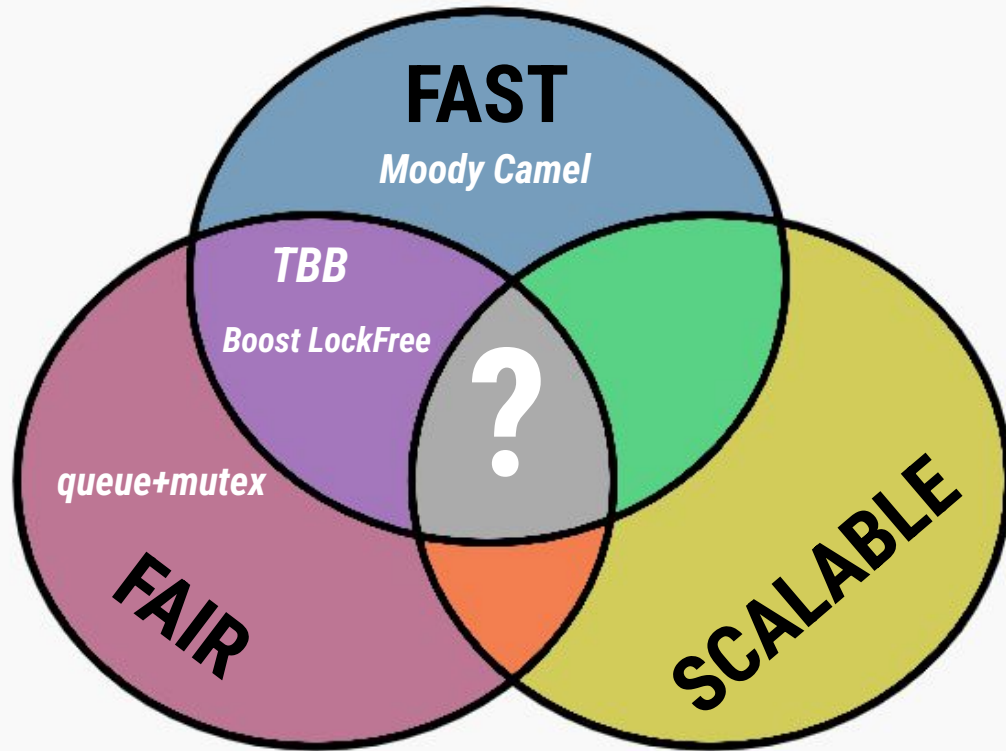
Queues As Schedulers - Considering Benchmark Results



Queues As Schedulers - Considering Benchmark Results



Queues As Schedulers - Considering Benchmark Results



Part 2

Signal Trees

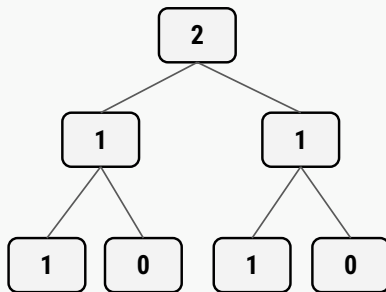
*“The difficulty lies not so much in developing new ideas
as in escaping from old ones.”*

- John Maynard Keynes

Signal Tree - An Introduction

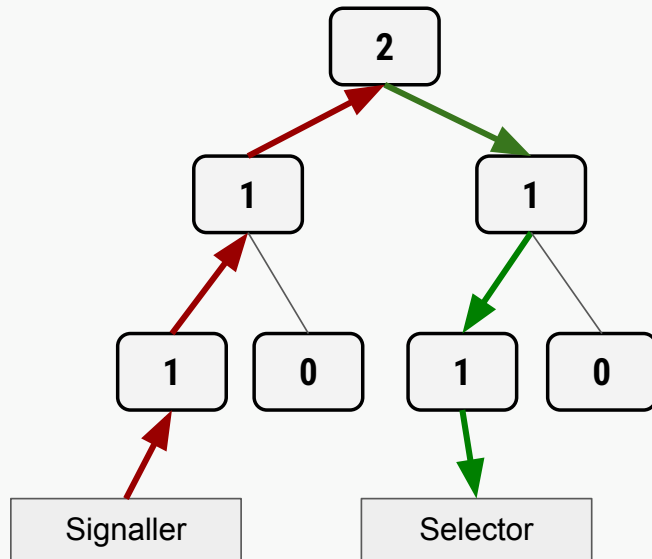
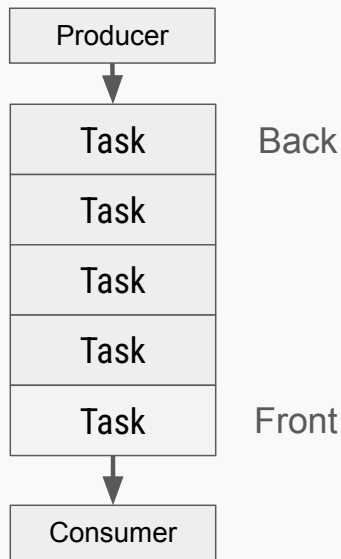
What is a Signal Tree?

- A Signal Tree is a readiness-selection structure.
- It does not transport work, nor does it preserve FIFO ordering.
- It preserves only the readiness state required to answer: "Who is ready?"
- Scheduling and data transport are intentionally separated



Signal Tree - As Schedulers

Signal Trees schedule readiness directly instead of routing work through a shared queue.



Signal Tree - As Schedulers

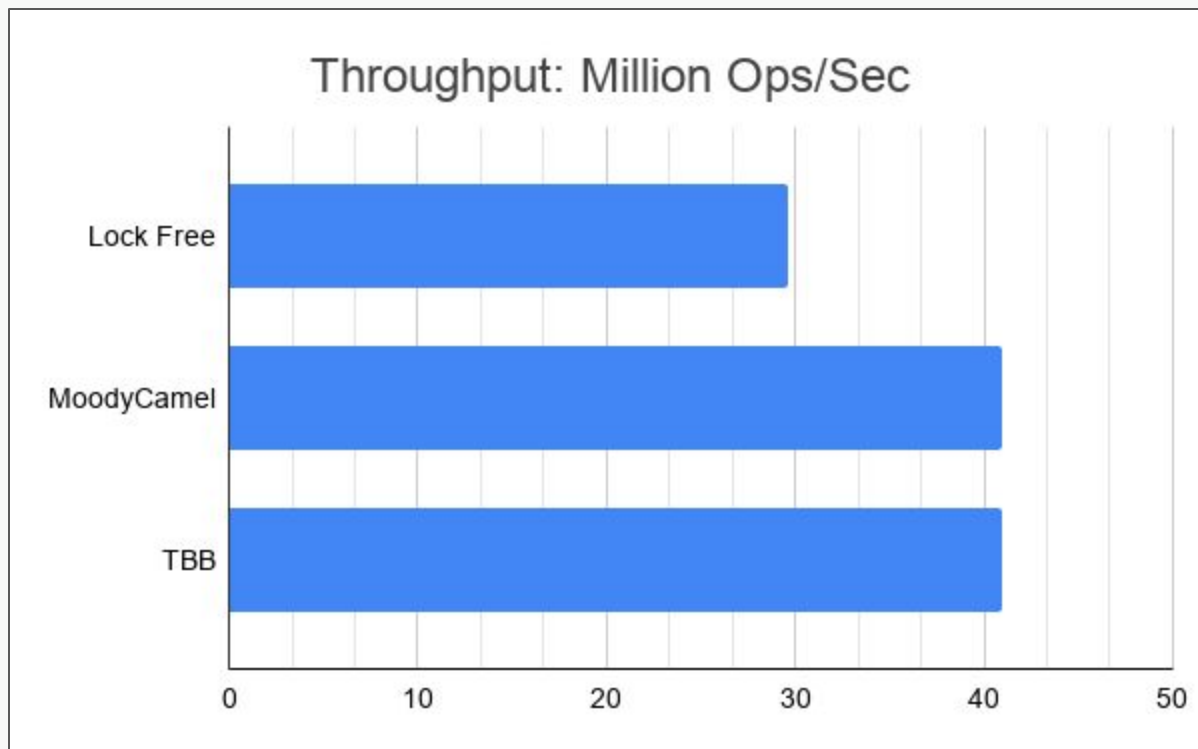
The Core Properties Of A Signal Tree:

- Lock-free selection
- Wait-free signal publication
- Fair readiness selection
- Distributed readiness state
- Scheduling independent of data transport
- Not FIFO

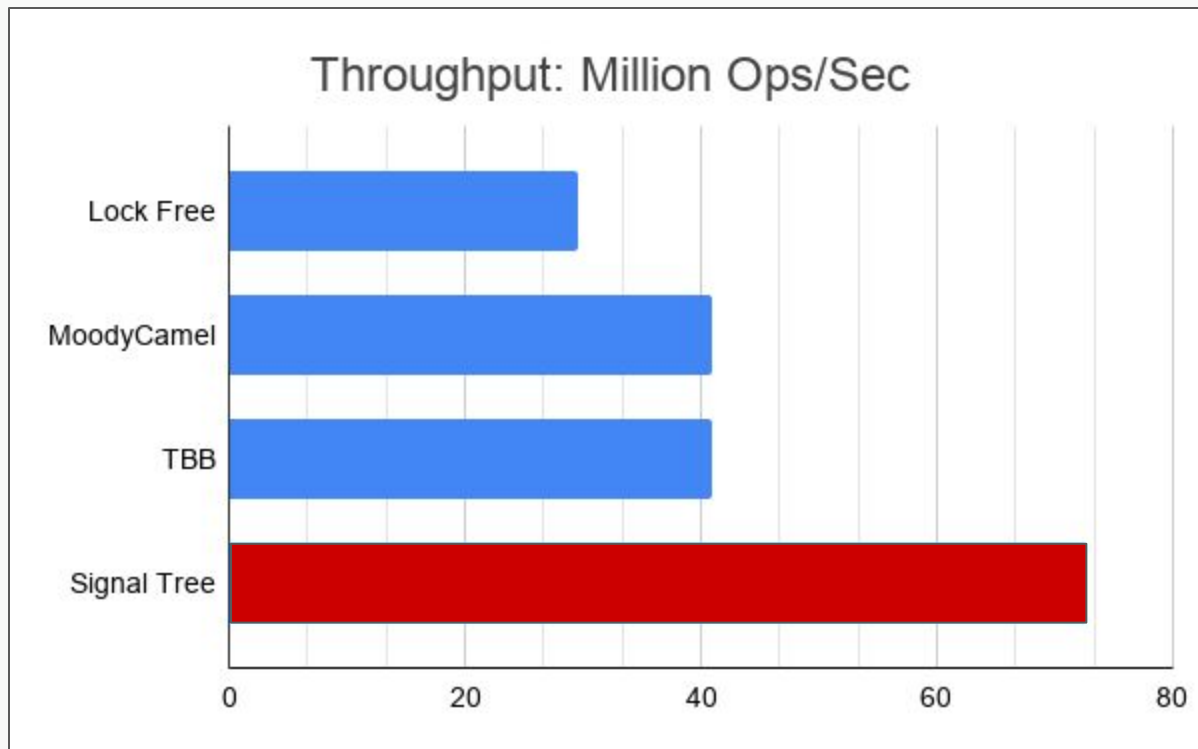
Benchmarks

Revisited ...

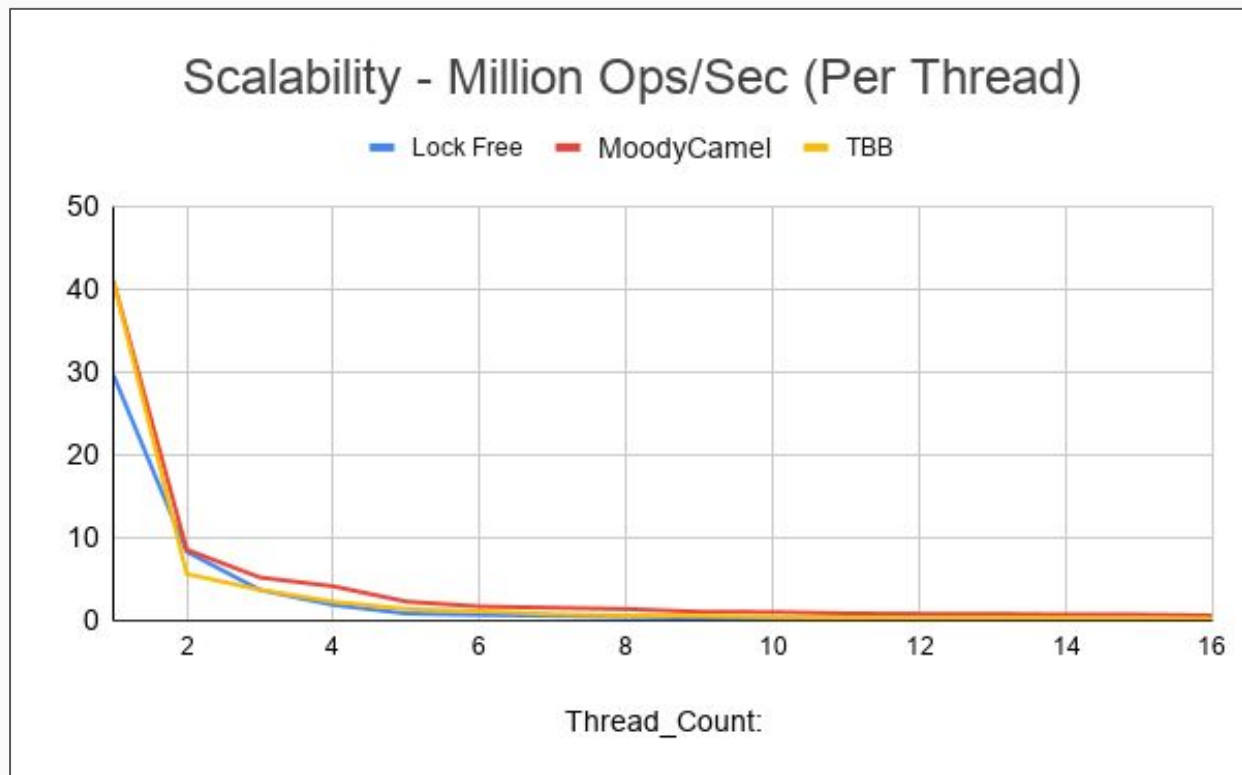
Signal Trees - Revisiting Throughput



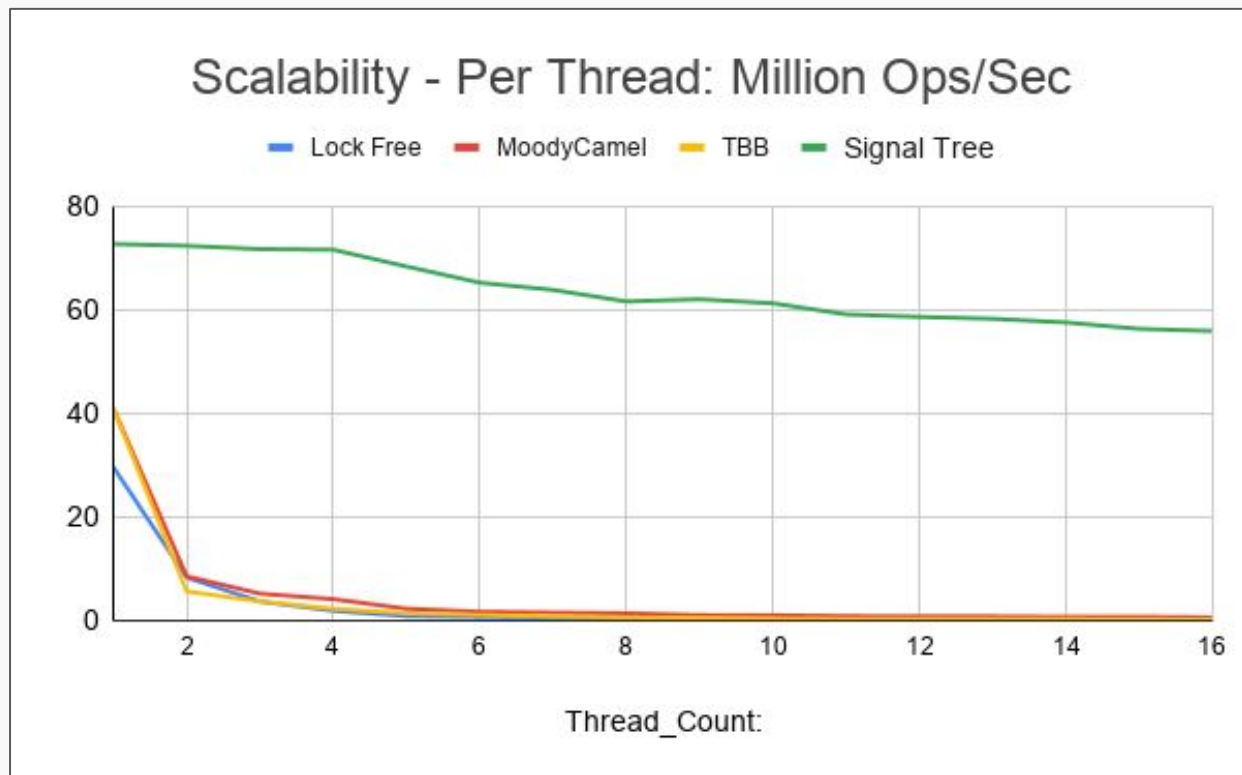
Signal Trees - Revisiting Throughput



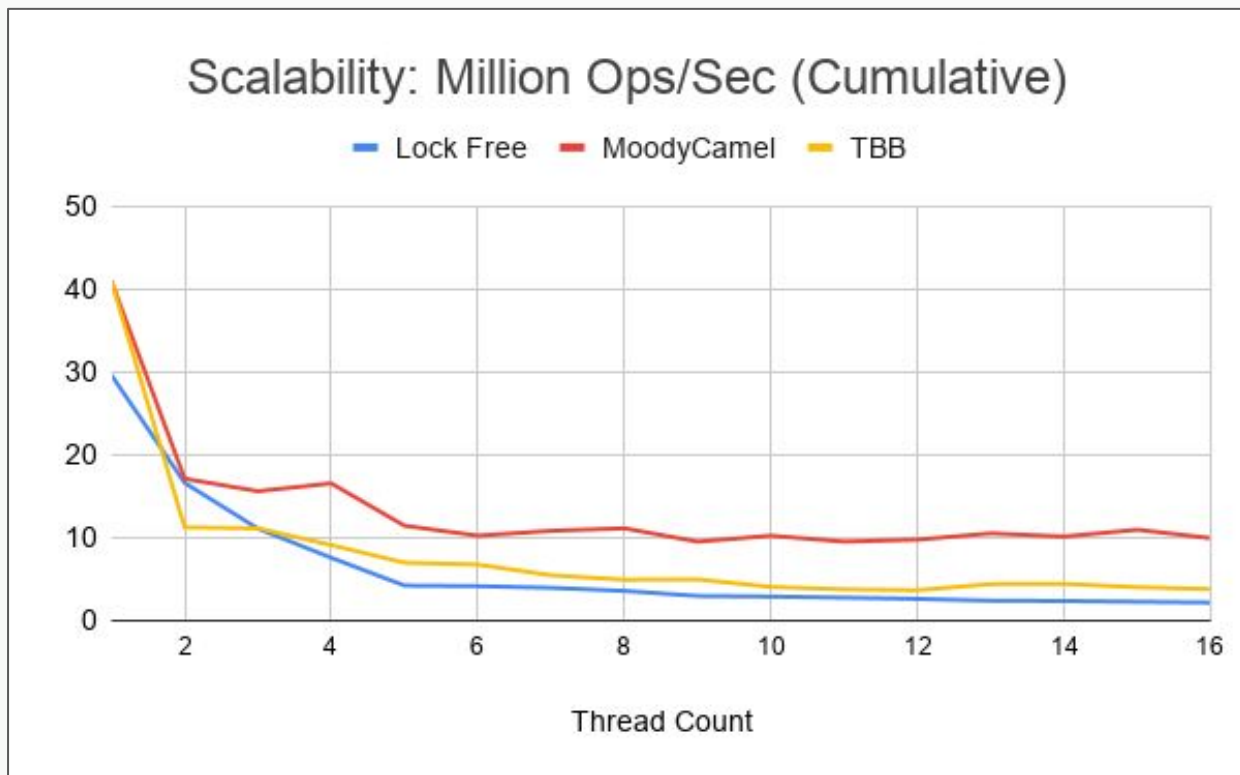
Signal Trees - Revisiting Scalability



Signal Trees - Revisiting Scalability



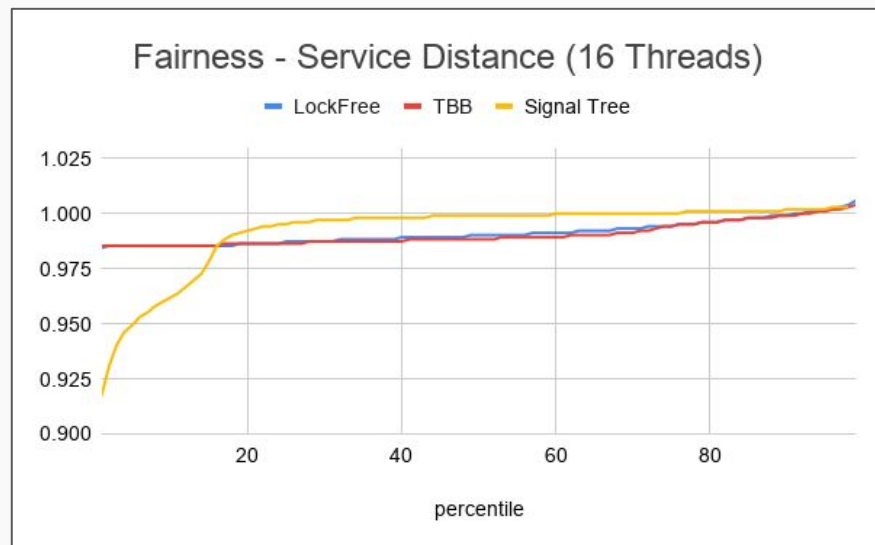
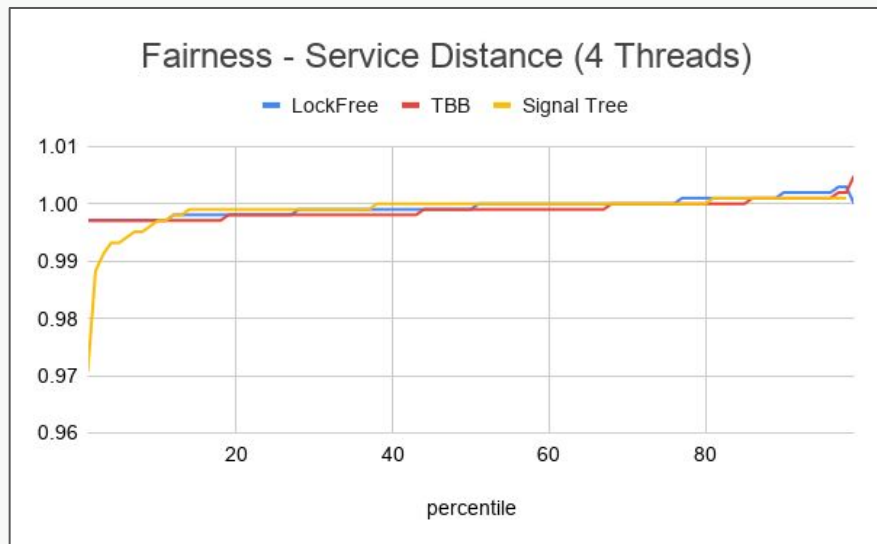
Signal Trees - Revisiting Scalability



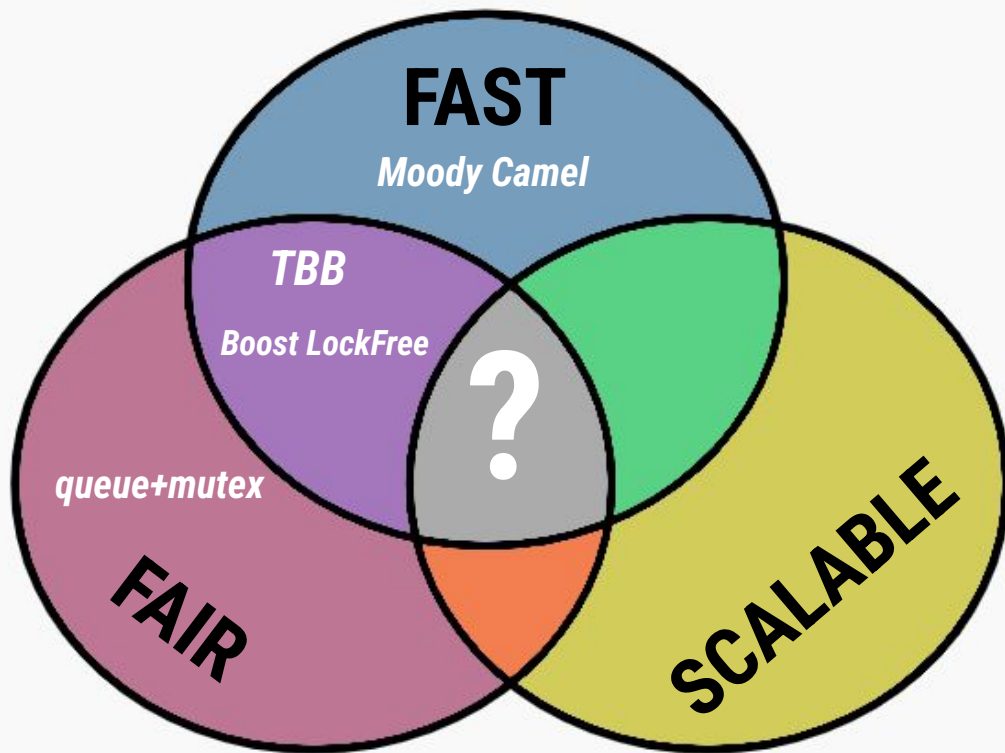
Signal Trees - Revisiting Scalability



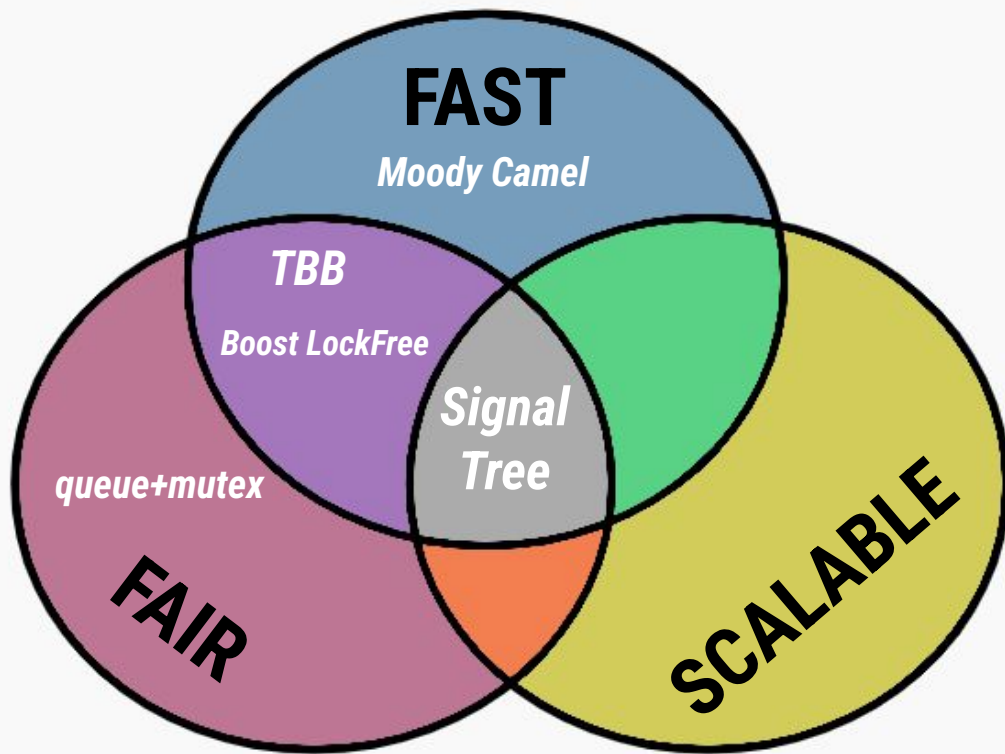
Signal Trees - Revisiting Fairness



Signal Trees - Where Does It Fit In?



Signal Trees - Where Does It Fit In?



Where Is This Improvement Coming From?

Part 3

Signal Trees - Structure and Invariants

The Properties of a Signal Tree:

Perfect Binary Tree:

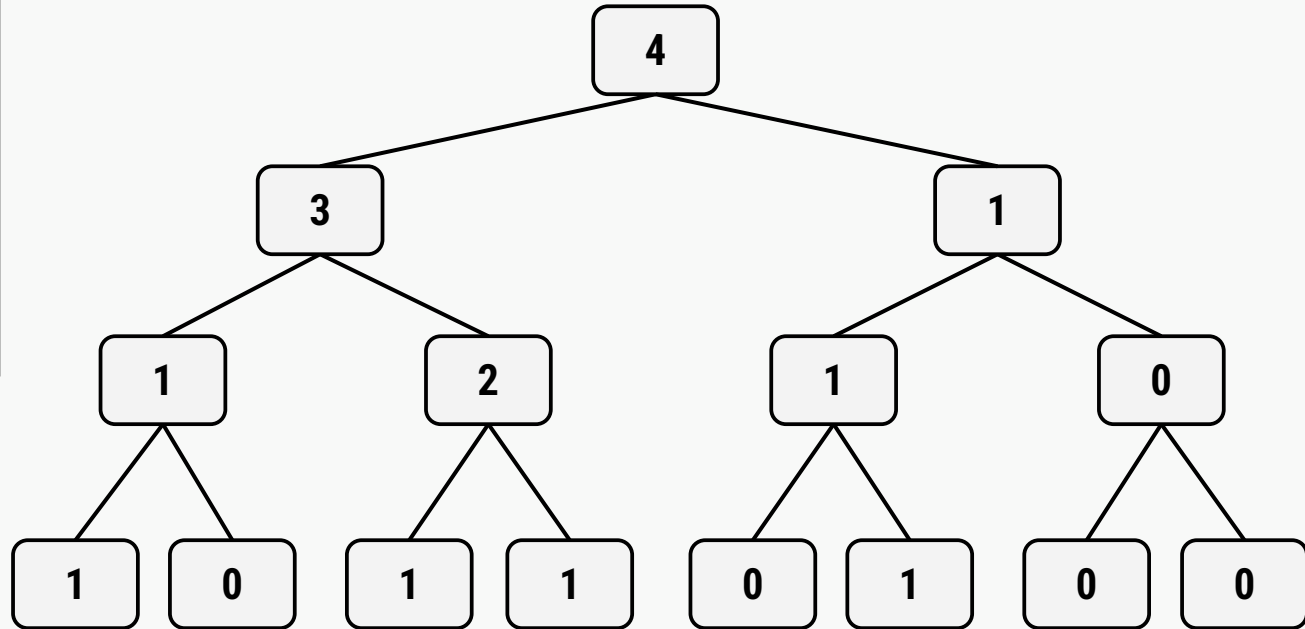
- Internal nodes have two children
- Leaf nodes all at same level

Leaf Nodes Represent Signals:

- Zero indicates reset (off)
- One indicates set (on)

Child Sum Property:

- Internal nodes are sum of children



The Properties of a Signal Tree:

Perfect Binary Tree:

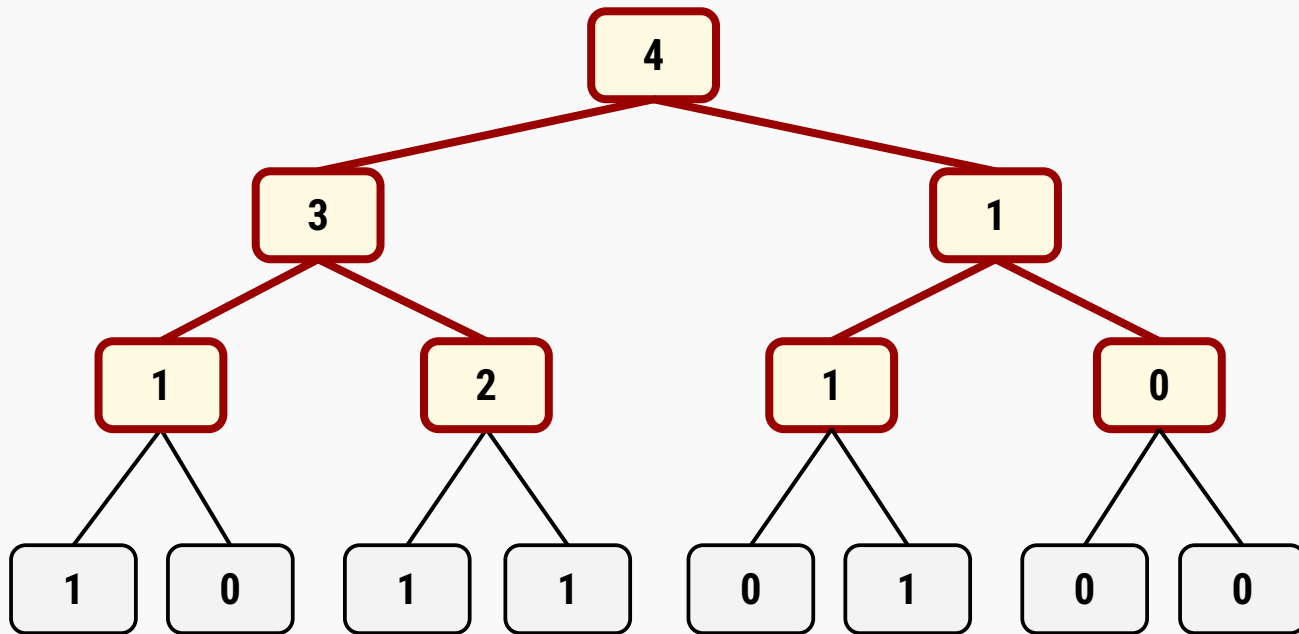
- **Internal nodes have two children**
- Leaf nodes all at same level

Leaf Nodes Represent Signals:

- Zero indicates reset (off)
- One indicates set (on)

Child Sum Property:

- Internal nodes are sum of children



The Properties of a Signal Tree:

Perfect Binary Tree:

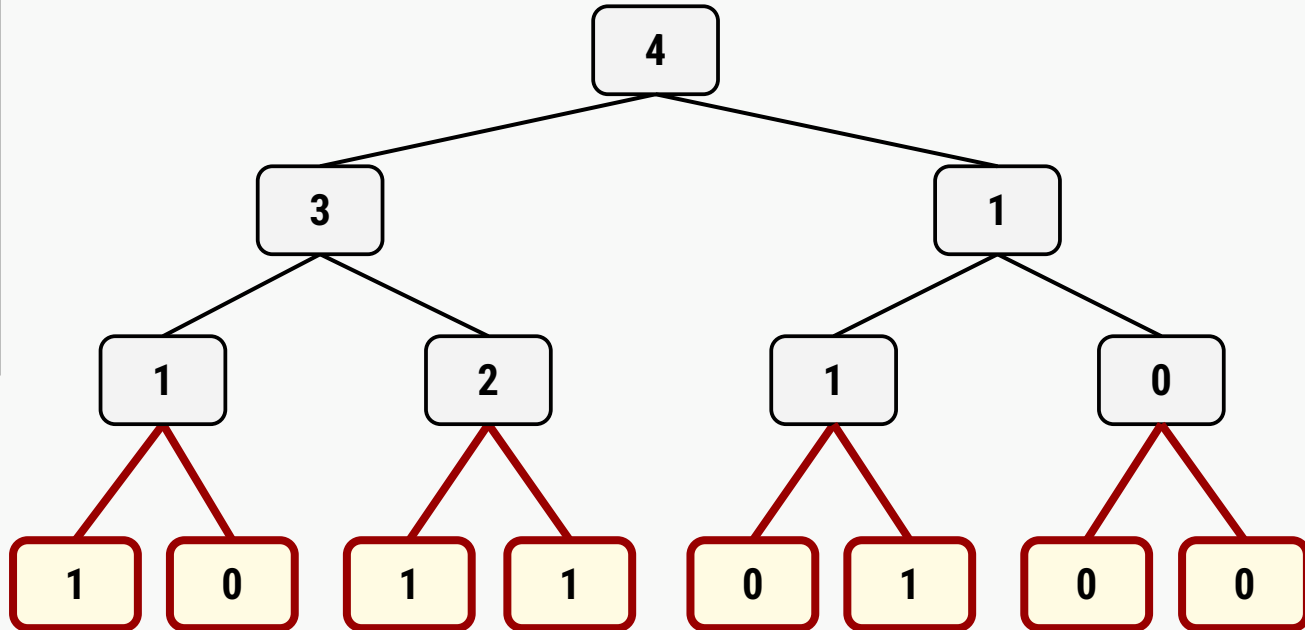
- Internal nodes have two children
- **Leaf nodes all at same level**

Leaf Nodes Represent Signals:

- Zero indicates reset (off)
- One indicates set (on)

Child Sum Property:

- Internal nodes are sum of children



The Properties of a Signal Tree:

Perfect Binary Tree:

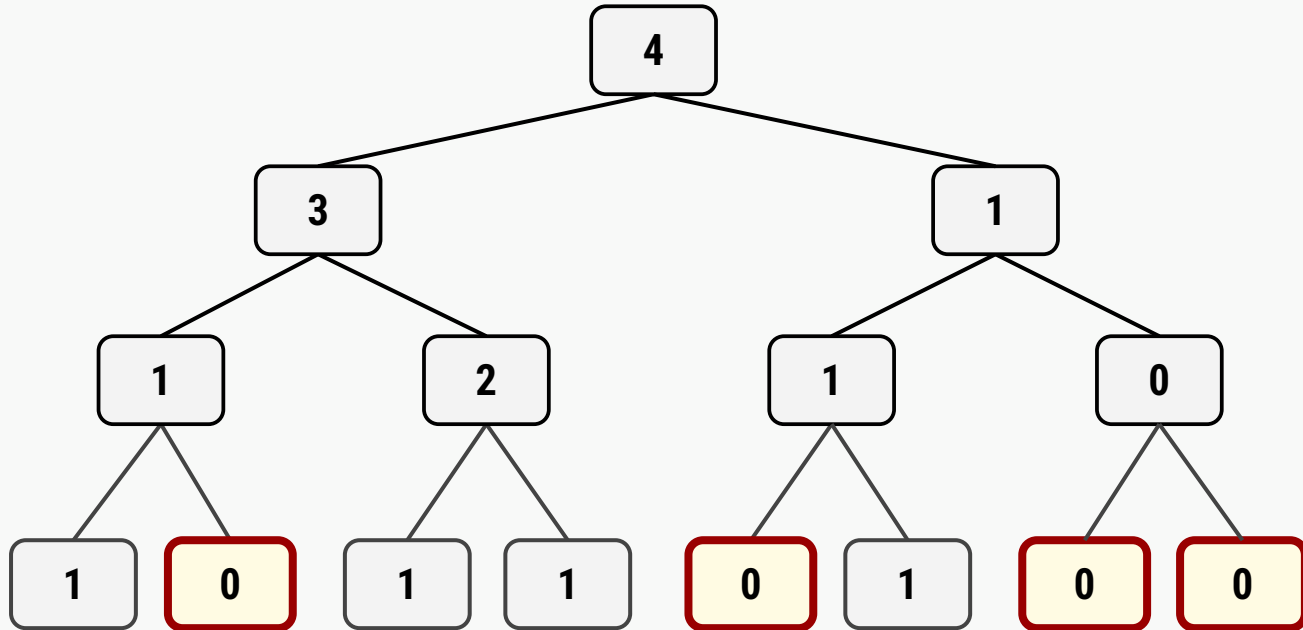
- Internal nodes have two children
- Leaf nodes all at same level

Leaf Nodes Represent Signals:

- **Zero indicates reset (off)**
- One indicates set (on)

Child Sum Property:

- Internal nodes are sum of children



The Properties of a Signal Tree:

Perfect Binary Tree:

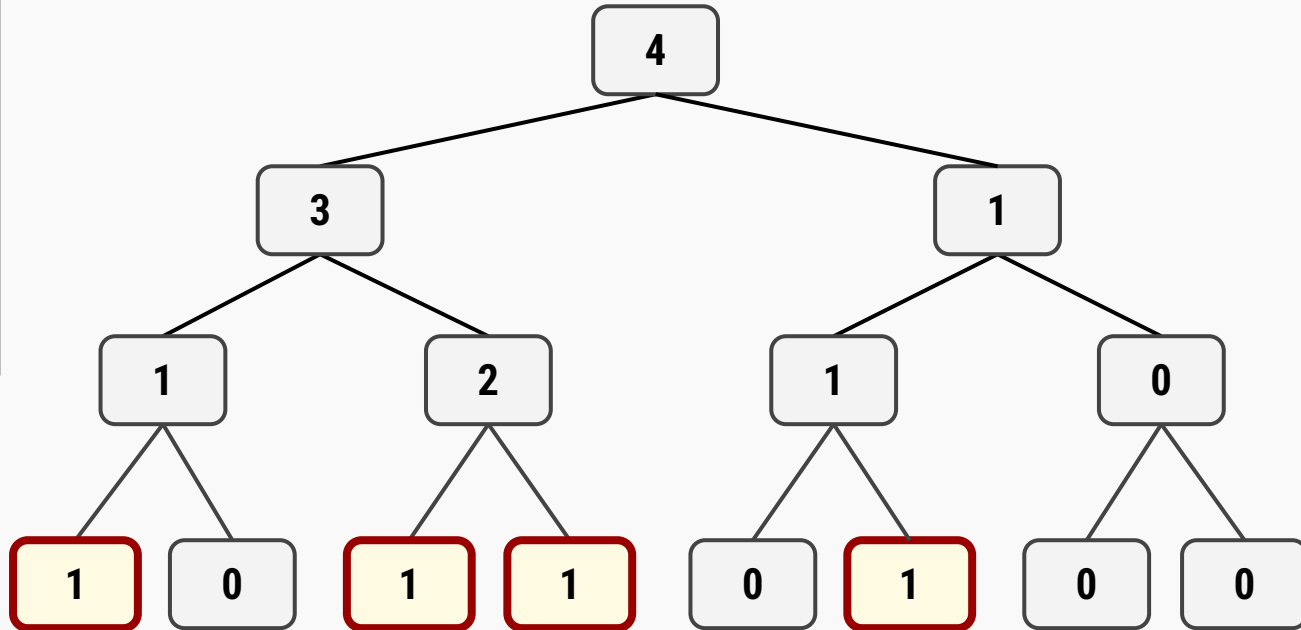
- Internal nodes have two children
- Leaf nodes all at same level

Leaf Nodes Represent Signals:

- Zero indicates reset (off)
- **One indicates set (on)**

Child Sum Property:

- Internal nodes are sum of children



The Properties of a Signal Tree

Perfect Binary Tree:

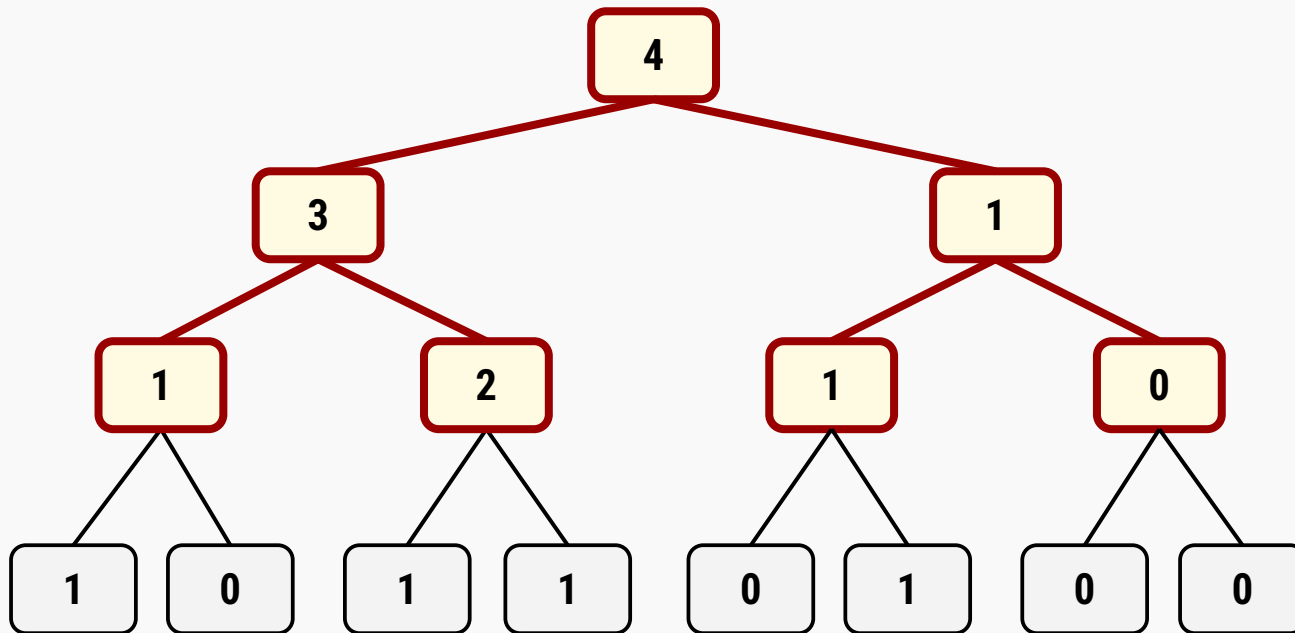
- Internal nodes have two children
- Leaf nodes all at same level

Leaf Nodes Represent Signals:

- Zero indicates reset (off)
- One indicates set (on)

Child Sum Property:

- Internal nodes are sum of children



The Properties of a Signal Tree:

Perfect Binary Tree:

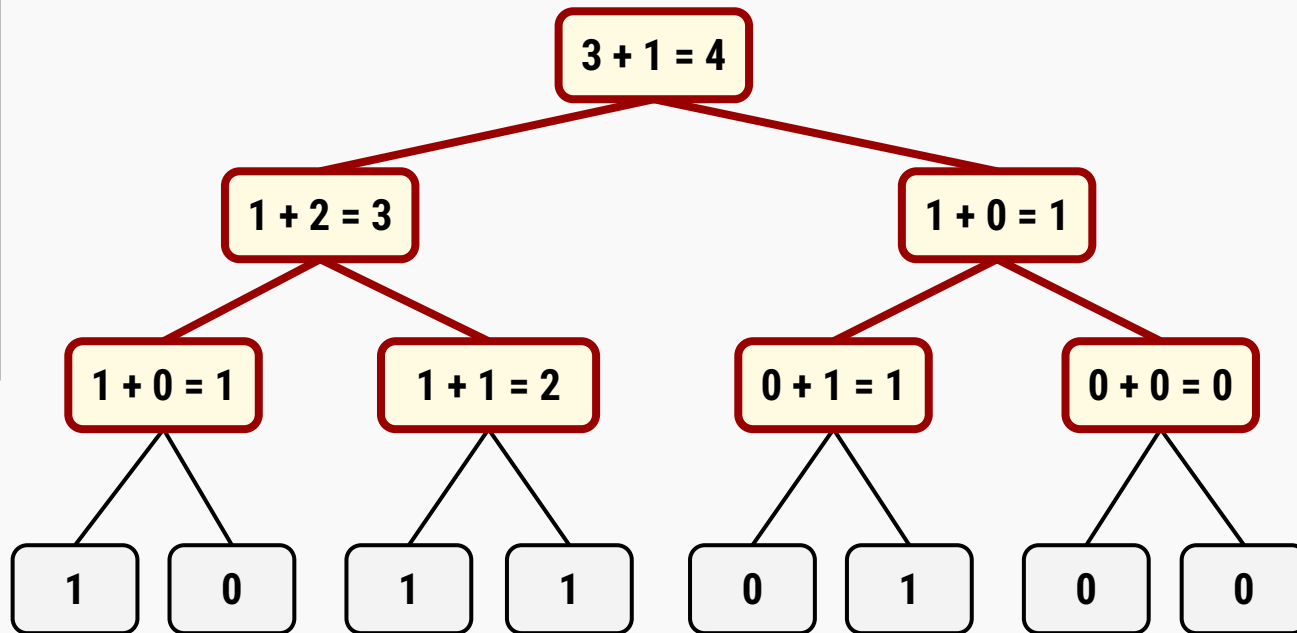
- Internal nodes have two children
- Leaf nodes all at same level

Leaf Nodes Represent Signals:

- Zero indicates reset (off)
- One indicates set (on)

Child Sum Property:

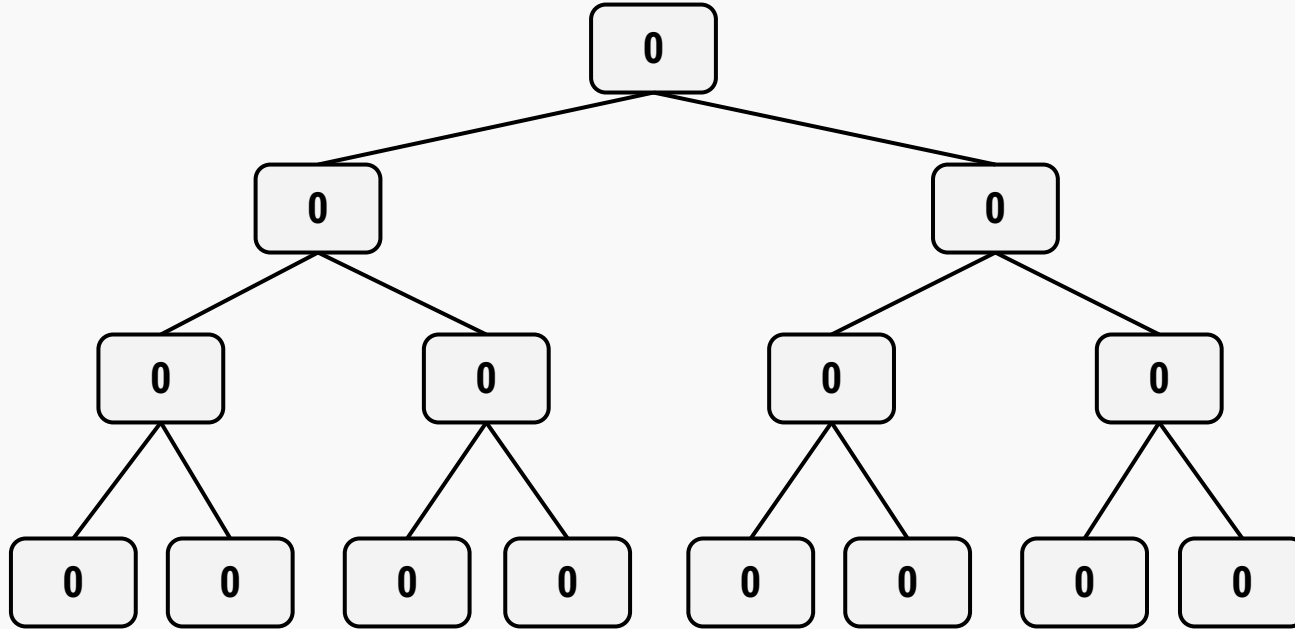
- Internal nodes are sum of children



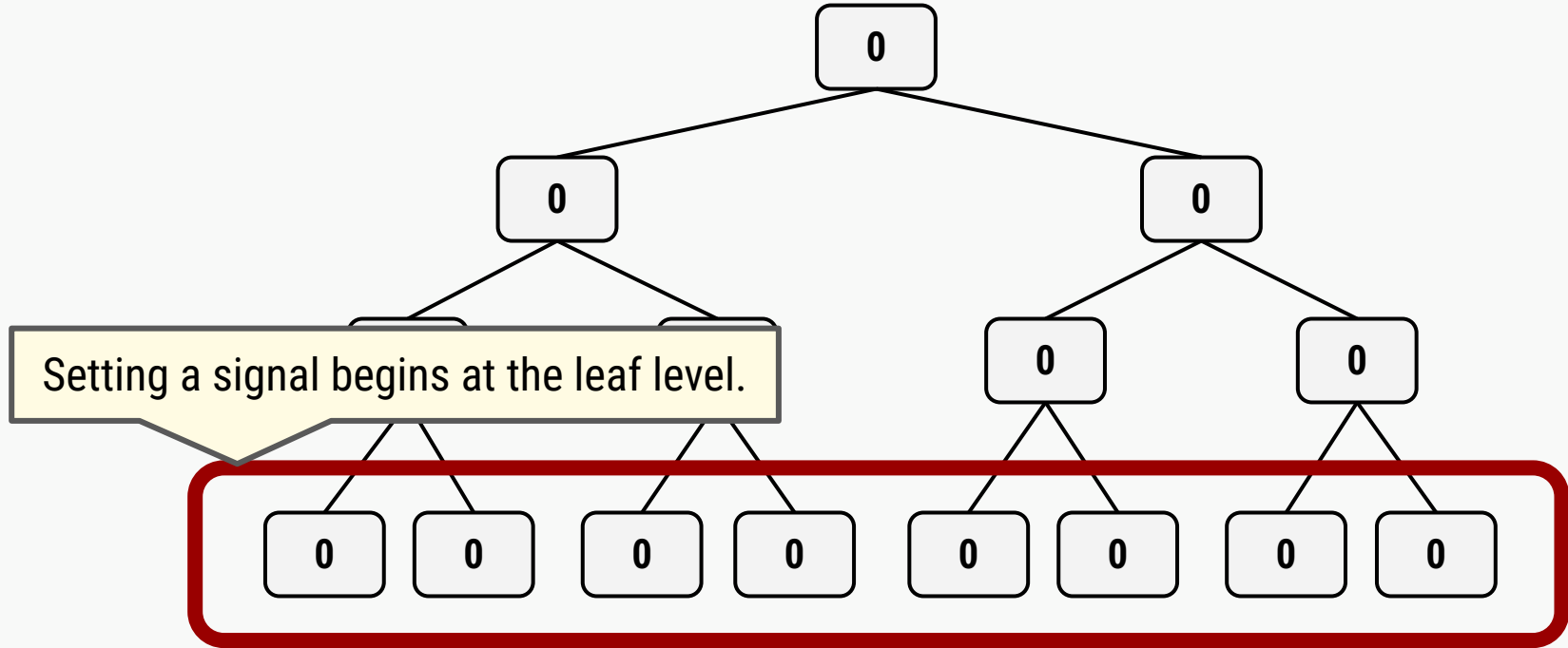
Setting Signals

Indicating “ready state”

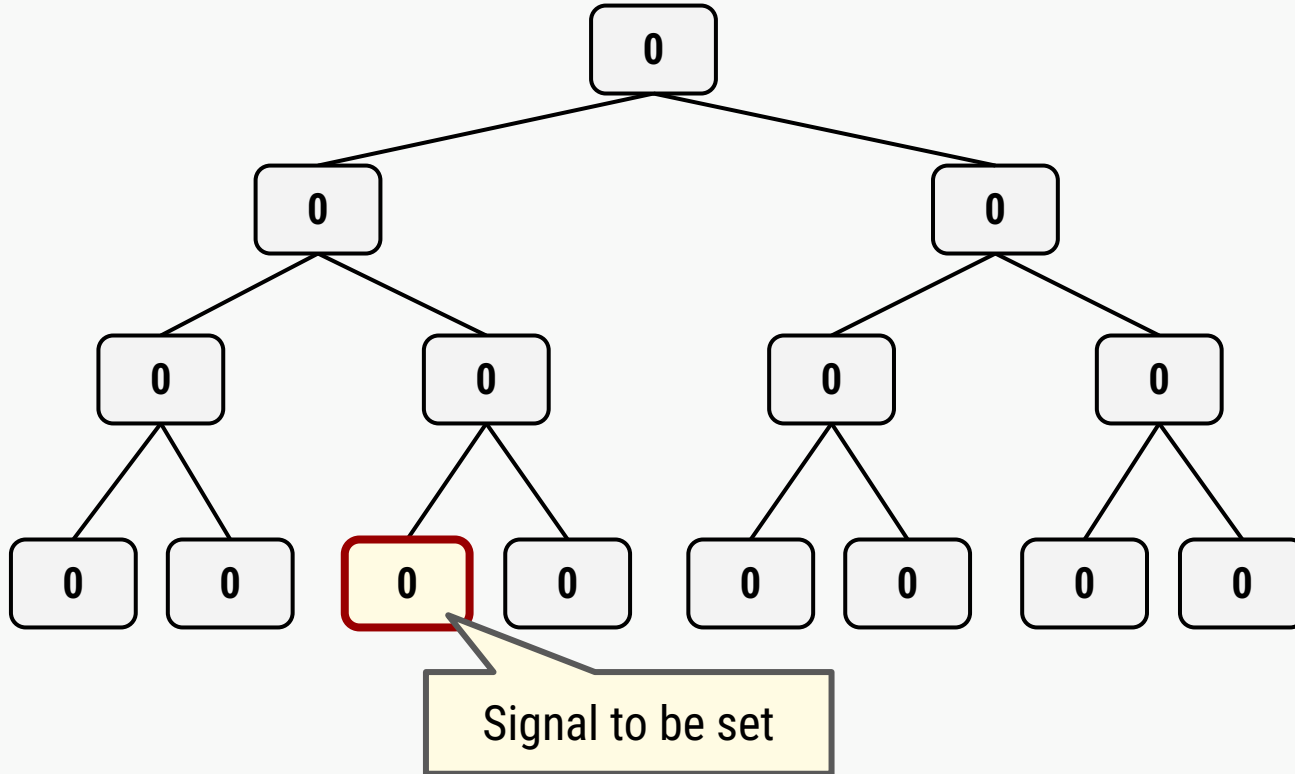
Setting A Signal: Signal Propagation



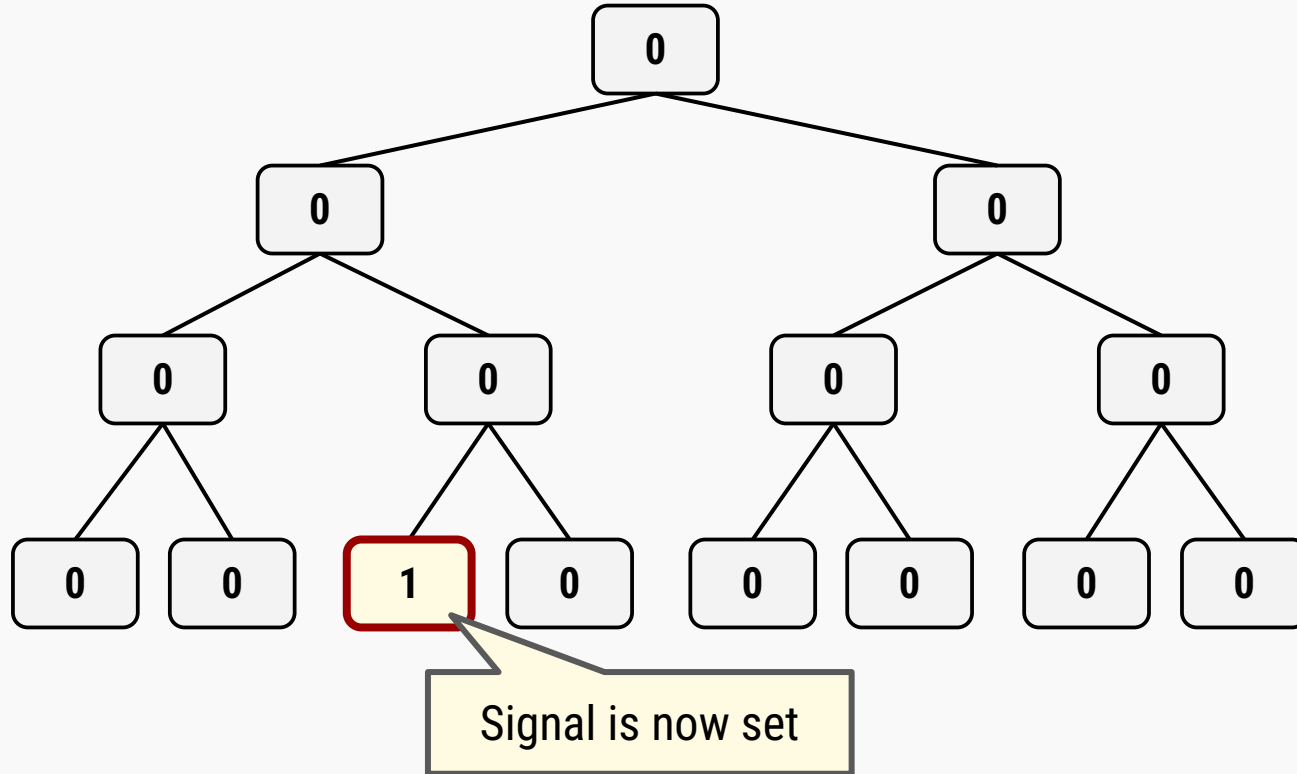
Setting A Signal: Signal Propagation



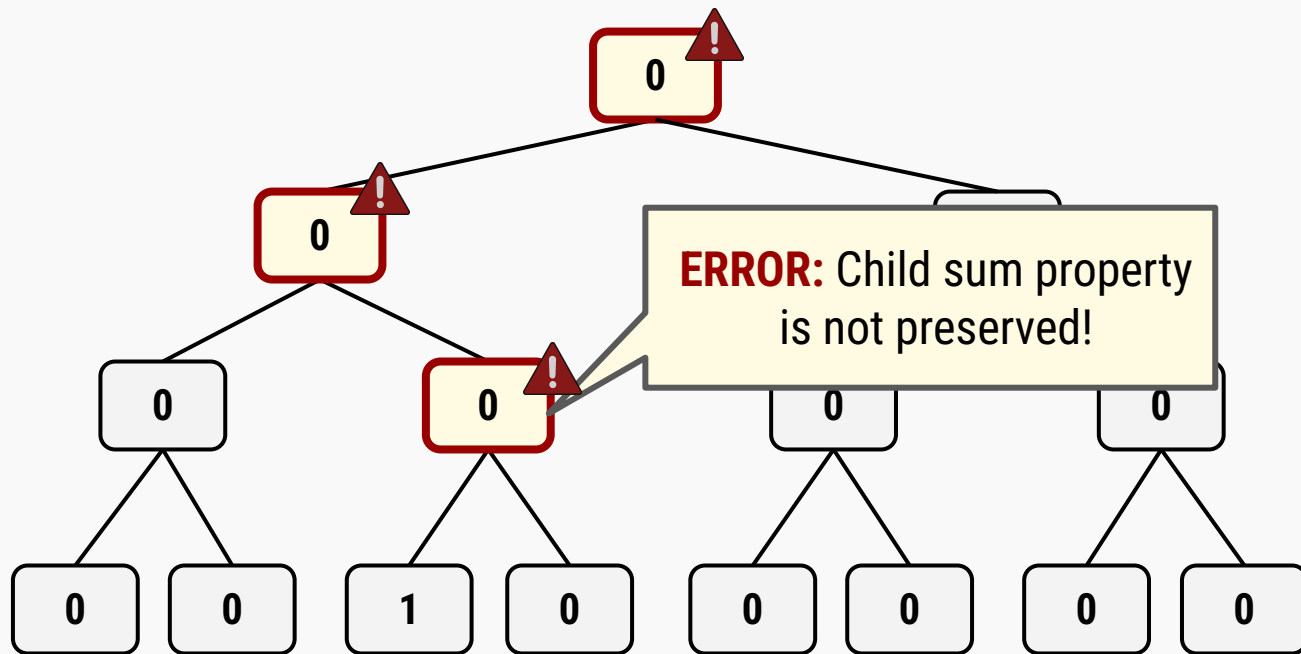
Setting A Signal: Signal Propagation



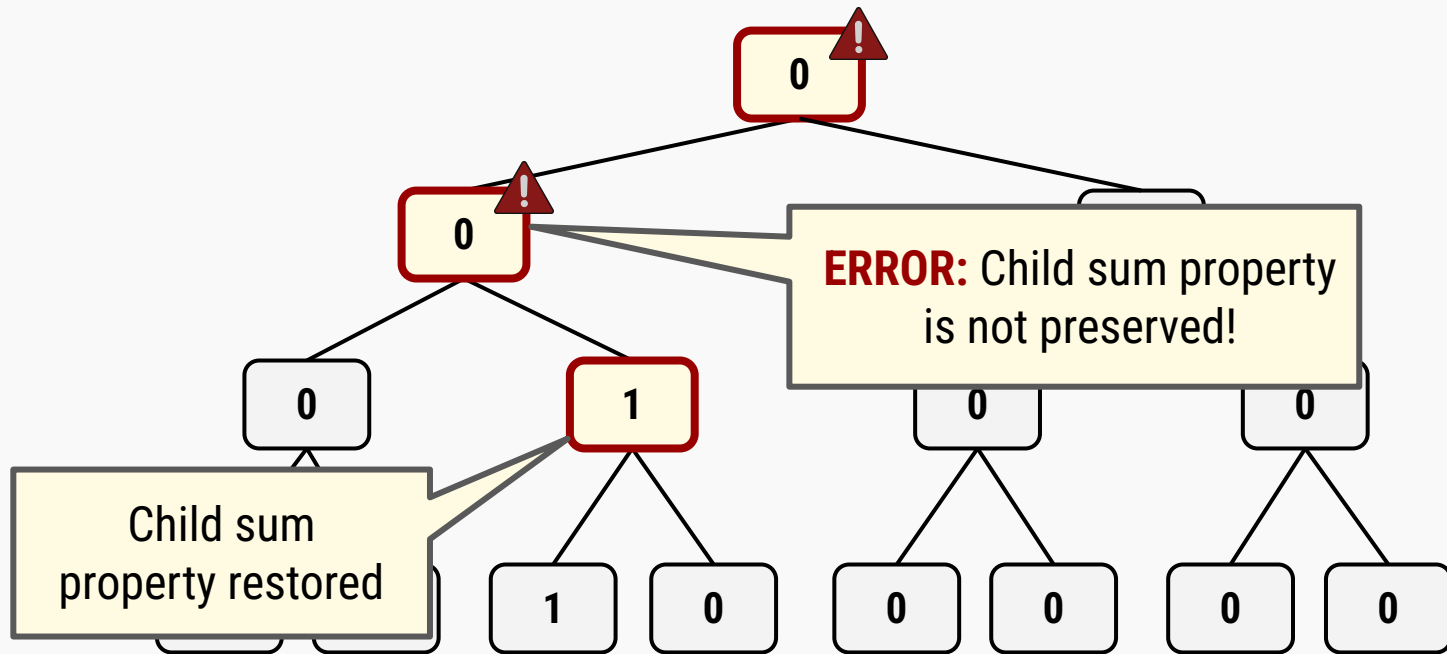
Setting A Signal: Signal Propagation



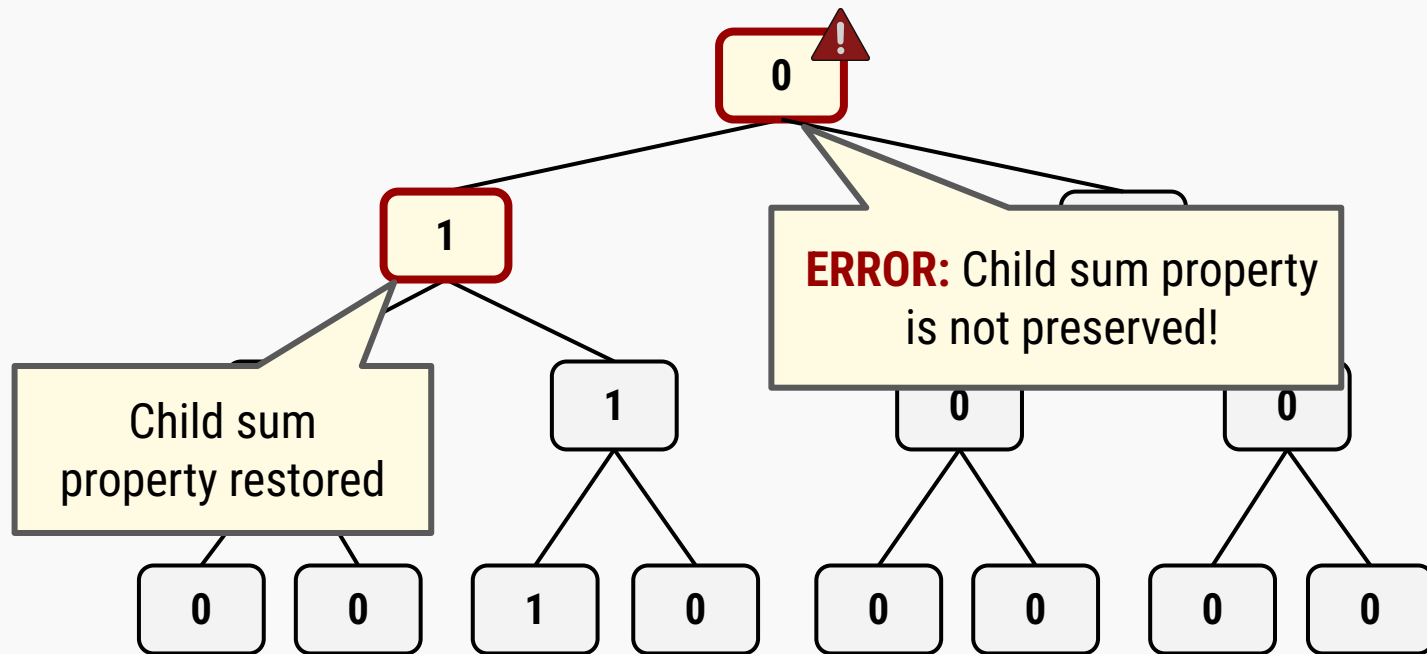
Setting A Signal: Signal Propagation



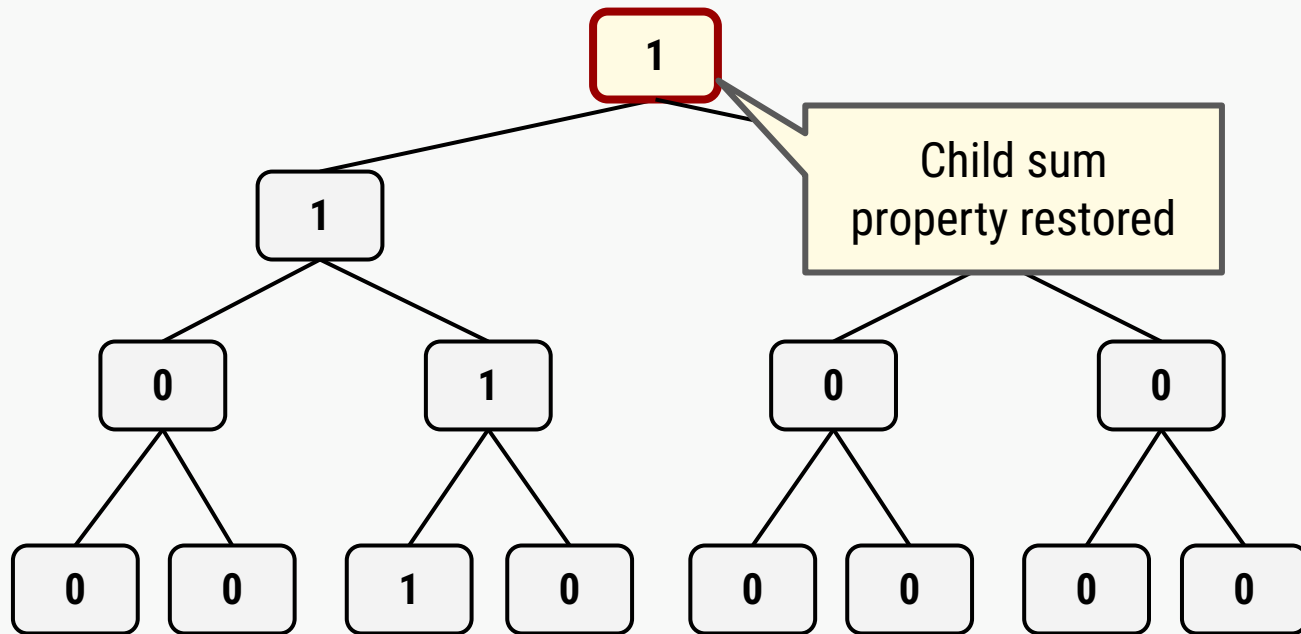
Setting A Signal: Signal Propagation



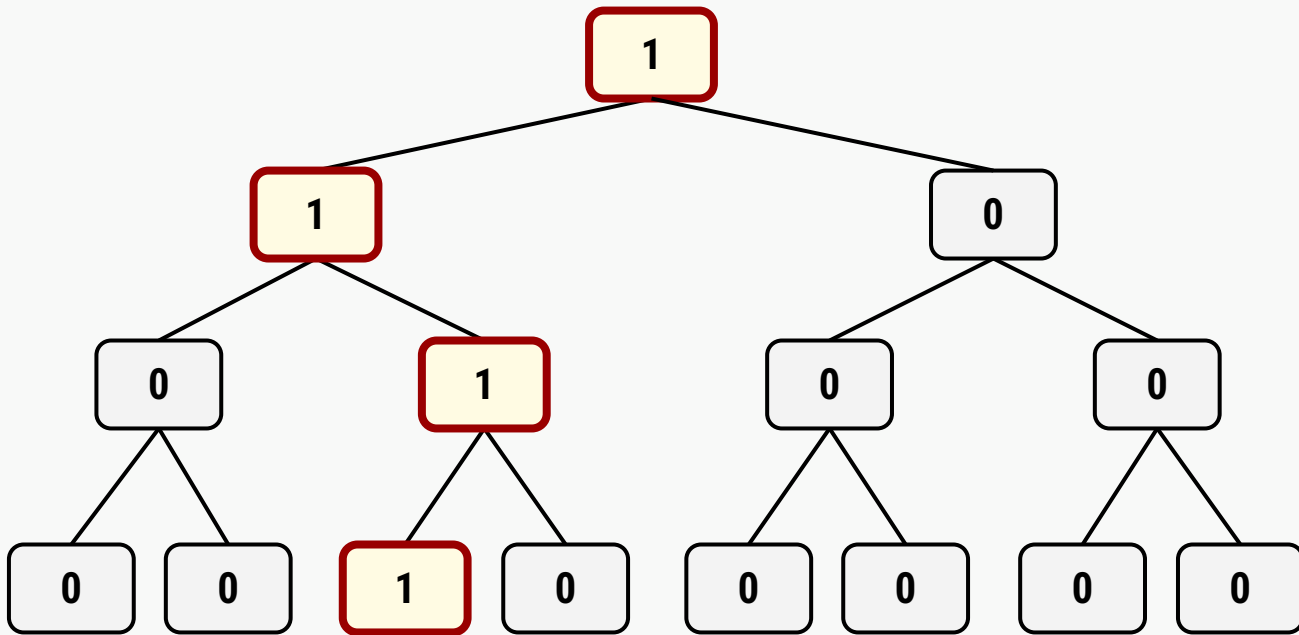
Setting A Signal: Signal Propagation



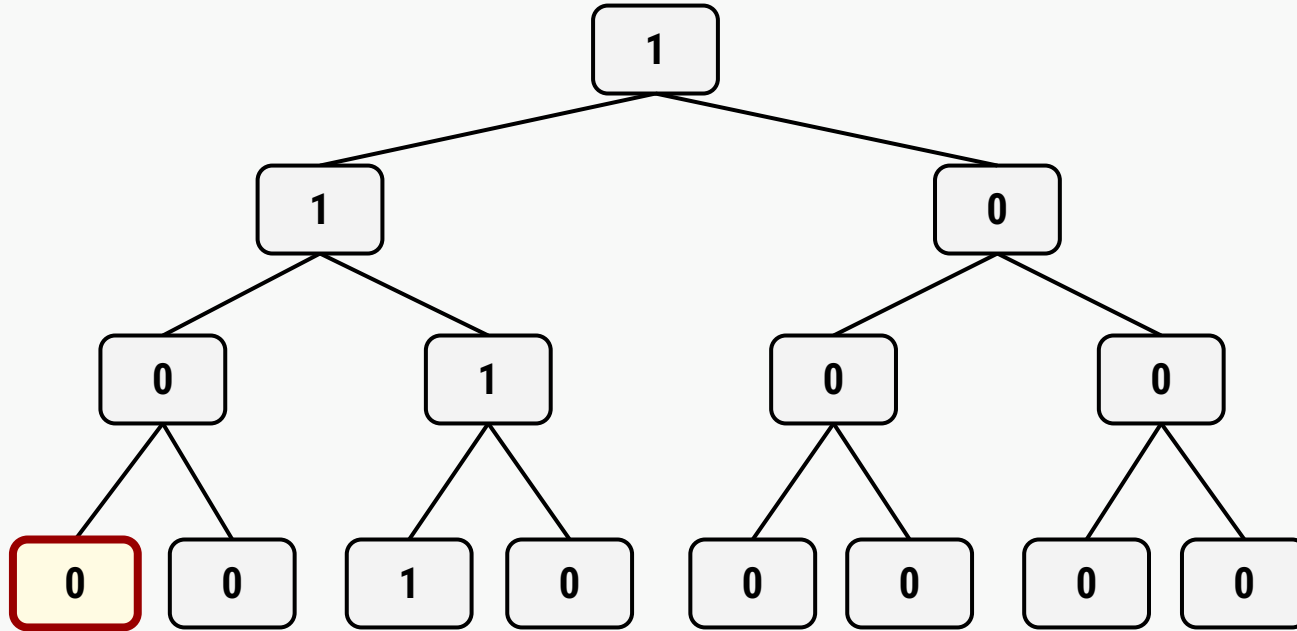
Setting A Signal: Signal Propagation



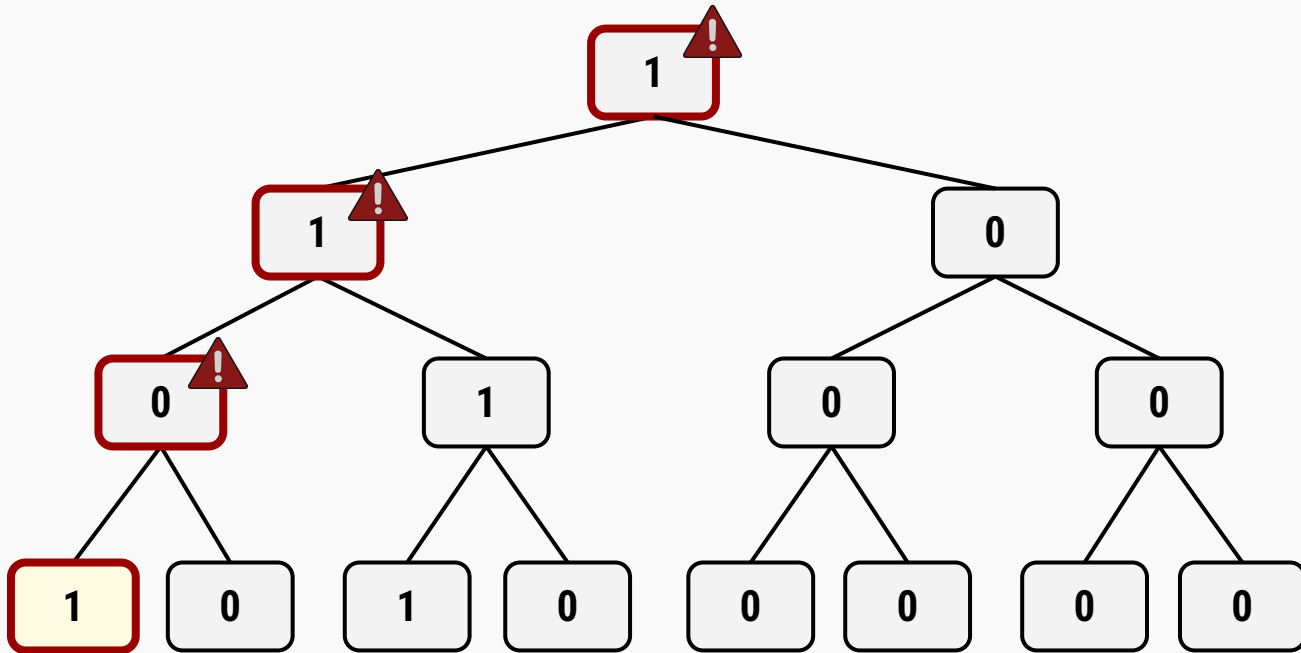
Setting A Signal: Signal Propagation



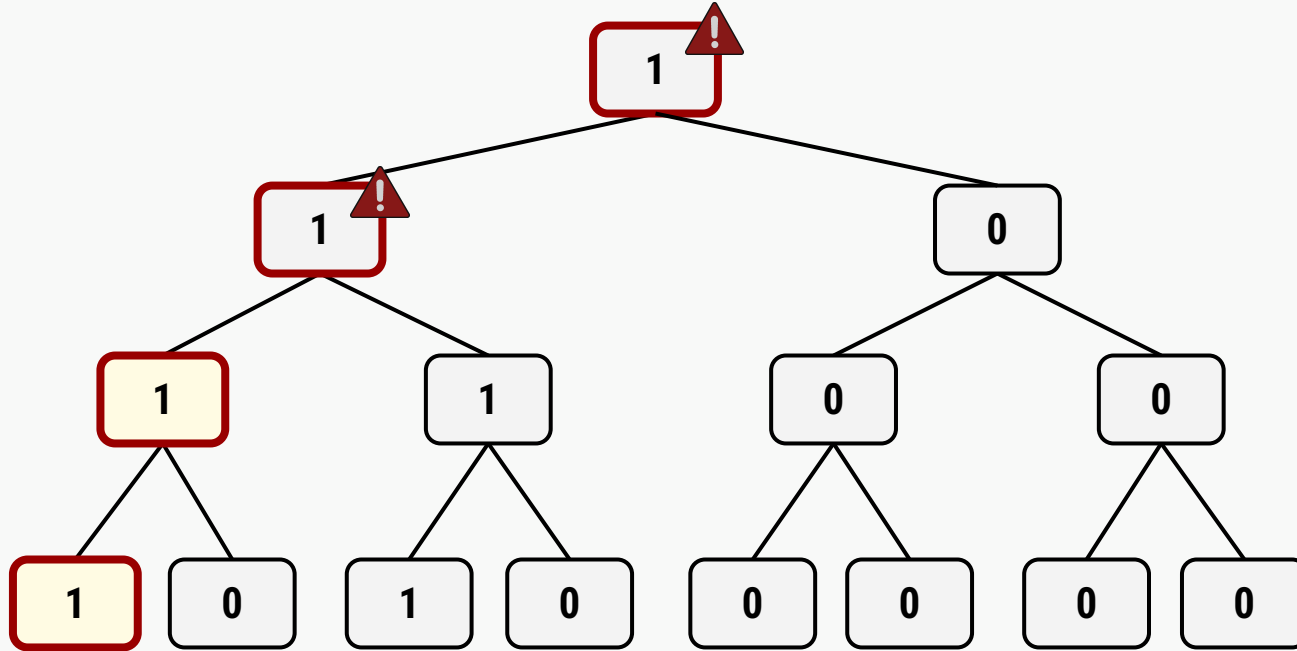
Setting A Signal: Signal Propagation



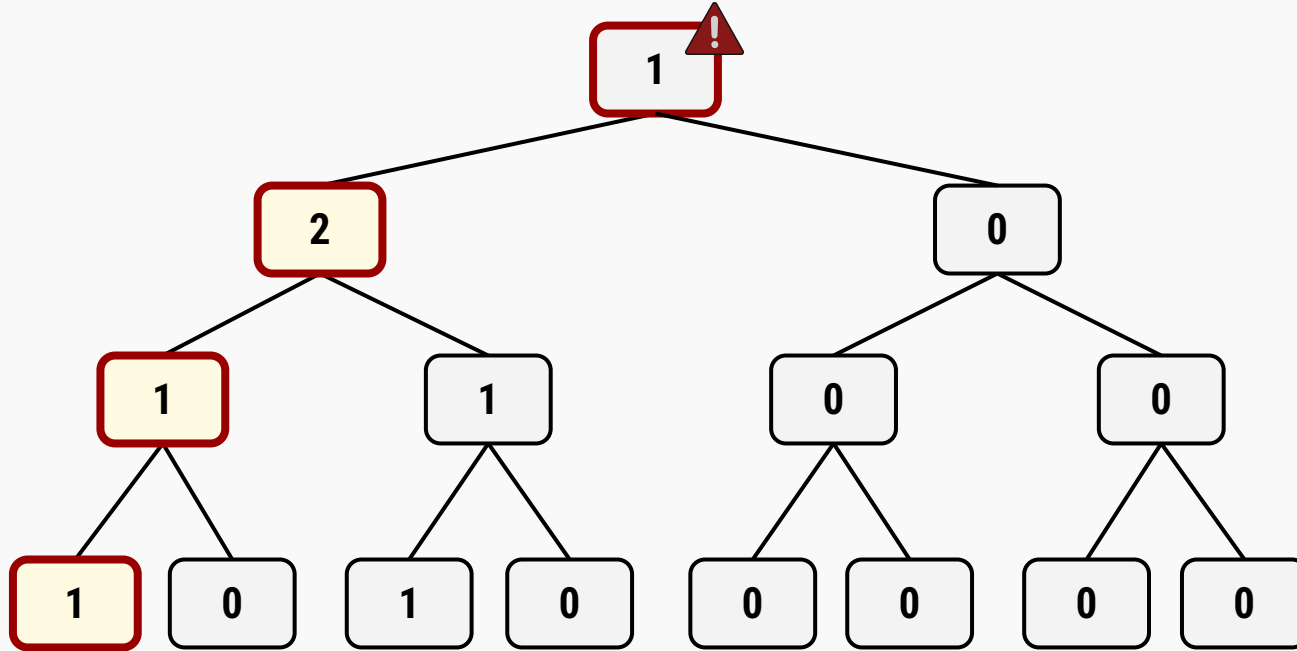
Setting A Signal: Signal Propagation



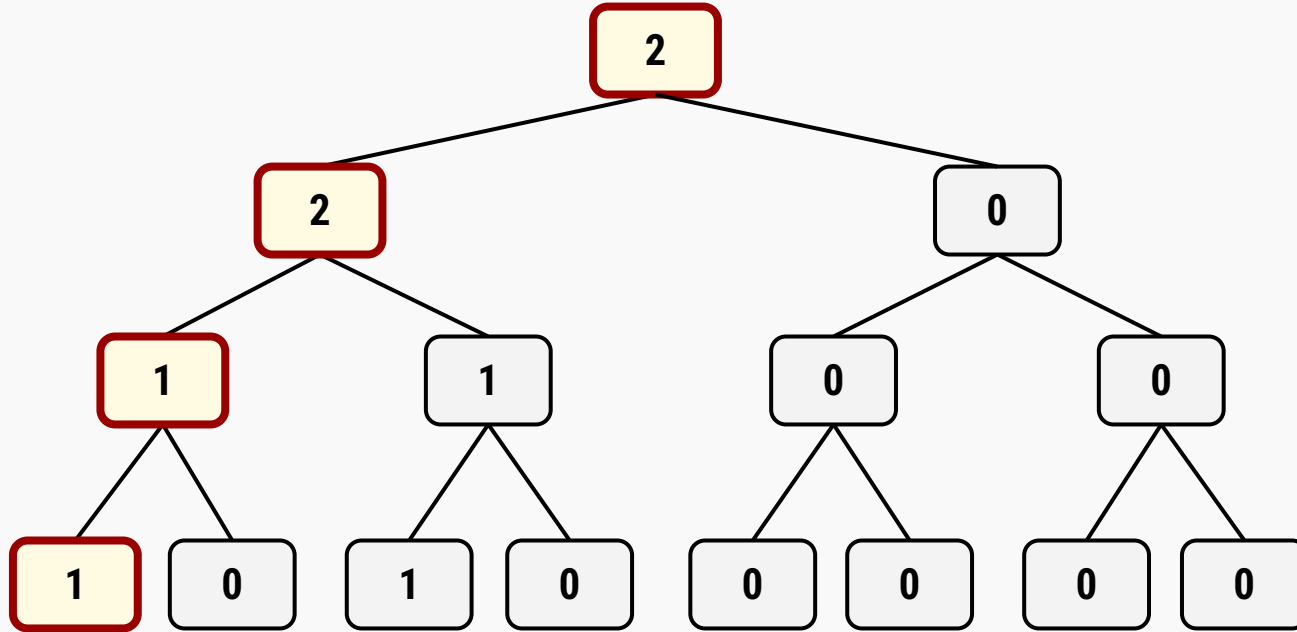
Setting A Signal: Signal Propagation



Setting A Signal: Signal Propagation



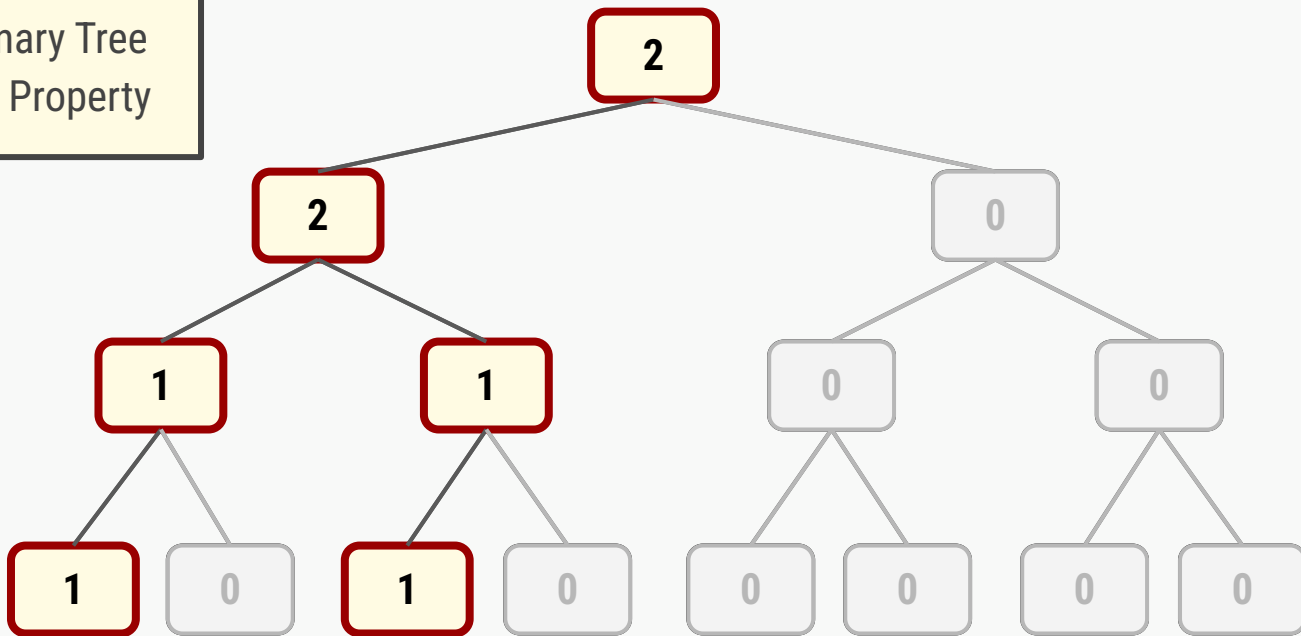
Setting A Signal: Signal Propagation



Proper Signal Trees

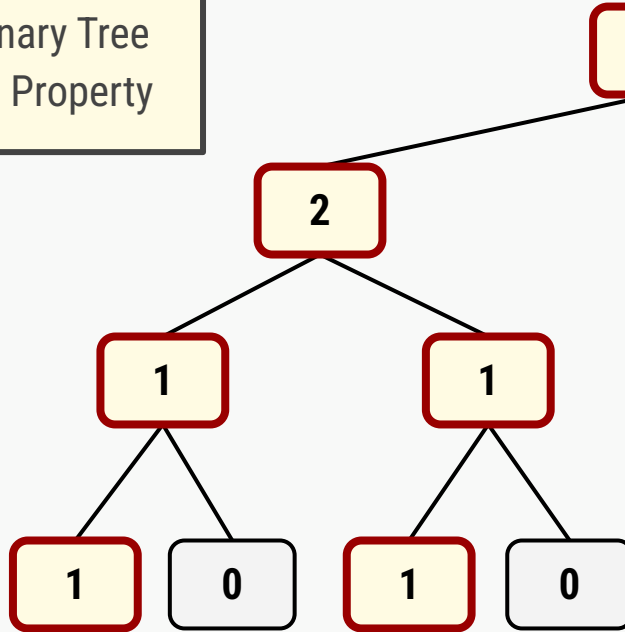
A Proper Signal Tree:

- ✓ Perfect Binary Tree
- ✓ Child Sum Property



A Proper Signal Tree:

- ✓ Perfect Binary Tree
- ✓ Child Sum Property



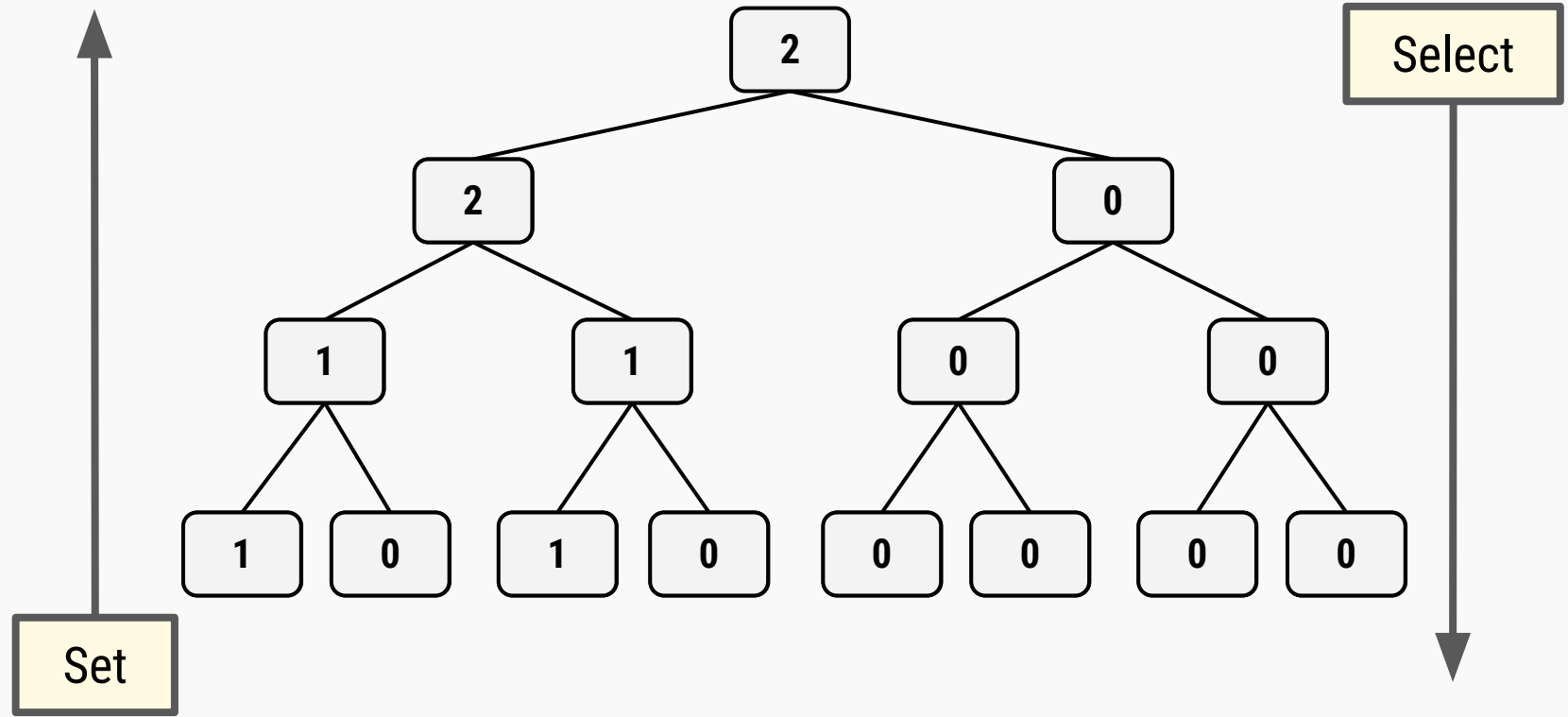
Every internal node represents the total number of ready signals contained beneath that point in the tree.

The root node therefore represents the total number of ready signals in the entire Signal Tree.

Selecting Signals

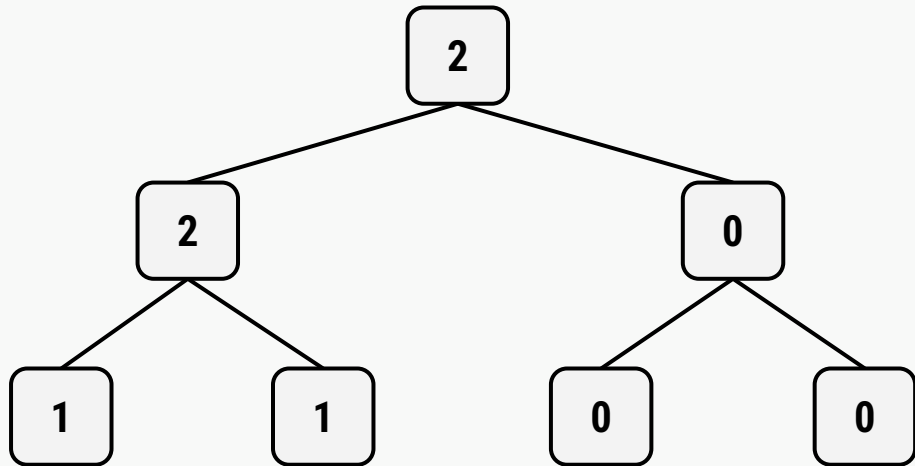
Choosing a ready signal

Selecting A Signal



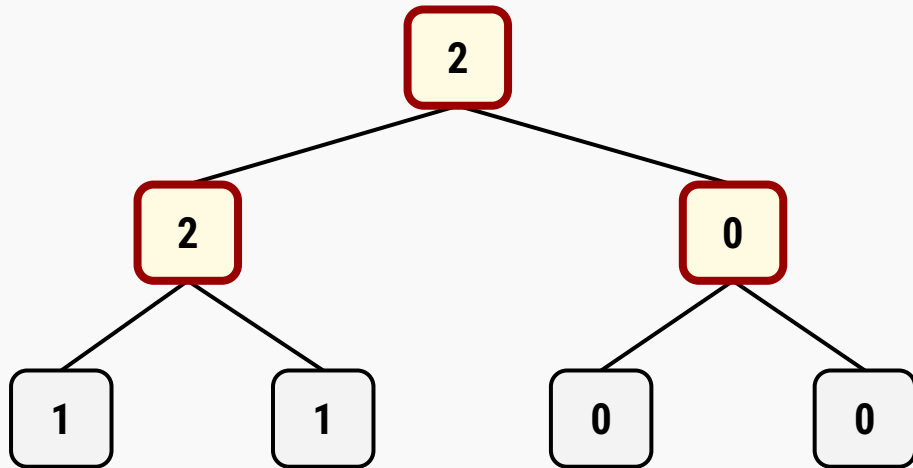
Selecting A Signal - Descending The Tree

- Internal node counts summarize ready descendants (*Child Sum Prop*)
- Zero-count nodes:
 - Contains no ready descendants and therefore no signals.
 - Can not be descended through.
- A successful decrement guarantees a traversable path to a ready signal



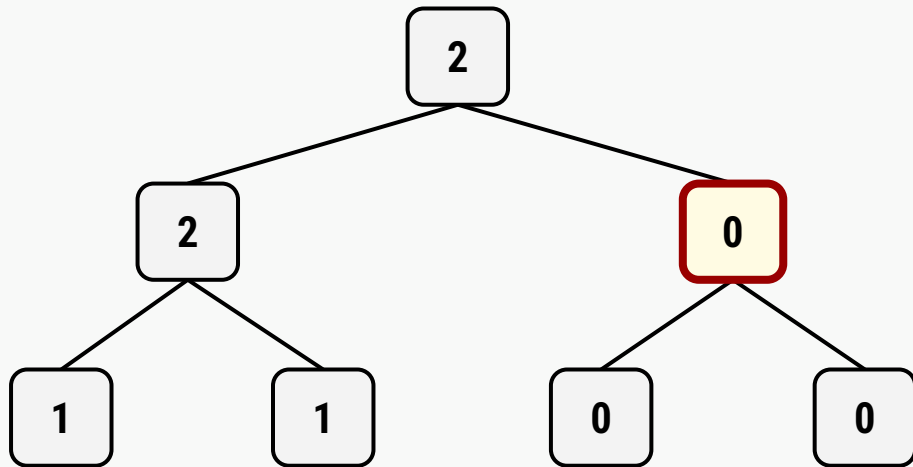
Selecting A Signal - Descending The Tree

- Internal node counts summarize ready descendants (*Child Sum Prop*)
- Zero-count nodes:
 - Contains no ready descendants and therefore no signals.
 - Can not be descended through.
- A successful decrement guarantees a traversable path to a ready signal



Selecting A Signal - Descending The Tree

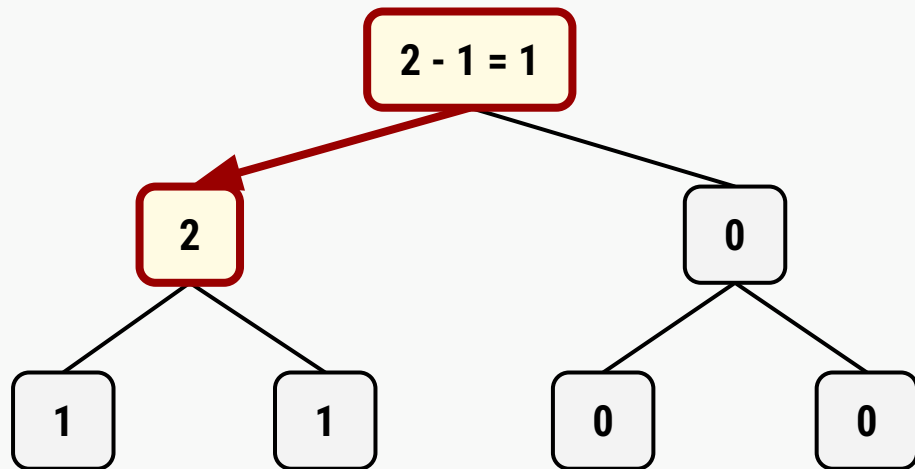
- Internal node counts summarize ready descendants (*Child Sum Prop*)
- Zero-count nodes:
 - Contains no ready descendants and therefore no signals.
 - Can not be descended through.
- A successful decrement guarantees a traversable path to a ready signal



Selecting A Signal - Descending The Tree

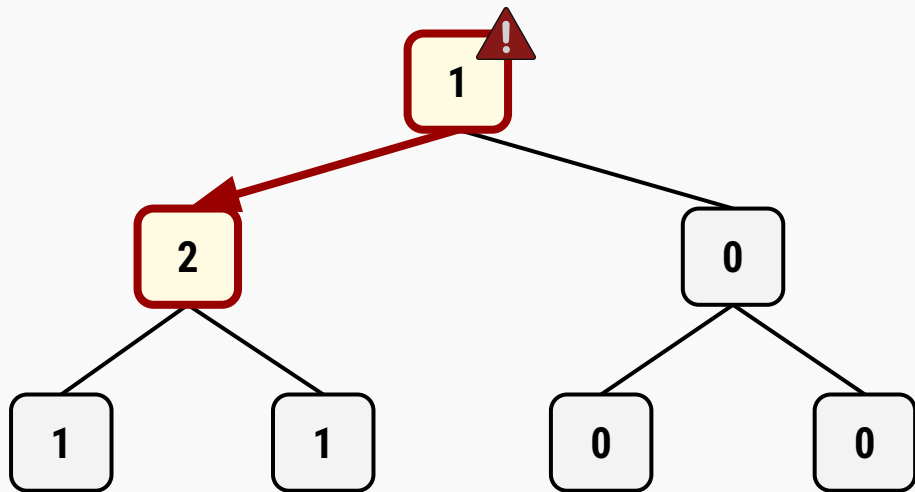
- Internal node counts summarize ready descendants (*Child Sum Prop*)
- Zero-count nodes:
 - Contains no ready descendants and therefore no signals.
 - Can not be descended through.

- A successful decrement guarantees a traversable path to a ready signal



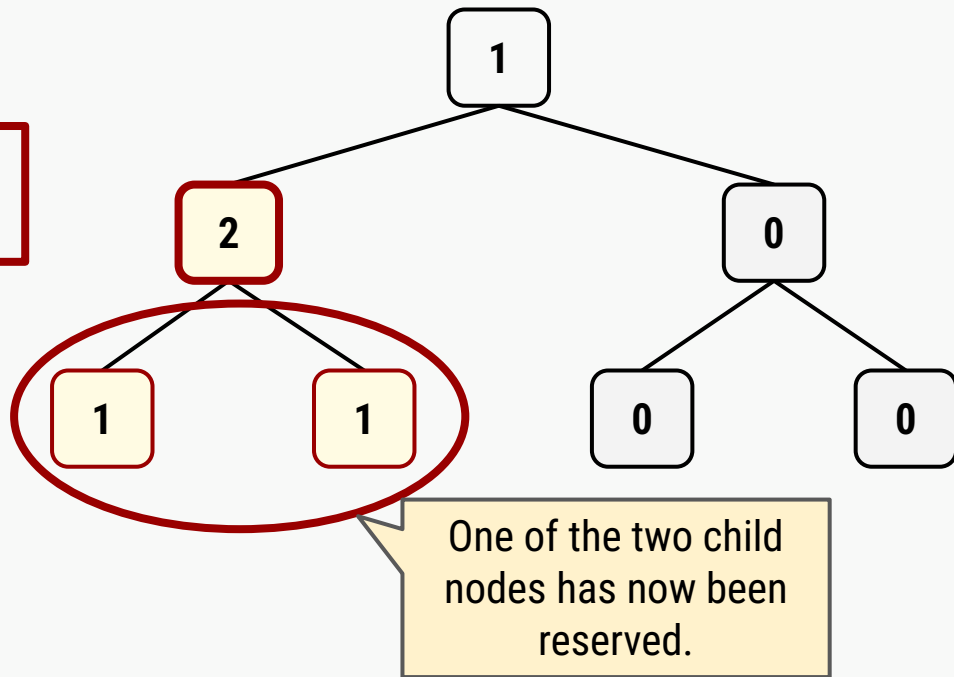
Selecting A Signal - The Parent Decrement Is A Reservation

- Parent count may temporarily differ from child sums. (*Violate Child Sum Property*)
- The decrement reserves one ready signal beneath this subtree
- Other selectors now observe reduced availability at this node
- The subtree below remains independently valid
- Successful descent through the tree restores the Child Sum Property.



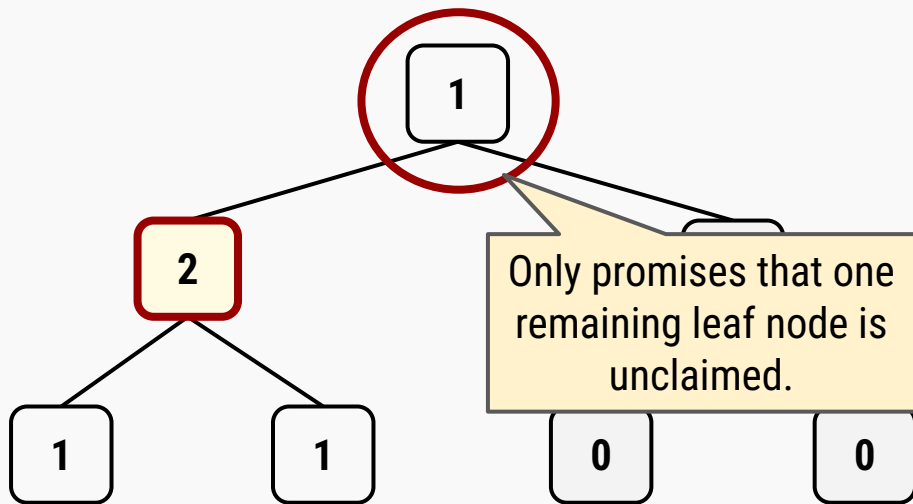
Selecting A Signal - The Parent Decrement Is A Reservation

- Parent count may temporarily differ from child sums. (*Violate Child Sum Property*)
- The decrement reserves one ready signal beneath this subtree
- Other selectors now observe reduced availability at this node
- The subtree below remains independently valid
- Successful descent through the tree restores the Child Sum Property.



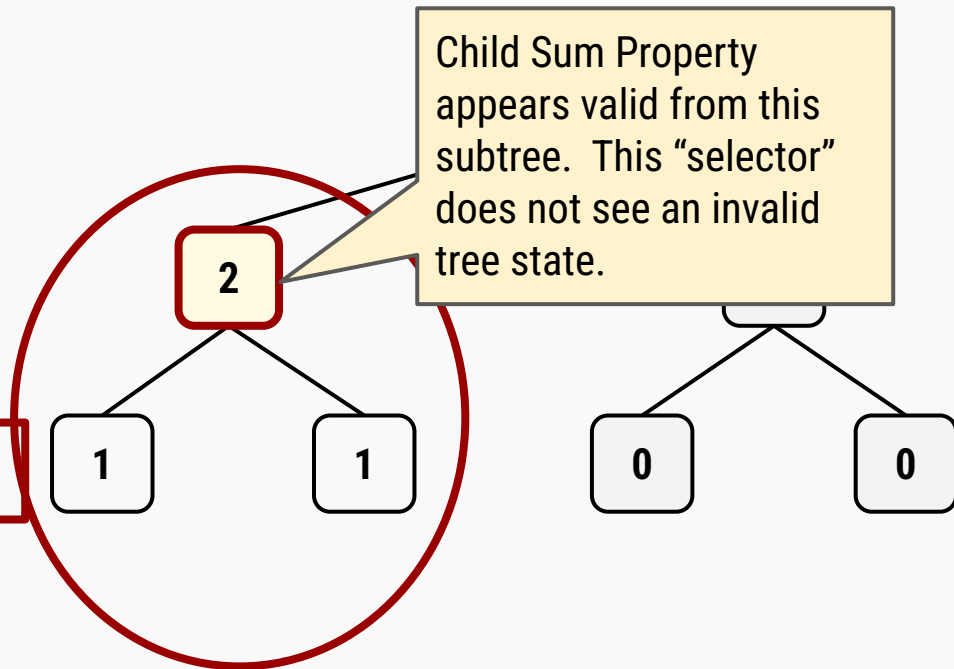
Selecting A Signal - The Parent Decrement Is A Reservation

- Parent count may temporarily differ from child sums. (*Violate Child Sum Property*)
- The decrement reserves one ready signal beneath this subtree
- Other selectors now observe reduced availability at this node
- The subtree below remains independently valid
- Successful descent through the tree restores the Child Sum Property.



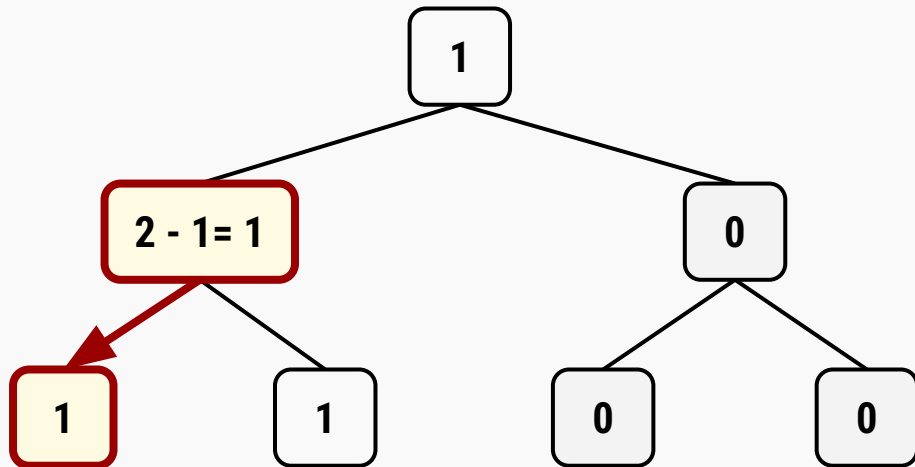
Selecting A Signal - The Parent Decrement Is A Reservation

- Parent count may temporarily differ from child sums. (*Violate Child Sum Property*)
- The decrement reserves one ready signal beneath this subtree
- Other selectors now observe reduced availability at this node
- The subtree below remains independently valid
- Successful descent through the tree restores the Child Sum Property.



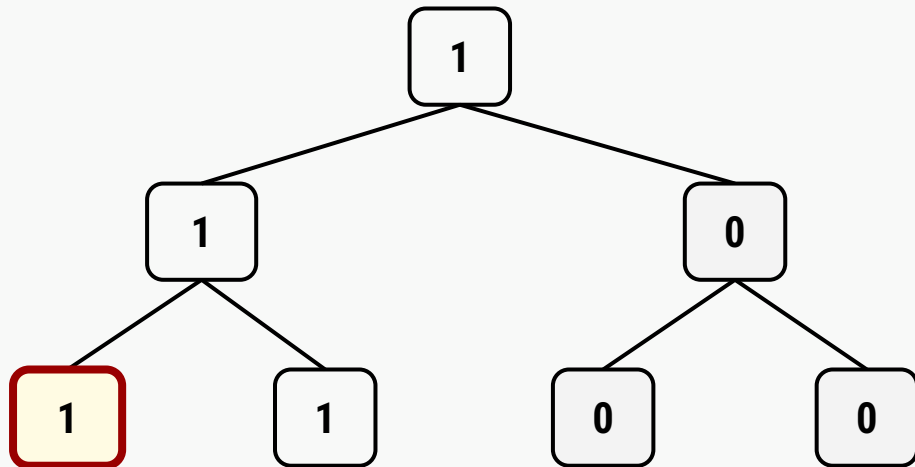
Selecting A Signal - The Parent Decrement Is A Reservation

- Parent count may temporarily differ from child sums. (*Violate Child Sum Property*)
- The decrement reserves one ready signal beneath this subtree
- Other selectors now observe reduced availability at this node
- The subtree below remains independently valid
- Successful descent through the tree restores the Child Sum Property.



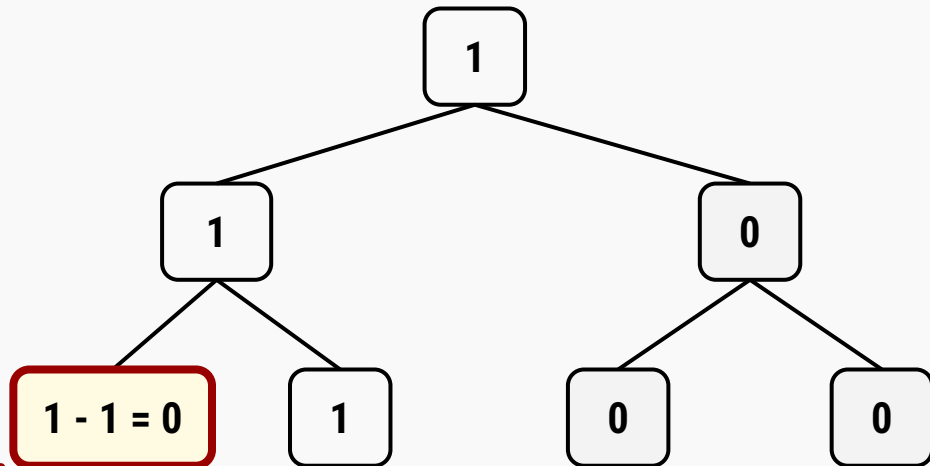
Selecting A Signal - The Parent Decrement Is A Reservation

- Parent count may temporarily differ from child sums. (*Violate Child Sum Property*)
- The decrement reserves one ready signal beneath this subtree
- Other selectors now observe reduced availability at this node
- The subtree below remains independently valid
- Successful descent through the tree restores the Child Sum Property.



Selecting A Signal - The Parent Decrement Is A Reservation

- Parent count may temporarily differ from child sums. (*Violate Child Sum Property*)
- The decrement reserves one ready signal beneath this subtree
- Other selectors now observe reduced availability at this node
- The subtree below remains independently valid
- Successful descent through the tree restores the Child Sum Property.

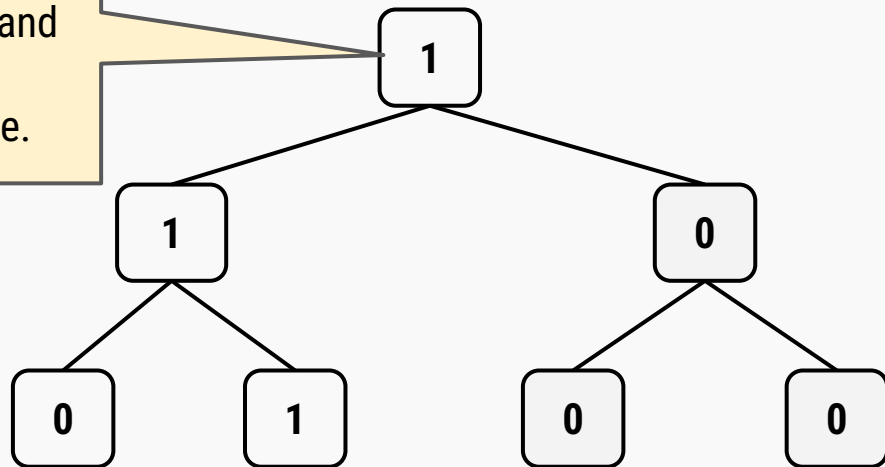


Selecting A Signal - The Parent Decrement Is A Reservation

- Parent count may temporarily violate Child Sum Property.
- The decrement reserves one unit of work beneath this subtree
- Other selectors now observe reduced availability at this node
- The subtree below remains independently valid

Selection completed and Child Sum Property restored for entire tree.

- Successful descent through the tree restores the Child Sum Property.

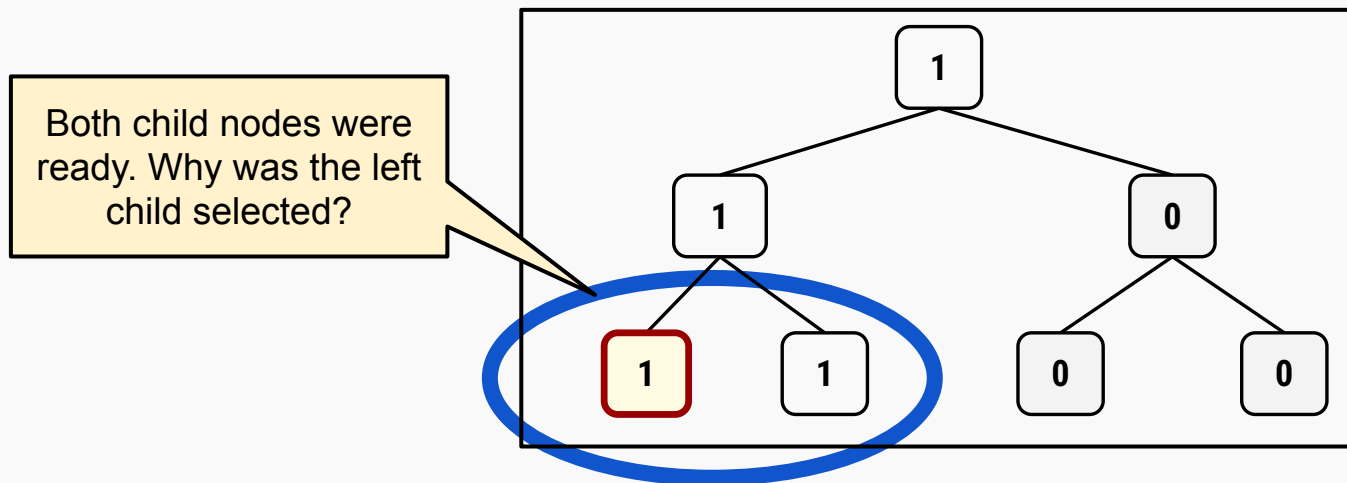


Selection Bias

Ensuring Fair Signal Selection

Selection Bias: Hinting and Fairness

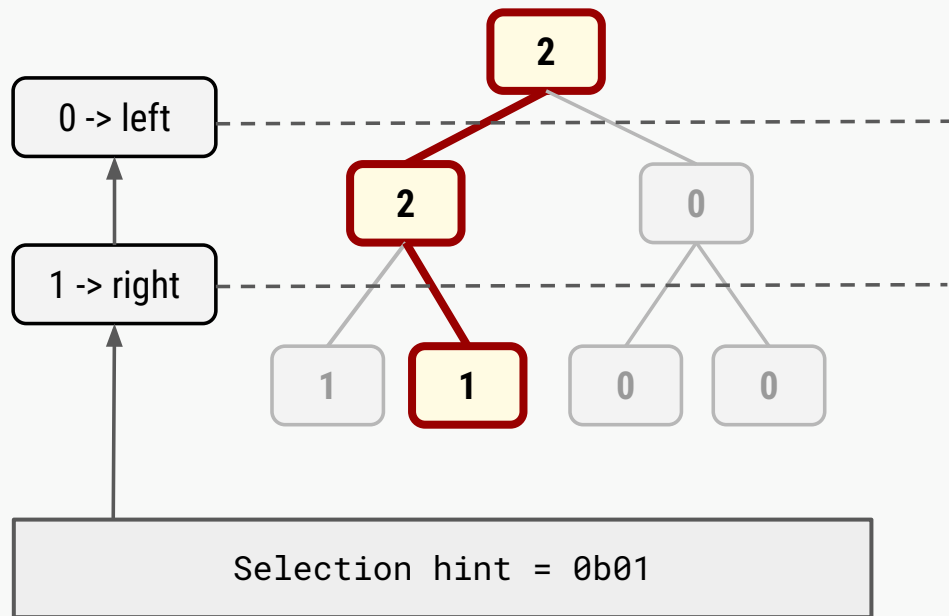
In the selection slides, how did the selector decide to select the left child and not the right child when both were non-zero? How was this choice made?



Selection Bias: Hinting and Fairness

Selection hints store traversal preference state

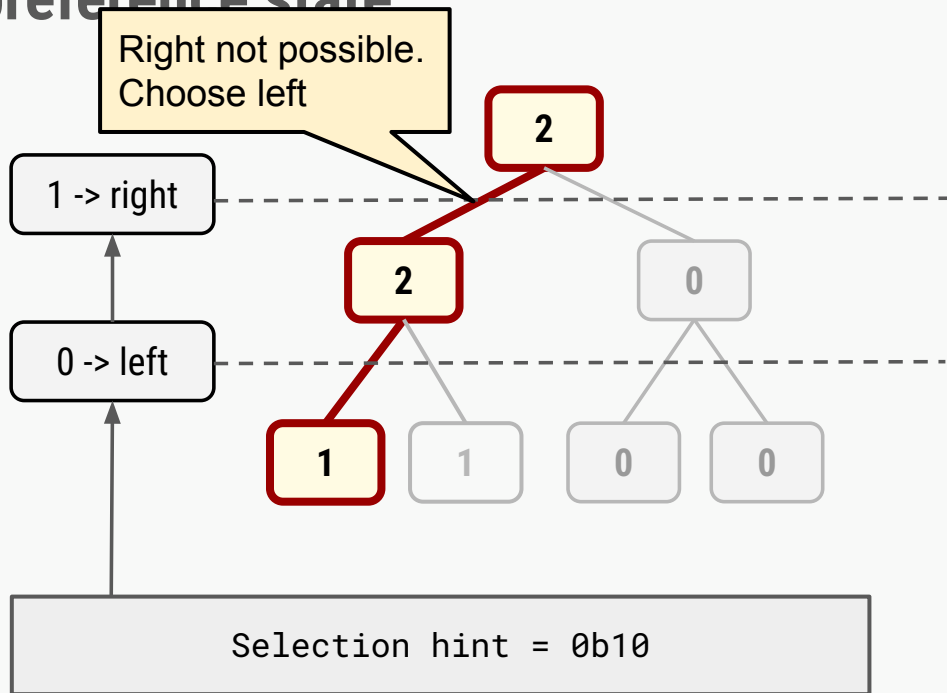
- One hint bit per tree level
- Each bit selects a *preferred* traversal direction:
 - 0 = *prefer left*
 - 1 = *prefer right*
- Hints distribute traversal pressure across the tree
- Traversal still skips empty subtrees



Selection Bias: Hinting and Fairness

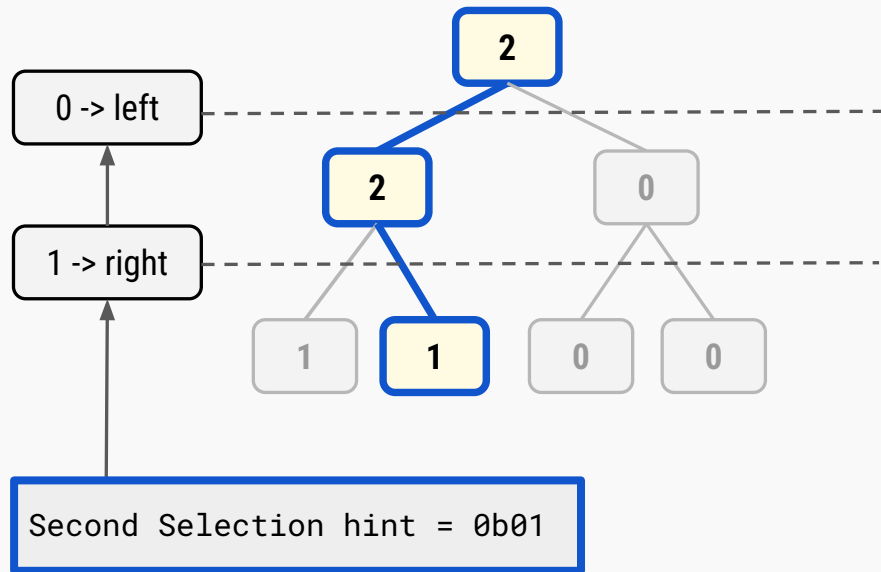
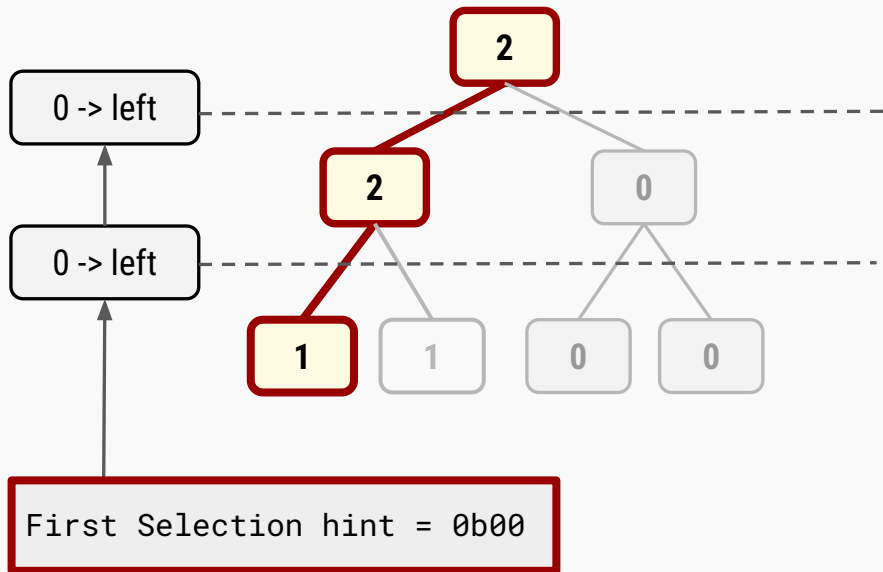
Selection hints store traversal preference state

- One hint bit per tree level
- Each bit selects a *preferred* traversal direction:
 - 0 = *prefer left*
 - 1 = *prefer right*
- Hints distribute traversal pressure across the tree
- Traversal still skips empty subtrees



Selection Bias: Advancing The Selection Hint

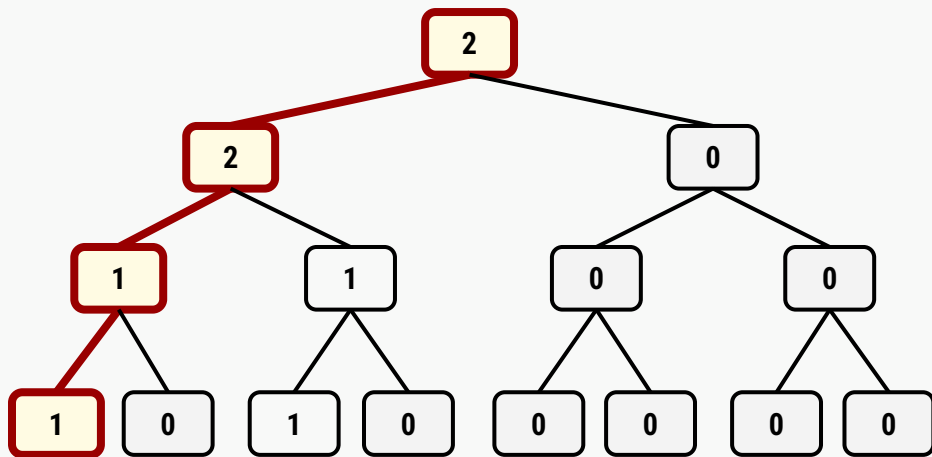
The selection hint is adjusted after each selection.



Advancing The Hint: Find The Next Right Turn

During traversal, whenever selection goes left while a ready right subtree exists, that right subtree becomes the next candidate region.

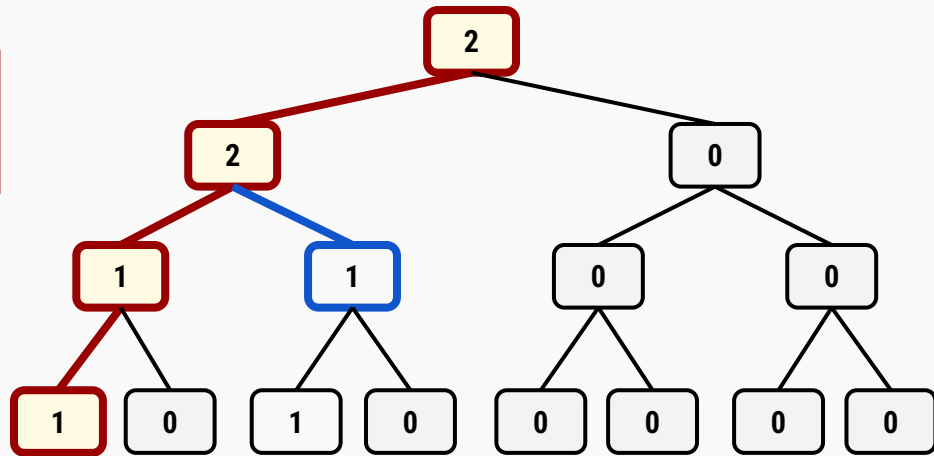
- Selection follows the current hint when possible
- If it chooses left while right is also ready, remember that right subtree
- After selection, advance the hint to the last remembered right subtree
- Clear lower hint bits so traversal starts at the left edge of that subtree



Advancing The Hint: Find The Next Right Turn

During traversal, whenever selection goes left while a ready right subtree exists, that right subtree becomes the next candidate region.

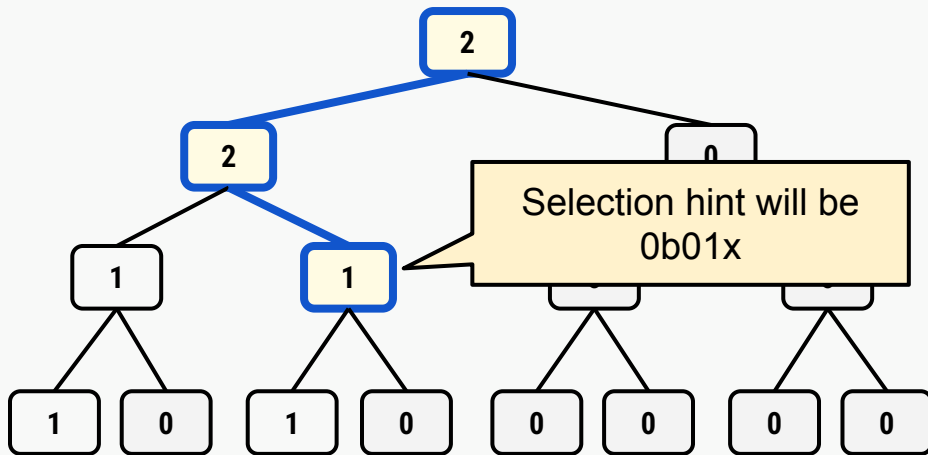
- Selection follows the current hint when possible
- If it chooses left while right is also ready, remember that right subtree
- After selection, advance the hint to the last remembered right subtree
- Clear lower hint bits so traversal starts at the left edge of that subtree



Advancing The Hint: Find The Next Right Turn

During traversal, whenever selection goes left while a ready right subtree exists, that right subtree becomes the next candidate region.

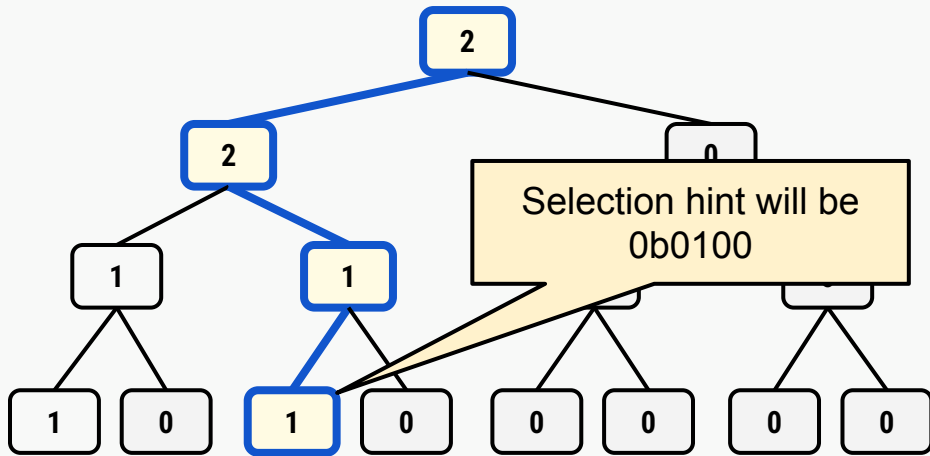
- Selection follows the current hint when possible
- If it chooses left while right is also ready, remember that right subtree
- After selection, advance the hint to the last remembered right subtree
- Clear lower hint bits so traversal starts at the left edge of that subtree



Advancing The Hint: Find The Next Right Turn

During traversal, whenever selection goes left while a ready right subtree exists, that right subtree becomes the next candidate region.

- Selection follows the current hint when possible
- If it chooses left while right is also ready, remember that right subtree
- After selection, advance the hint to the last remembered right subtree
- Clear lower hint bits so traversal starts at the left edge of that subtree



Signal Trees

Implementation Details

Implementation

Signal Trees are not true binary trees:

- Nodes are packed counters forming lanes within `std::atomic<std::uint64_t>` nodes
 - Internal representation still mirrors binary tree
 - Lanes allow for more compact trees with greatly reduced number of levels
 - Reduces CAS operations required during `select()`
 - Allows selections to consider more than two nodes per CAS operation

Implementation

Leaf level `select()` is wait-free:

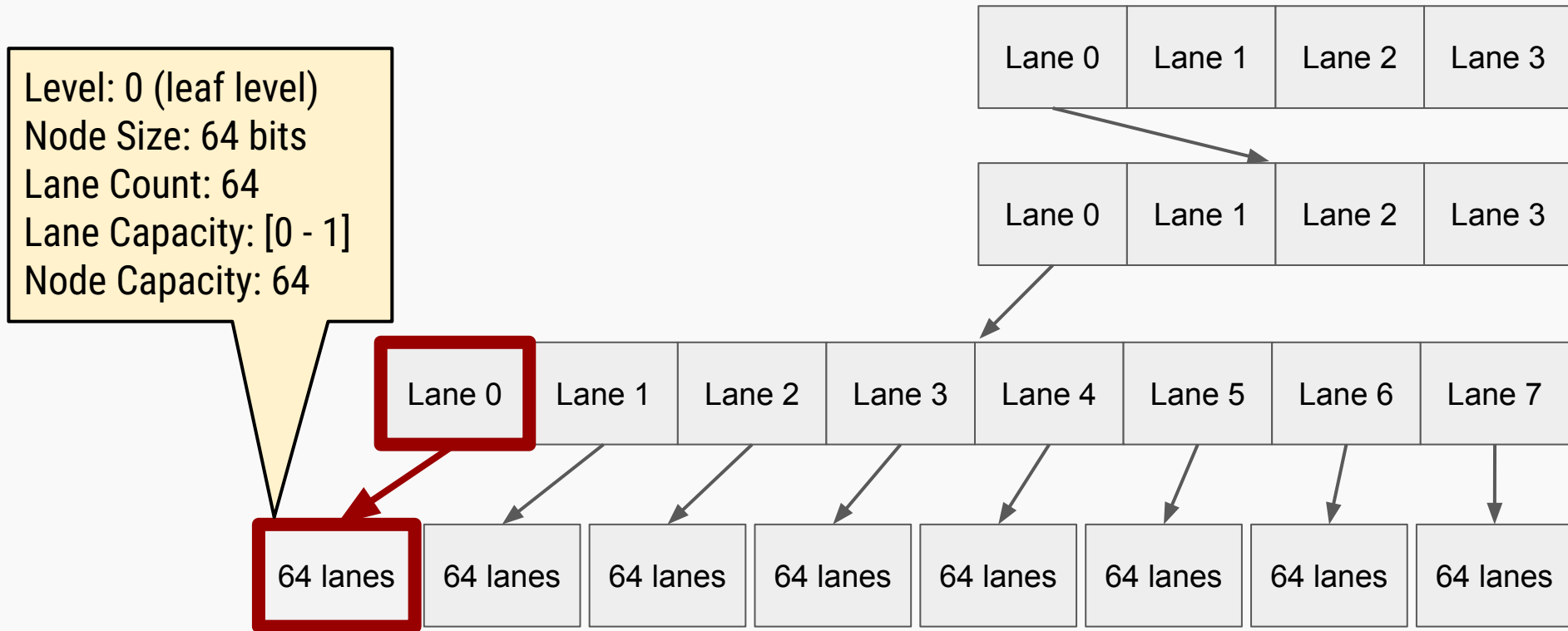
- Leaf nodes contain 64 one bit lanes representing 64 individual signals
- Leaf level updates Are wait-free
- The full `select()` operation is lock-free, but the final leaf update is wait-free.

Implementation

Tree depth is compile time configurable:

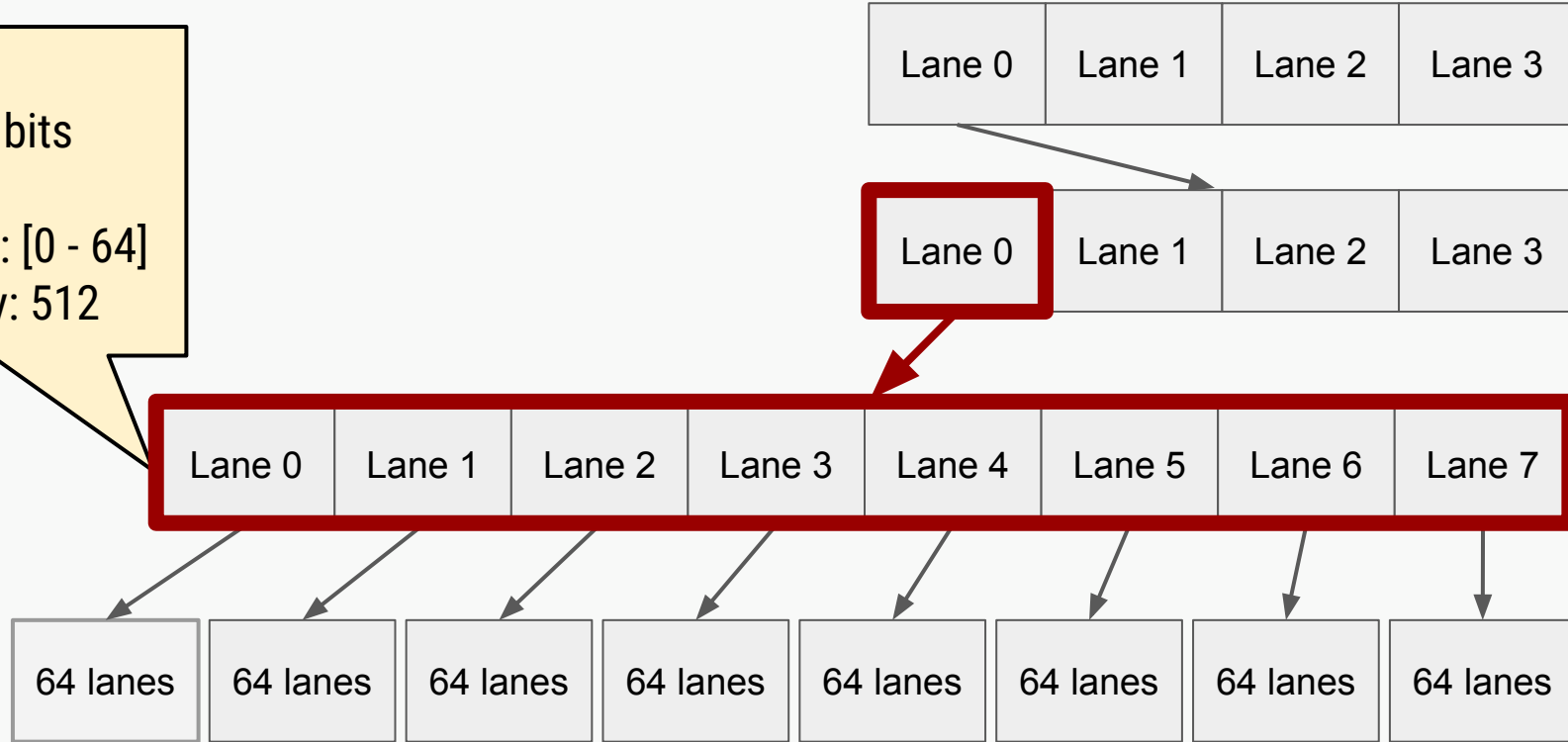
- Trees are defined by number of levels - determining their total capacity as well as the fixed number of CAS operations required to complete a select() call.
- Multiple Signal Trees can be combined to act as one larger scheduler
 - Tradeoff: Faster throughput with identical fairness but less ability to select via prioritization.
- Single level Signal Trees decay to bitsets with comparable fairness under the same selection policy
 - Sharded single-level Signal Trees yield greatest throughput while retaining near FIFO fairness - best choice if priority selection is not required.

Nodes as lanes: A four level Signal Tree (capacity = 8,192)

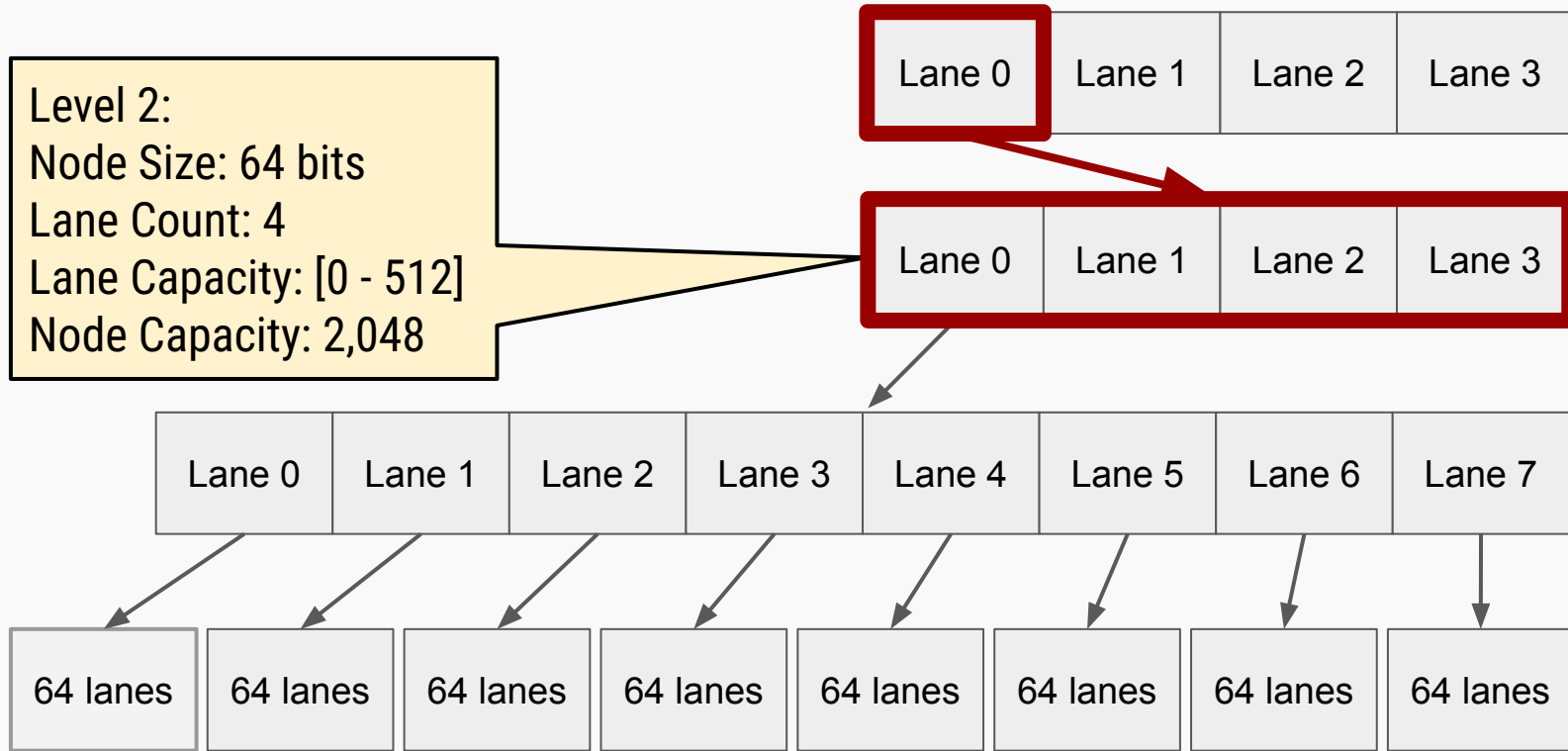


Nodes as lanes: A four level Signal Tree (capacity = 8,192)

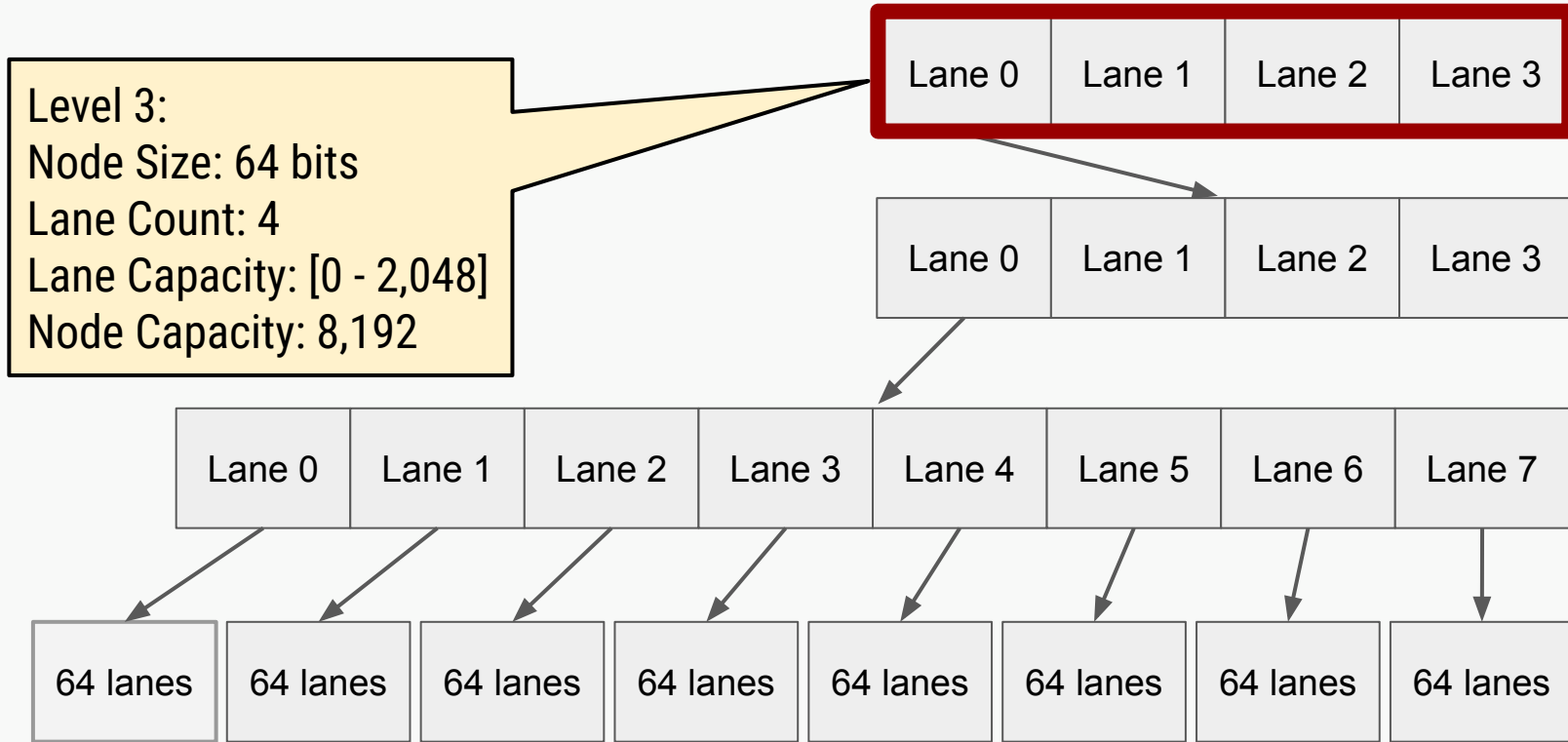
Level 1:
Node Size: 64 bits
Lane Count: 8
Lane Capacity: [0 - 64]
Node Capacity: 512



Nodes as lanes: A four level Signal Tree (capacity = 8,192)



Nodes as lanes: A four level Signal Tree (capacity = 8,192)



Signal Tree Capacity (64 bit nodes)

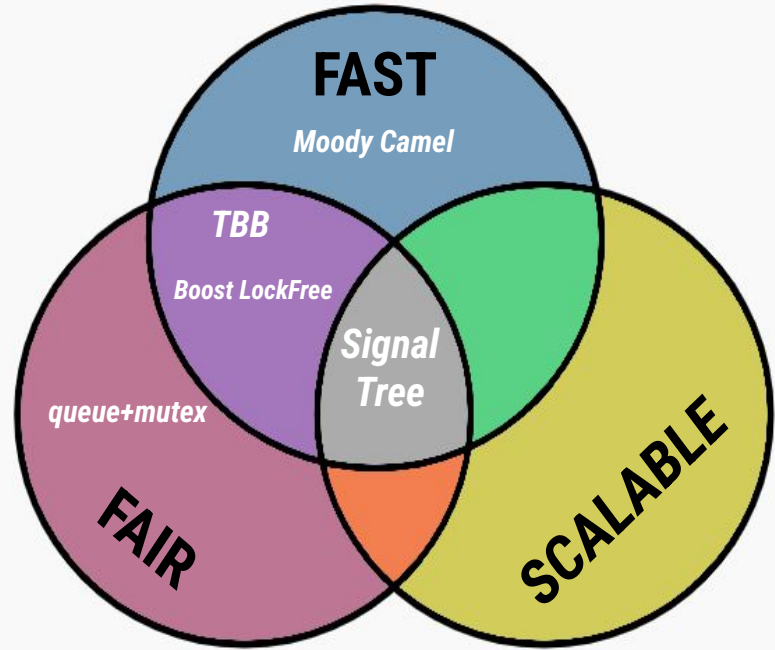
Levels	Lane Count/Node	Lane Capacity	Tree Capacity
1	64	1	64
2	8	64	512
3	4	512	2,048
4	4	2,048	8,192
5	4	8,192	32,768
6	4	32,768	131,072
7	2	131,072	262,144
8	2	262,144	524,288

Signal Tree Capacity (128 bit nodes)

Levels	Lane Count/Node	Lane Capacity	Tree Capacity
1	128	1	128
2	16	128	2,048
3	8	2,048	16,384
4	8	16,384	131,072
5	4	131,072	524,288
6	4	524,288	2,097,152
7	4	2,097,152	8,388,608
8	4	8,388,608	33,554,432

Recap: Three Properties of an Ideal Scheduler

- ✓ **Throughput:** *Signal scheduling is on the critical path of every task system. How fast can ready signals be identified and selected?*
- ✓ **Scalability:** *Modern hardware has many cores. Adding threads should add performance. How closely does throughput scale in relation to thread count?*
- ✓ **Fairness:** *Starvation is silent and catastrophic in production systems. Does every ready signal get serviced reliably and with equal priority?*



Part 4

Signal Trees - Building An Execution Layer

Execution Layer: Signal Trees Are Not Executors

Signal Trees schedule readiness. Execution layers decide how work executes.

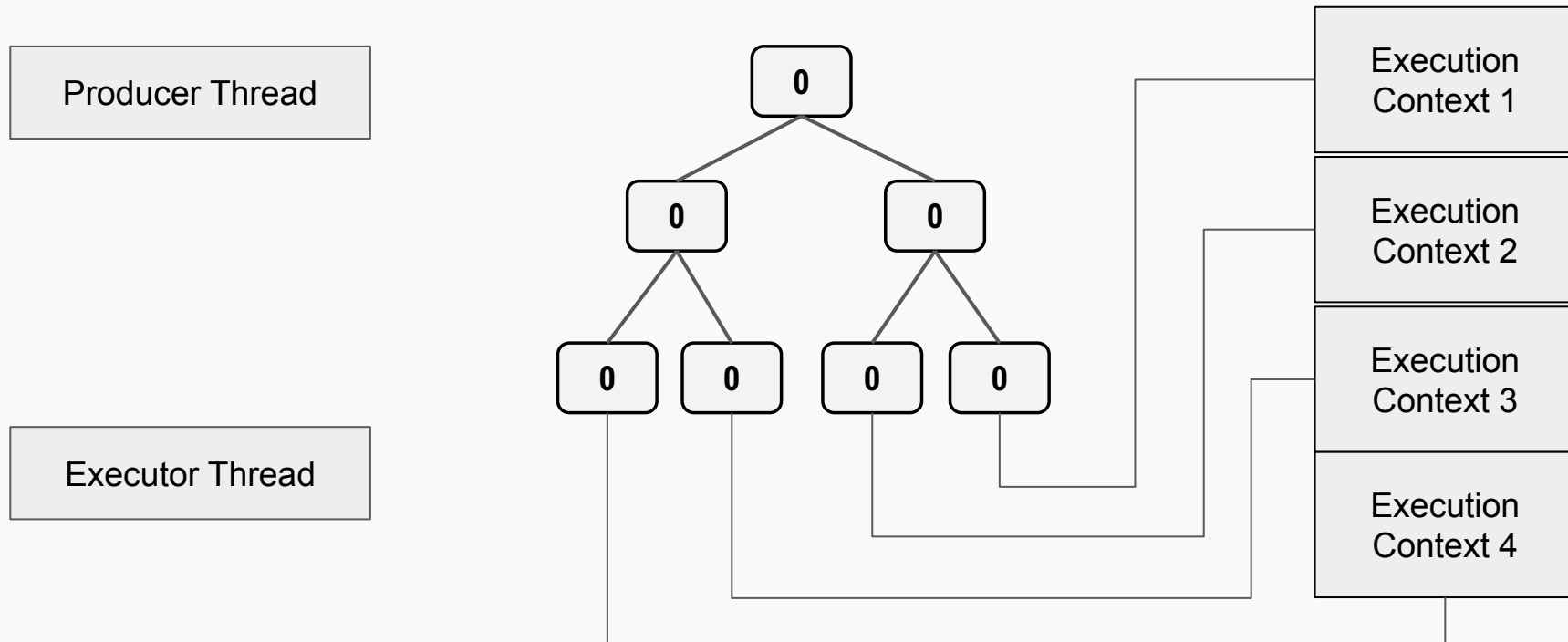
Signal Tree:

- readiness publication
- readiness selection
- fairness
- scalability

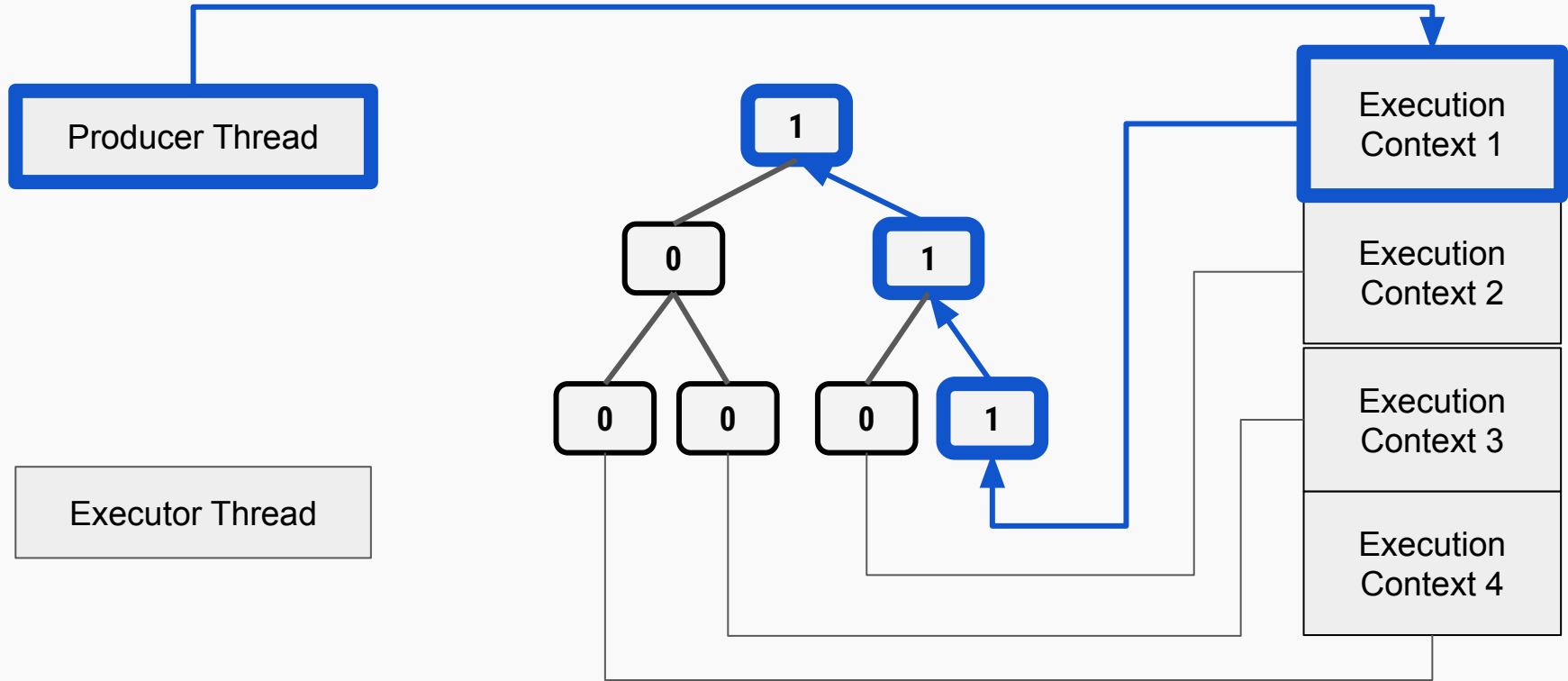
Execution Layer:

- task execution
- ownership
- locality
- batching
- continuation
- work lifetime

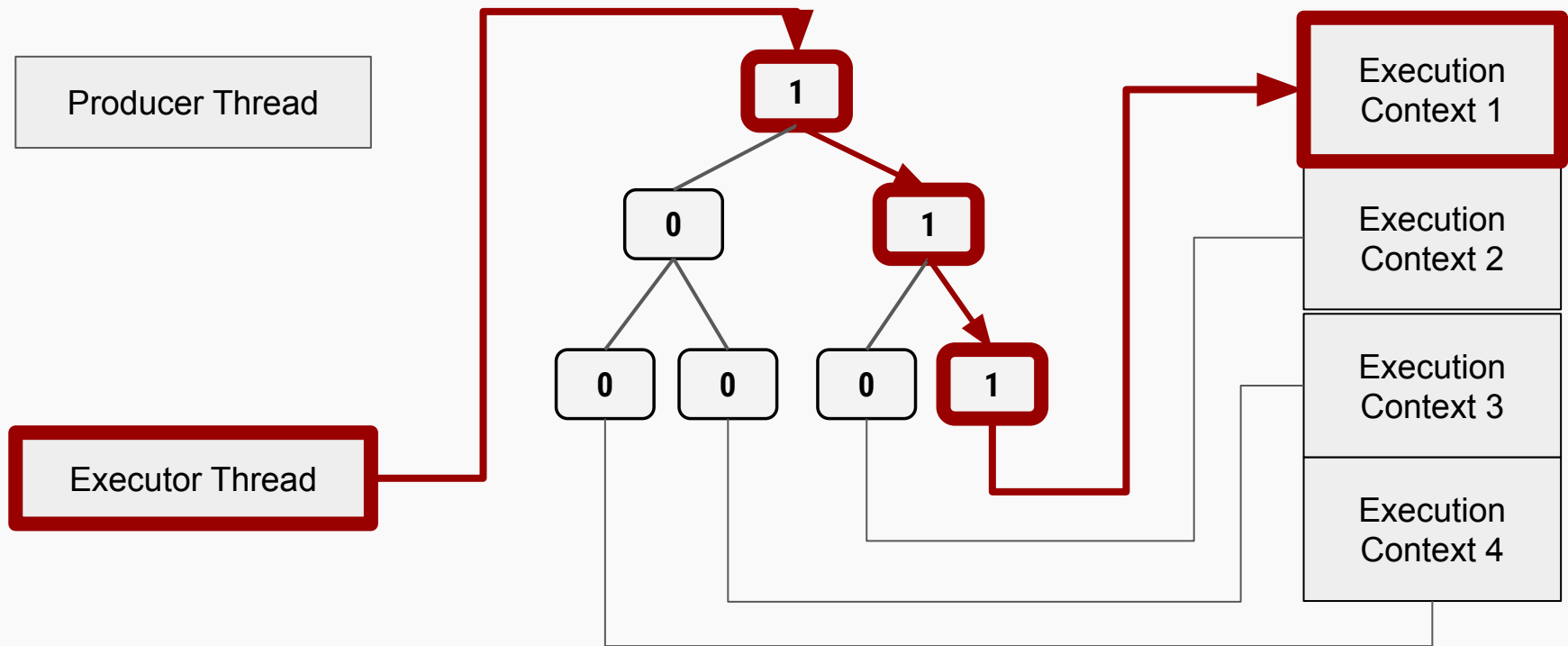
Execution Layer: Basic Execution Architecture



Execution Layer: Basic Execution Architecture



Execution Layer: Basic Execution Architecture



Execution Layer: What lives inside an execution context?

Execution Contexts may contain:

- callable/task/coroutine/functor
- mailbox or queue(s)
- local state
- batching state
- affinity/locality information
- continuation state
- priority metadata

A Signal Tree does not care what the execution context contains. It only tracks readiness.

Execution Layer: A Simple Scheduling Cycle

1. Producer updates execution context
2. Execution Context signals readiness through its associated signal
3. Worker selects ready context from Signal Tree - clearing signal
4. Worker executes context
5. Context reschedules itself if more work remains

Signal Trees do not replace execution models. They replace centralized readiness coordination.

The End

github.com/buildingcpp/signal_tree
github.com/buildingcpp/work_contract

Signal Trees

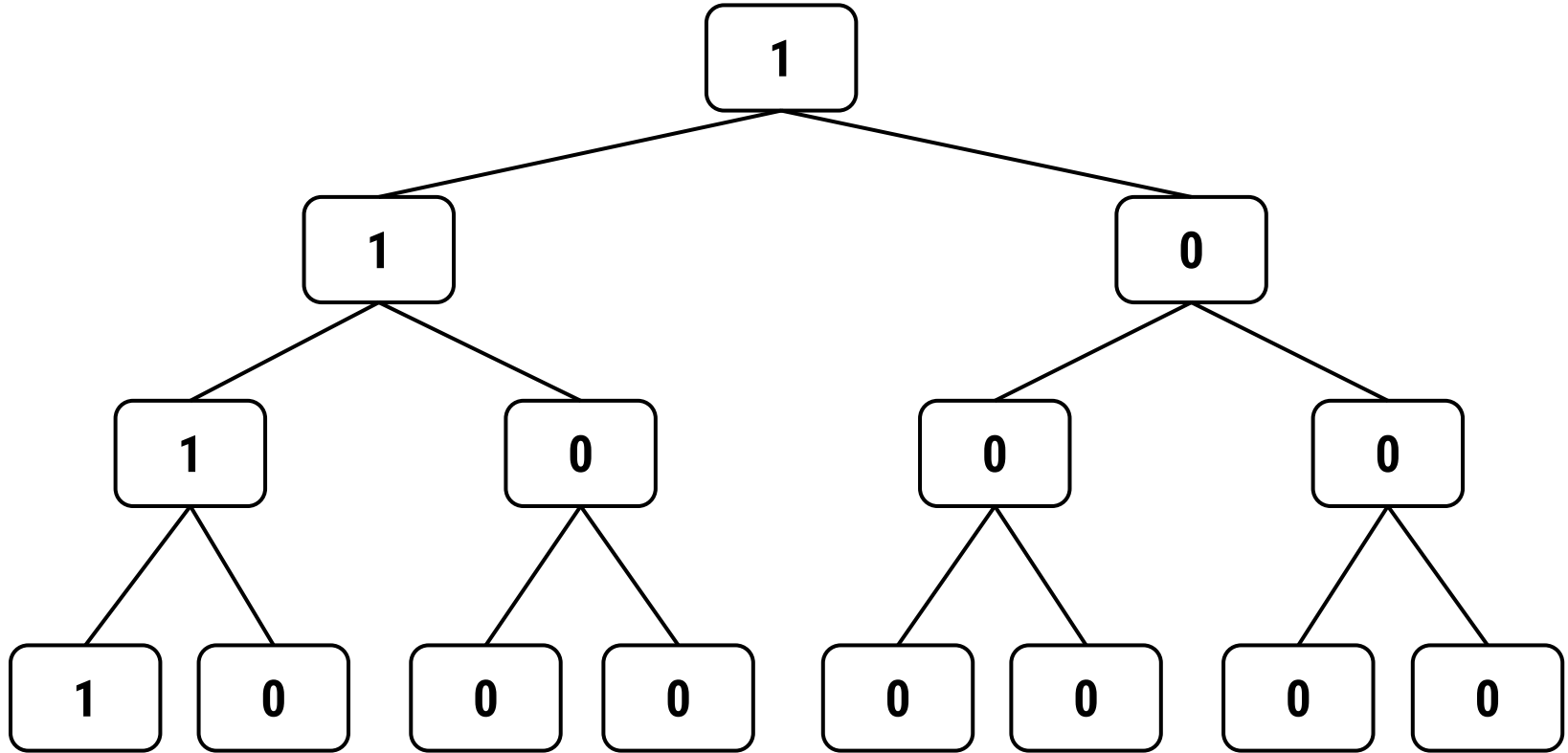
A Deep Dive into a High-Performance Alternative to Queue-Based Task Scheduling

Michael Maniscalco

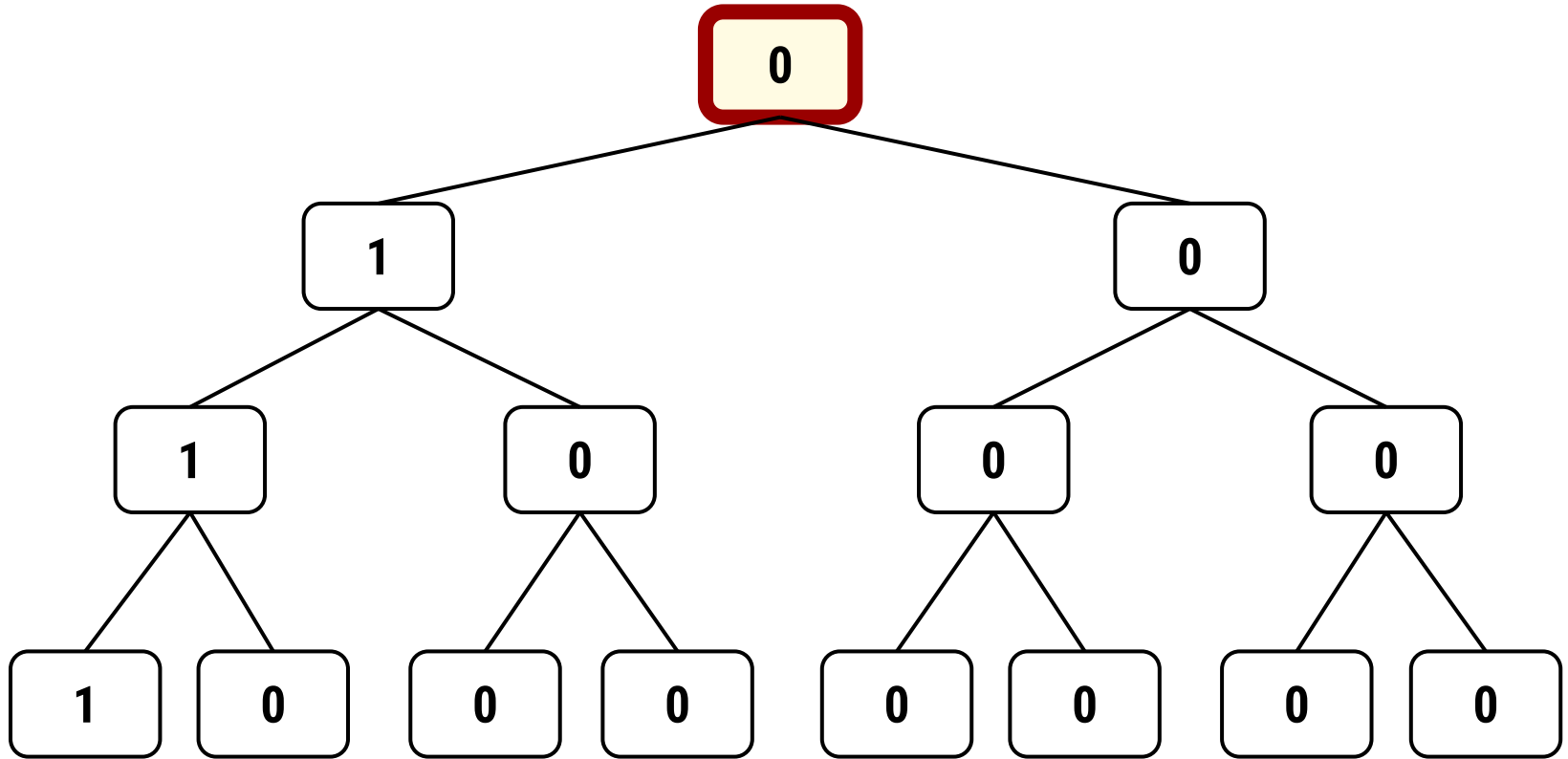
```
bcpp::signal_tree signalTree(8192);

std::uint64_t hint = 0;
for (auto i = 0; i < 8192; ++i)
{
    signalTree.set(i);
    auto signalId = signalTree.select(hint);
    if (signalId != i)
        throw std::runtime_error("oops");
}
```

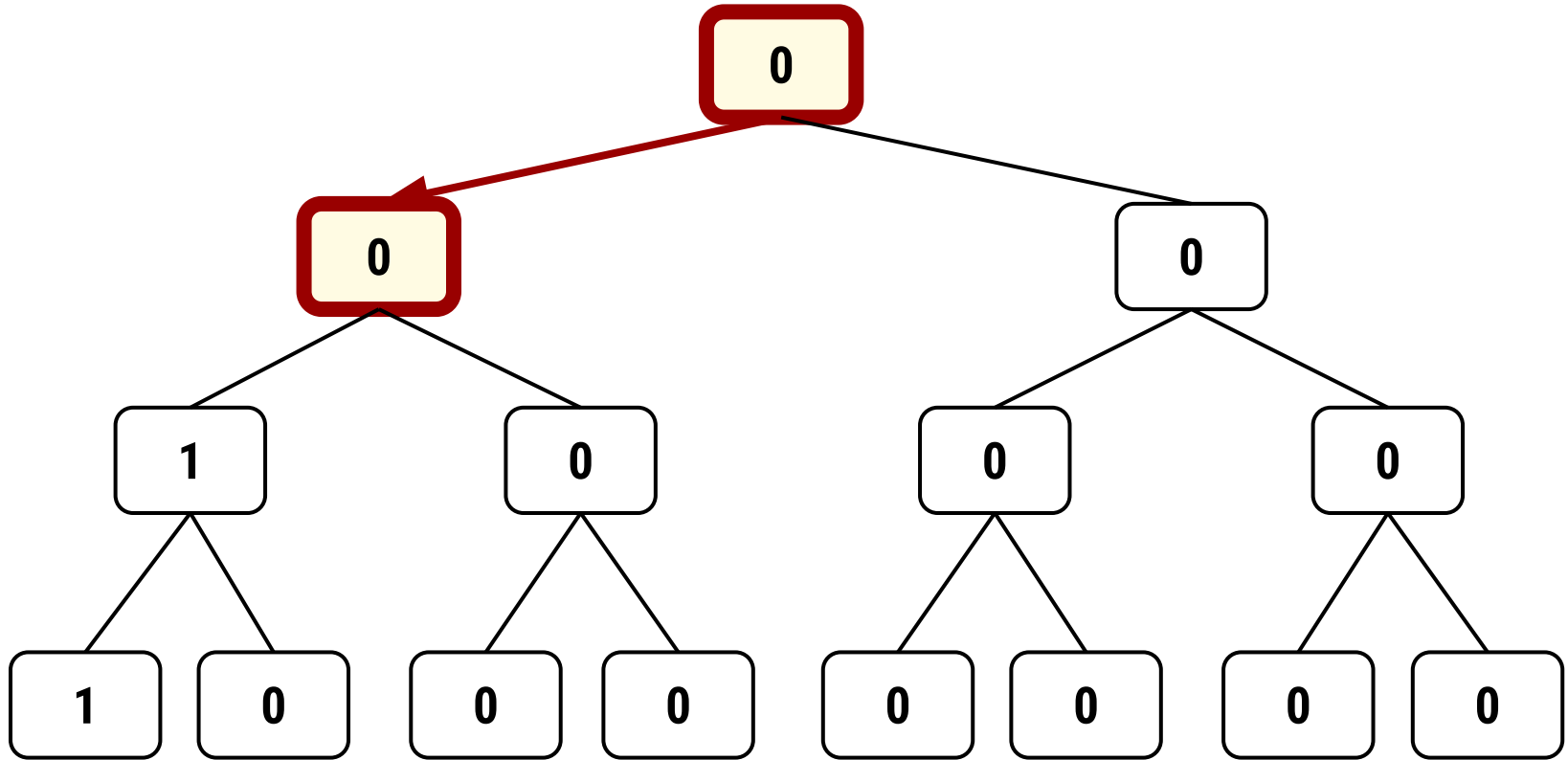
When Setter and Selector cross paths



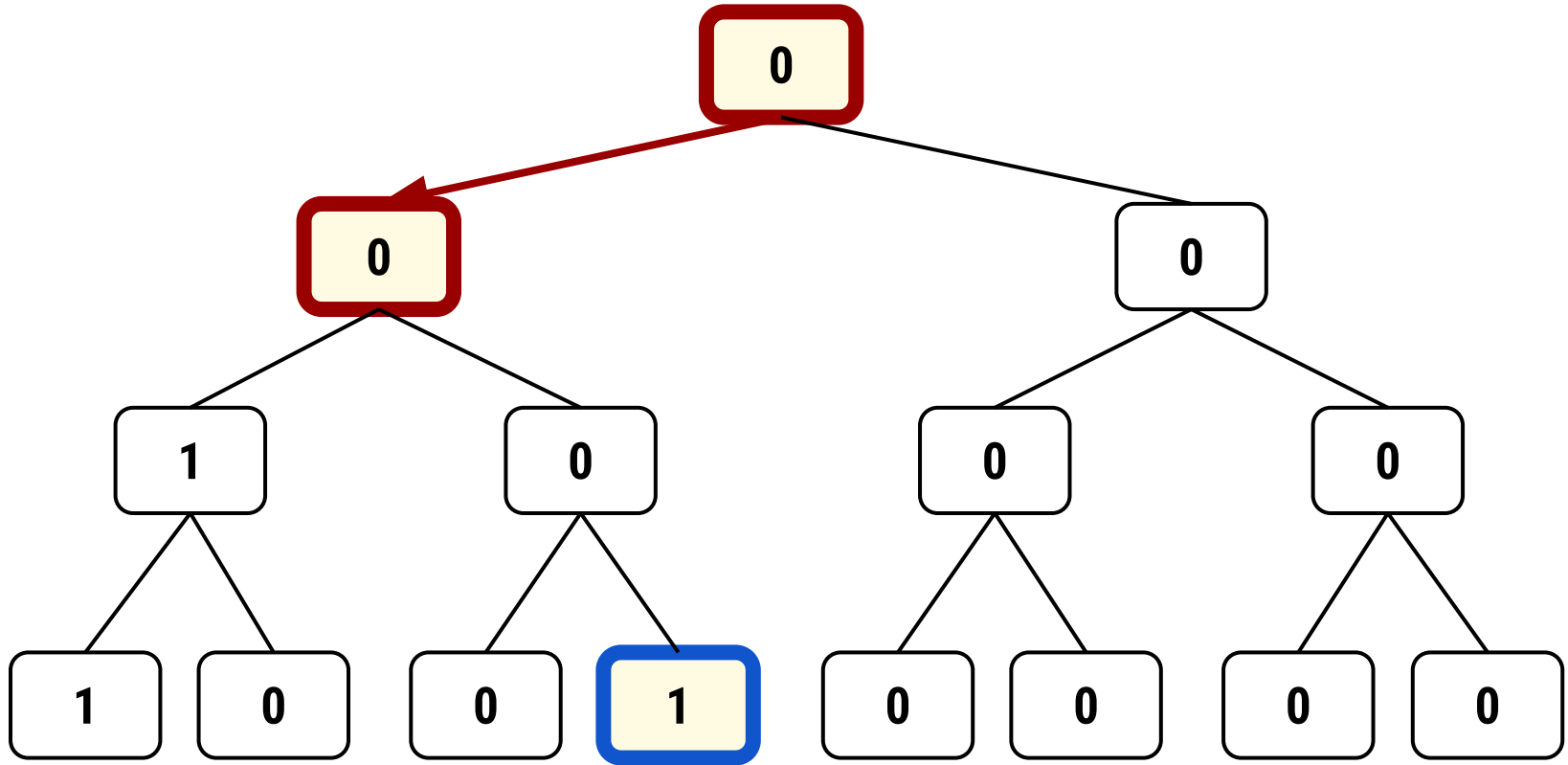
When Setter and Selector cross paths



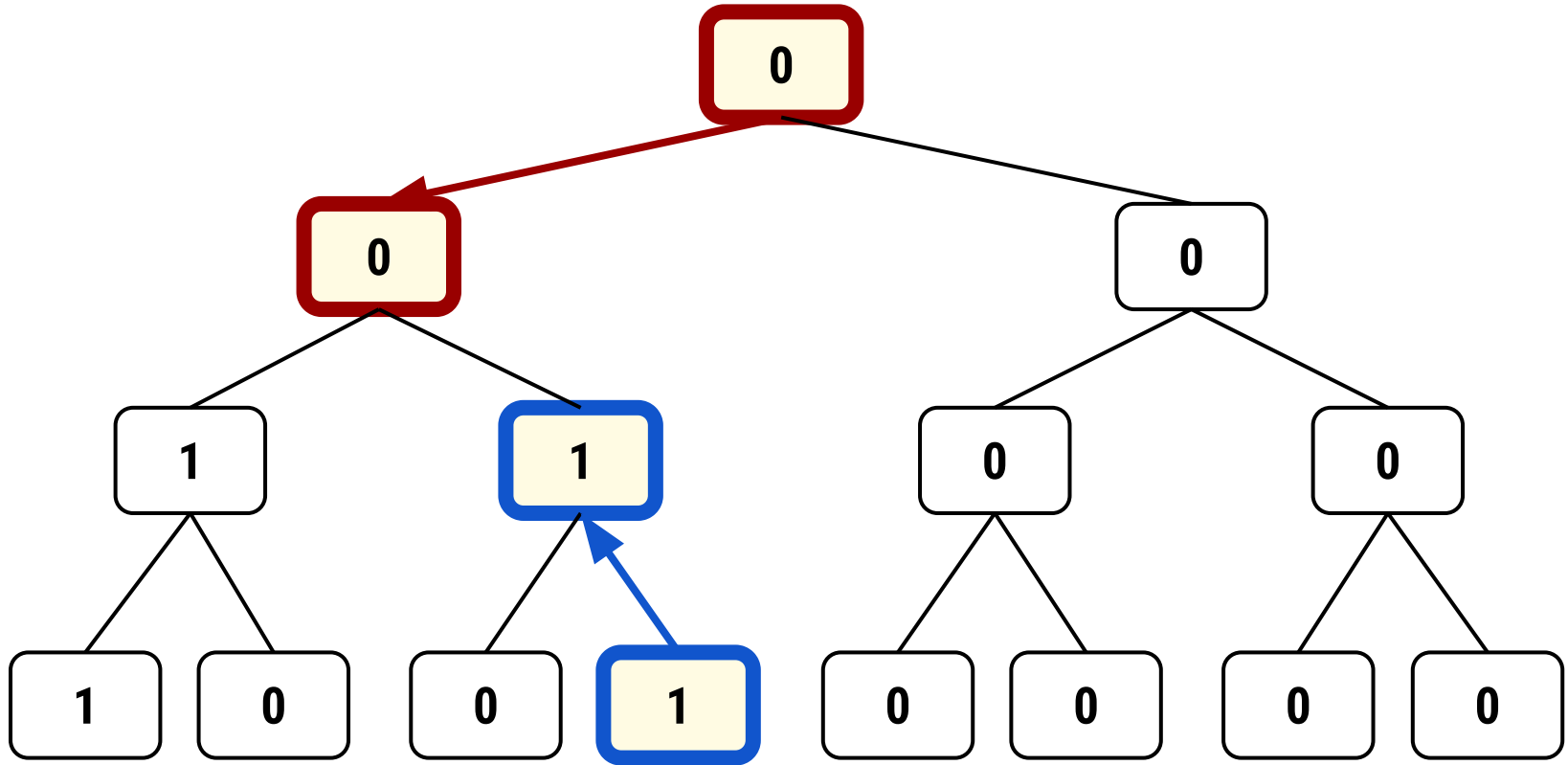
When Setter and Selector cross paths



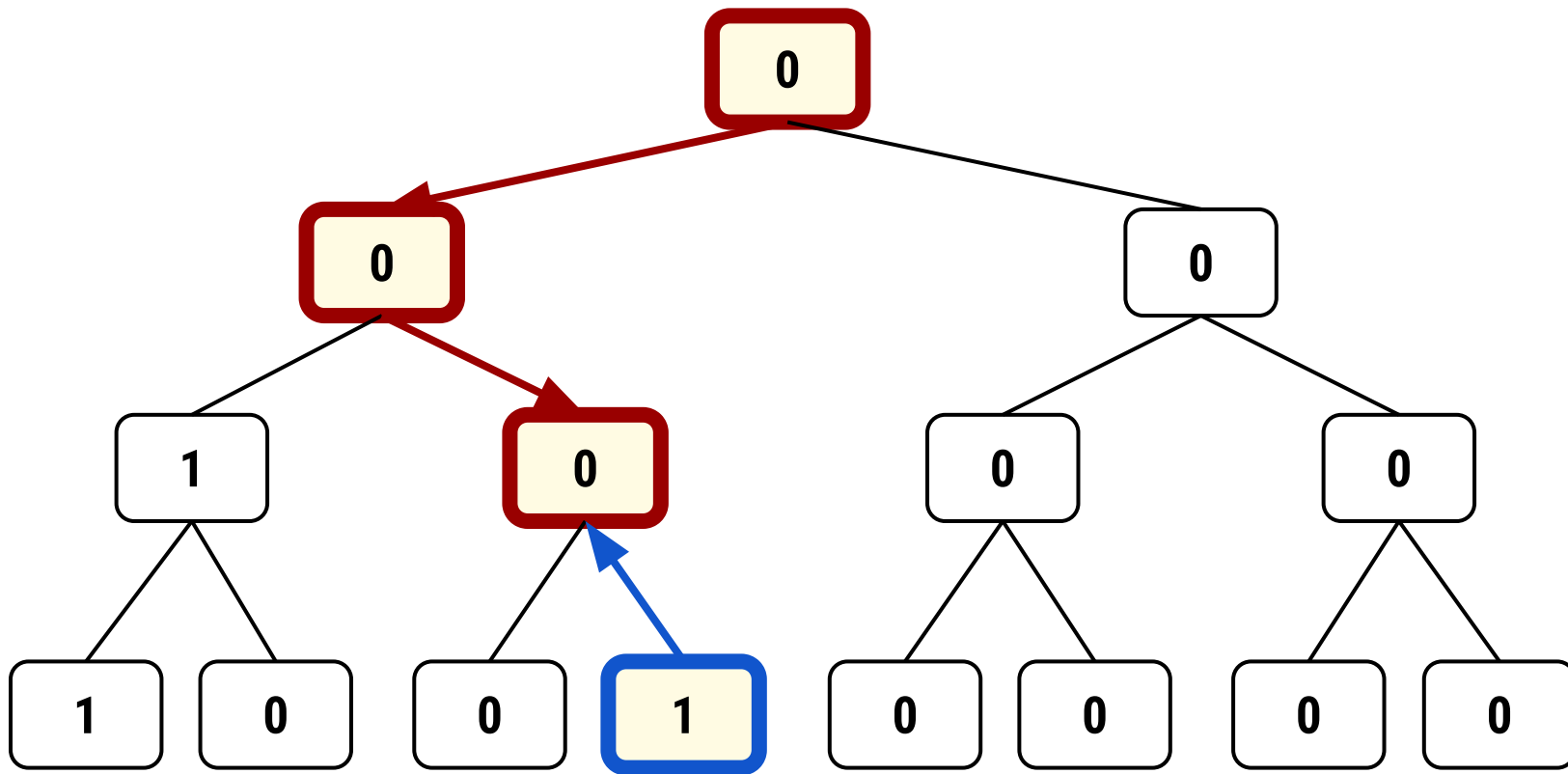
When Setter and Selector cross paths



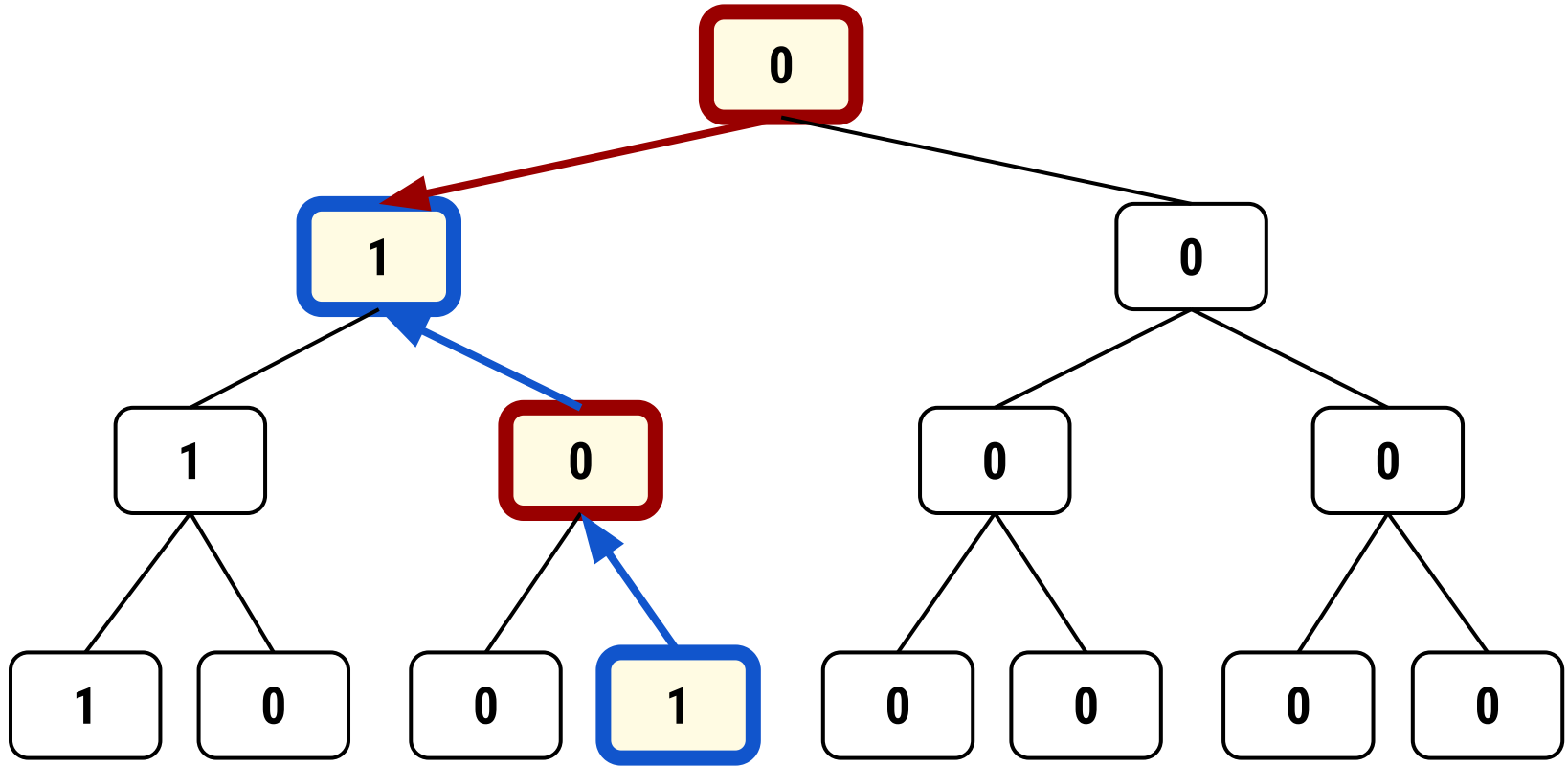
When Setter and Selector cross paths



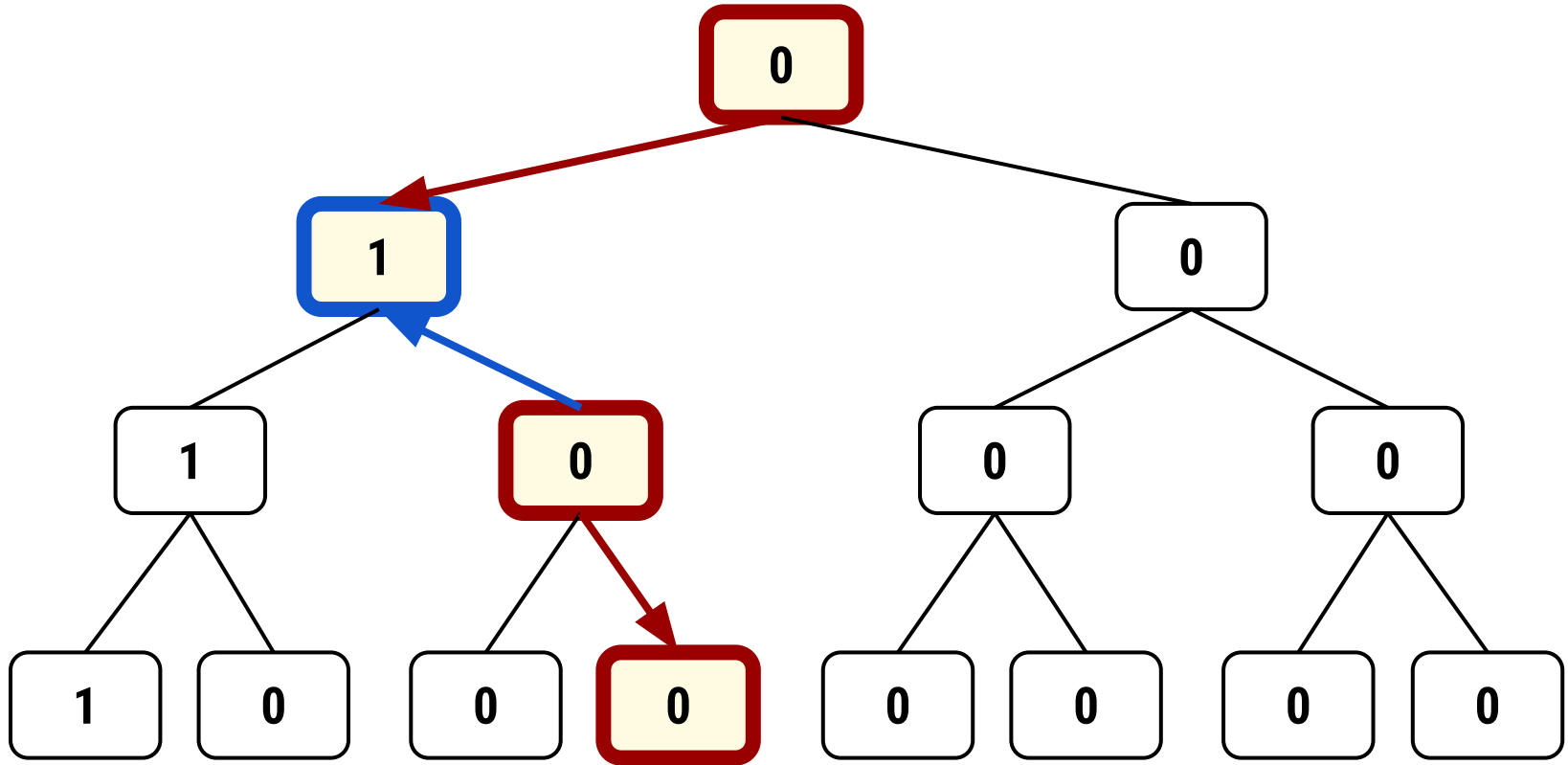
When Setter and Selector cross paths



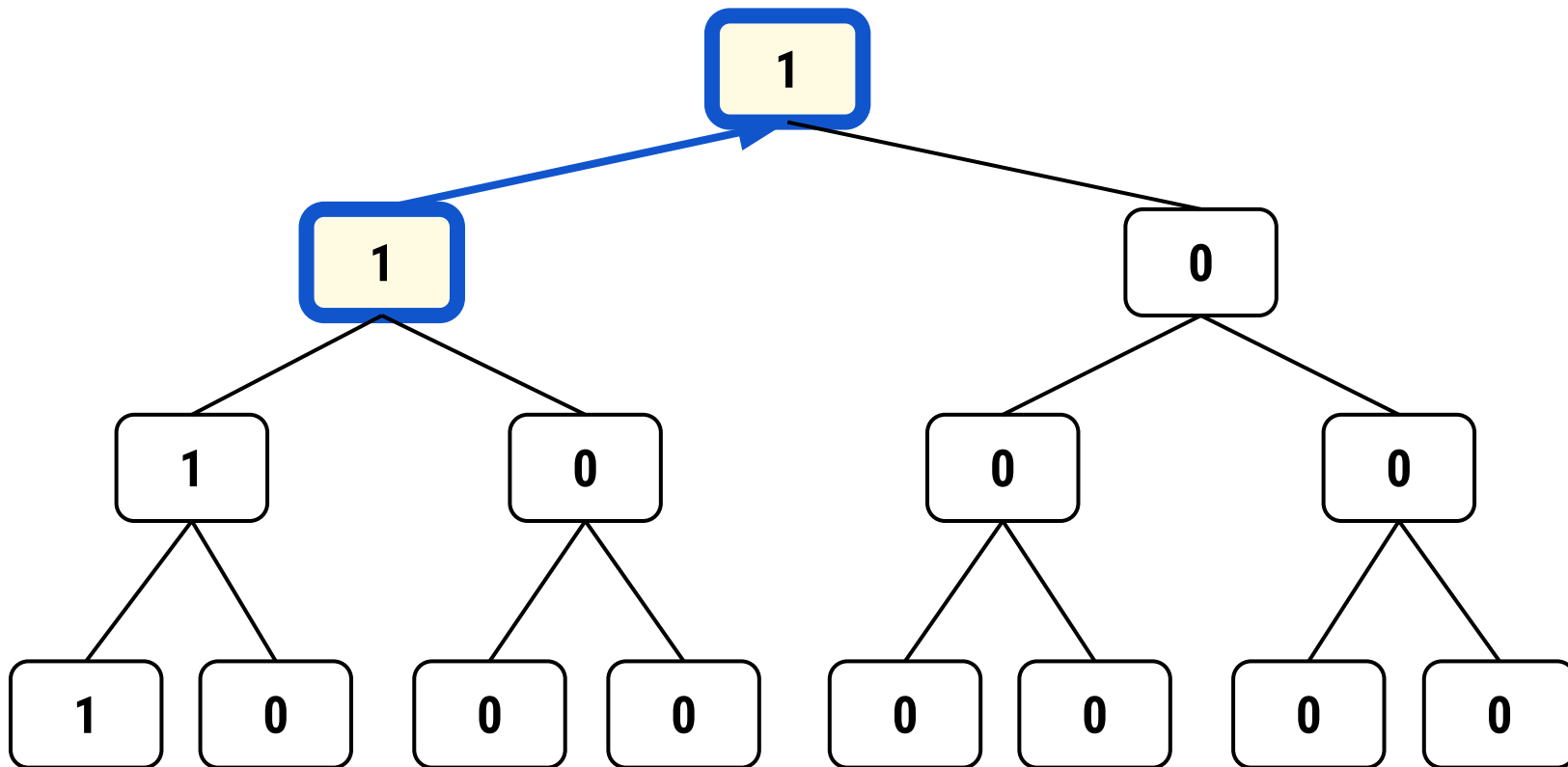
When Setter and Selector cross paths



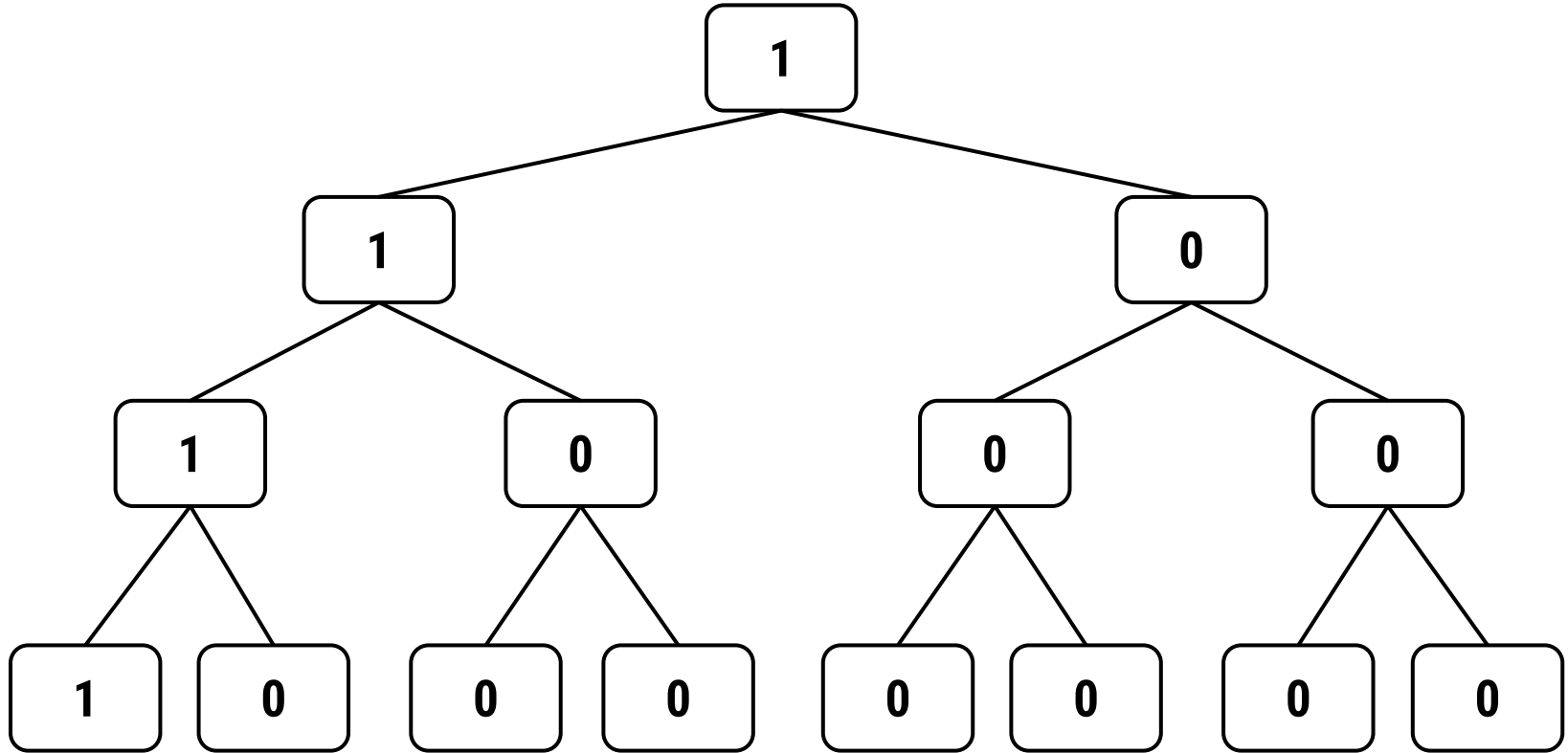
When Setter and Selector cross paths



When Setter and Selector cross paths



When Setter and Selector cross paths



Throughput million ops/sec (cumulative)

